Infor Mongoose

Replication Reference

Publication Information

Release: Infor Mongoose 9.03

Publication Date: May 23, 2017

Contents

About This Guide

This document describes Infor Mongoose replication. The various chapters explain:

- Replication concepts
- Examples of replication
- Details about which replication categories to use in rules between sites
- The forms used to set up replication
- How replication works "behind the scenes"
- How replication works with Infor10 ION
- Information on customized replication, such as using custom categories or replicating user extended tables
- Troubleshooting replication problems

Intended audience

The primary audience for this guide includes system architects and administrators tasked with implementing multi-site setups for Mongoose-based applications. Secondary audiences include technical and support personnel, training, and anyone else needing to implement replication in Mongoose-based applications.

Related documents

The primary source of related documentation is the online help for your system. In addition, you can find these documents in the product documentation section for your application from the Infor Xtreme Support portal, as described in "Contacting Infor" on page 6.

- The installation guide for your application
- The system administration guide for your application
- *Multi-Site Planning Guide*
- *Multi-Site Implementation Guide*

- *Guide to the Application Event System*

Contacting Infor

If you have questions about Infor products, go to the Infor Xtreme Support portal at http://www.infor.com/inforxtreme.

If we update this document after the product release, we will post the new version on this Web site. We recommend that you check this Web site periodically for updated documentation.

If you have comments about Infor documentation, contact documentation@infor.com.

Chapter 1: Overview

1

About Replication

Data replication is the copying of data between SyteLine site databases. The site for which data will be copied is the source, or local, site. The site receiving the copied data is the target, or remote, site.

Replication also allows the system to transmit stored procedure execution requests (SPs) between the source and target sites.

SyteLine supports two replication methods: transactional and non-transactional.

Transactional Replication

This method (also sometimes referred to as "synchronous" or "connected") performs updates on the target site, based on the transaction being performed at the source site. All changes on both the local database and the target database are committed or rolled back together. Transactional replication assumes that the source site and the target site are always connected through SQL Server and have the same database schema (same application version).

Transactional replication allows immediate updates between databases without the use of queues or XML-formatted content. Only database triggers and direct stored procedure calls are used to replicate the data.

Data errors (for example, requesting an item that does not exist at the specified remote site) are caught immediately, and the system does not allow the user to save the record with the error.

Transactional replication generally should be used only between sites on the same intranet. For best performance, both site databases should be on SQL Server machines within the same LAN, or even on the same SQL Server machine.

Non-Transactional Replication

This method (also sometimes referred to as "asynchronous" or "delayed") assumes that the source site and target site might *not* be connected through SQL Server. Non-transactional replication uses

inbound/outbound queues and XML documents to transfer data or to pass application calls (remote procedure calls, or RPCs).

The sites need not be on the same version of SyteLine; however, if you use different versions, you might need to define XSL transformations so the data arrives at the target site in a usable format for that version. Transformations are required if something significant has changed from one version of the data source to the other, or the target site has more differences than just new nullable columns.

You can also use non-transactional replication to communicate with other applications, if transformations are provided. For more information, see the document *Integrating IDOs with External Applications*.

Non-transactional replication can occur "immediately" (that is, updates are placed in the replication queue as soon as the Replicator service polling interval has elapsed), or at specified intervals (that is, updates are put in the queue when the time interval specified in the replication rule has elapsed). As soon as it reaches the queue, the data is processed by the replication services.

Data errors in non-transactional replication must be handled by the target site system. The user at the source site might not be aware that a replication-related error occurred.

Non-transactional replication populates the ShadowValues table. This and other replication-related tables grow very quickly and can impact system performance.

General Notes About Replication

You can use transactional replication between some of your sites and non-transactional replication between others; it depends primarily on how the sites are connected and how quickly you want the data to be available.

When data is updated in the target system using replication, the replication process bypasses business rule validation and triggers, so replication should only be performed between "trusted" sites. Referential integrity and database-level constraints still apply. Note also that trigger validation is not applied to tables in any case.

_All Tables

The application database includes many tables ending in "_all." These tables contain data for all sites, while the corresponding base tables contain data only for the local site. The _all tables can have only a subset of the columns from the base table—just enough information to perform local processing on other sites' data.

Example

A system has two sites, OH and MI. OH is replicating its billing terms data to MI, and MI is replicating its billing terms data to OH. The OH_App database has a terms table with columns and rows similar to this:

terms_code	description	due_days	disc_days	disc_pct	prox_day	tax_disc	cash_only	prox_code
2%	2/10 Net 30	30	10	2	0	0	0	99
2%P	2/10 Proxy 30	0	10	2	30	0	0	99
4%	4/10 Net 30	30	10	4	0	0	0	99
4%P	4/10 Proxy 30	0	10	4	30	0	0	99
5%	5/10 Net 30	30	10	5	0	0	0	99

The MI_App site database has a terms table with columns and rows similar to this:

terms_code	description	due_days	disc_days	disc_pct	prox_
COD	COD Only	0	0	0	0
N15	Net 15 Days	15	0	0	0
N30	Net 30 Days	30	0	0	0
N45	Net 45 Days	45	0	0	0
NT	No Terms	0	0	0	0
P30	Proxy Day 30	0	0	0	30

Both OH and MI also have a terms_all table with columns and rows similar to this:

site_ref	terms_code	description	tax_disc
MI	COD	COD Only	0
MI	N15	Net 15 Days	0
MI	N30	Net 30 Days	0
MI	N45	Net 45 Days	0
MI	NT	No Terms	0
MI	P30	Proxy Day 30	0
OH	2%	2/10 Net 30	0

Notice that the terms_all table has a **site_ref** column to distinguish the records for each site. It does not include all the columns from the base table - only the ones that typically would be used in a multi-site environment.

When a billing term is added at OH or MI, the _all tables at both sites are updated.

General Notes About _All Tables

_All tables at the local site are populated with local site data through database triggers when the base table is updated. The _all table can also contain remote site data, populated through replication (depending on the replication rules defined at the remote sites).

_All tables are used when the information in a base table *is not* typically shared among sites (for example, customer orders or transfers). When the information in a base table *is* typically shared among all sites (for example, customer addresses), there might be no need for an _all table. Data is replicated directly from the base table in the source site to the base table in the target site. So, such tables usually are replicated directly.

Replication Categories and Rules

The default replication categories included in Mongoose are briefly described below. More detailed information about each category is in Chapter 3, “Replication Categories."

These standard categories have been tested to ensure that they include all the necessary database tables, stored procedures, and/or XML documents required to replicate the information used in the specified Mongoose function. For example, the A/P category contains the components needed for replicating your Accounts Payable data, the G/L category contains the components needed for replicating General Ledger, and so on. The categories can be included "as-is" in your replication rules.

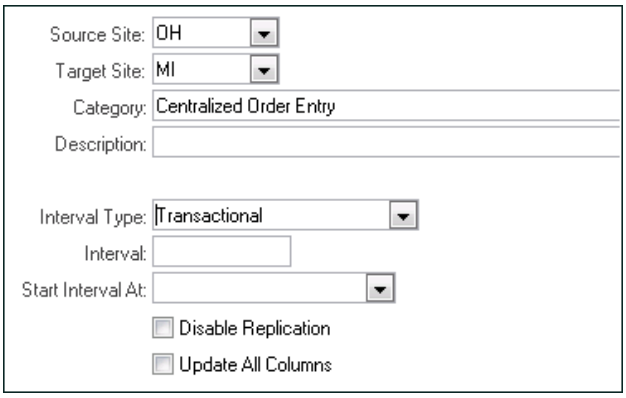
Category	Create a Source Site-to-Target Site Rule for this Category in order to...
A/P	View Accounts Payable posted transactions for the remote (target) site Distribute A/P payments to remote site A/P posted transactions Share vendor addresses to the remote site
A/R	View Accounts Receivable posted transactions for the remote site Distribute A/R payments to remote site A/R posted transactions Share customer addresses to the remote site
Centralized Order Entry	Enter order lines to be shipped from the remote site Enable global item defaulting with the remote site View available inventory for the remote site Share customer addresses to the remote site
ESB	Support the import and export of business object documents (BODs) to and from Infor10 ION
EXTFIN	Support the export of A/P, A/R, Analytical Ledger, and General Ledger posted transactions to an external financial application (the remote site is the site representing the external financial application) **CAUTION:** This category should only be used in rules for transferring data to an external financial application.
EXTFIN Customer	Support the export of customer data to an external financial application (the remote site is the site representing the external financial application) **CAUTION:** This category should only be used in rules for transferring data to an external financial application.
EXTFIN Vendor	Support the export of vendor data to an external financial application (the remote site is the site representing the external financial application) **CAUTION:** This category should only be used in rules for transferring data to an external financial application.
G/L	Support multi-site G/L reporting of data for the remote site Support site/entity relationships between the local site and the remote site (replication rules must be set up for this replication category between a site and an entity, and between lower-level entities and higher-level entities)

Category	Create a Source Site-to-Target Site Rule for this Category in order to...
Inventory/ Transfers	View available inventory for the remote site Support the setup of inter-site parameters between the local site and the remote site Enable multi-site transfer functionality with the remote site Enable global item defaulting with the remote site
Journal Builder	Enter pending multi-site journal entries at the local site and validate the information from the remote site.
Ledger Consolidation	Pass G/L data needed only for ledger consolidation. This is a subset of the G/L category that does not include journal and ledger tables; if you are already replicating the G/L category, do not replicate this one.
Ledger Detail	Support viewing ledger detail for transaction data stored at the remote site.
Manufacturer Item	Share manufacturer item information to the remote site. Share cross-reference information between manufacturer item records and Mongoose item records to the remote site.
Multi-Site CRM	Support Multi-Site CRM information on the Salesperson Home form.
Multi-Site Customers	Support use of the Multi-Site Customers form from a master site.
Multi-Site Items	Support use of the Multi-Site Items form from a master site.
Planning	Pass Planned Supply created by the (single-site) Planner to the supply site as planned transfer demand - for items flagged as Transferred and containing a Supply Site.
PO-CO Across Sites	Support the ability for demand from one site to be satisfied by another site in a purchase order - customer order relationship.
Purchase Order Builder	Enter builder purchase orders at the local site and validate the information from the remote site.
Shared Currency	Share currency and currency rate data with the remote site.
Site Admin	Support setup of inter-site parameters Share site/entity data, including intranet data and site group data, with the remote site.
Voucher Builder	Create vouchers and adjustments in other sites based on information from those sites.

How Categories Are Used with Rules

In order to replicate these categories to other sites, you must create one replication rule for each source-target site combination and category. For example, if you want to replicate order entry data

from site OH to site MI, and you want to use transactional replication, create a rule in the **Replication Rules** form on the OH site:



Once the rule is defined and saved, click **Regenerate Replication Triggers** on the **Replication Management** form. This regenerates the table triggers so that the data is replicated according to the specifications in the category and rule.

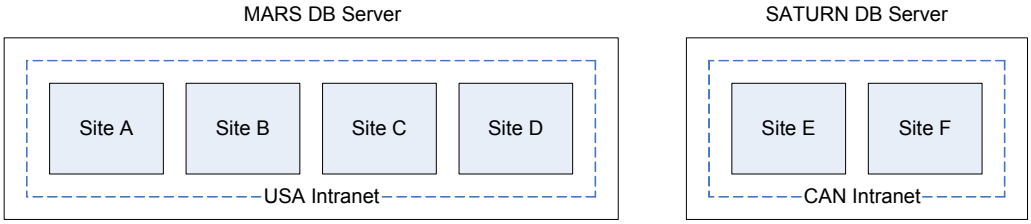
In certain cases, replication rules and categories determine which sites are available when performing Mongoose functions. For example, if there is no replication rule set up for **Centralized Order Entry** from the local site to any other site (or if the rules are disabled), then, when a user accesses the **Customer Order Lines** form at the local site, the form's **Ship Site** field is disabled and the order must be shipped from the local site. If there is at least one enabled replication rule for **Centralized Order Entry**, the **Ship Site** field is enabled and lists all available sites. When the user then selects a ship site, the system validates that there is an enabled replication rule for **Centralized Order Entry** from the local site to the selected remote site. If not, an error message is displayed.

For most other forms that specify a remote site, the system validates a record against the _all tables to determine whether the record can be saved. For example, assume you have a transactional replication rule being used to replicate the Inventory/Transfers category between sites. A person creating a transfer order line at the local site selects a remote site from which the item will be transferred, and then tries to save the record. The system checks the item_all table to determine whether the selected remote site database contains the specified item. If not, the record validation displays an error.

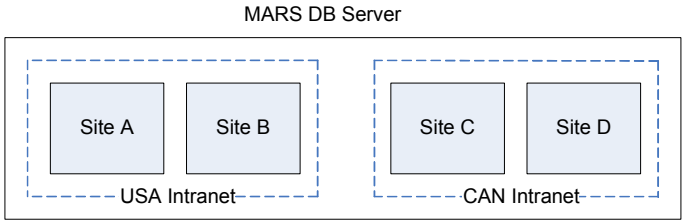
Intranets

When you configure a site, you assign it to a logical intranet (on the **Sites** form). Intranets are used to segregate a company's sites into virtual groupings that reflect the actual network setup.

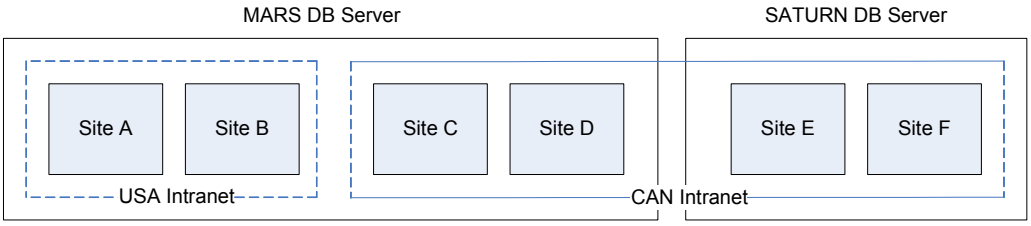
Usually, all the sites on the same LAN and/or database server will belong to the same intranet, and usually sites on different database servers are in different intranets.



It is possible to set up sites on the same database server that are in different intranets.



It is also possible to set up one intranet with sites on different servers.



Generally, transactional replication should be performed only between sites on the same database server, for performance reasons. However, if your database servers are on a very fast network, transactional replication between sites on different servers is possible.

When a system includes multiple intranets on different servers, the data generally should be transferred between intranets through non-transactional replication (XML documents). When the schema in the source and target databases are not identical—for example, SyteLine to another application that uses the Mongoose Framework, or different versions of your application—an XSL transformation (style sheet) should be used. Each intranet has an ASP page that provides intranet access to the inbound queue. The ASP is the gateway to other databases. The address of the ASP is defined on the **Intranets** form.

Transport Protocol

Non-transactional replication between sites on *different* intranets generally uses HTTP POST (internet) communication. Non-transactional replication between sites on the *same* intranet does not use HTTP POST; instead, it is handled directly by one utility server and its MSMQ infrastructure.

When SyteLine communicates with other applications using ION, the transport protocol is ESB. The protocol is specified on the **Intranets** form.

Master Sites and Shared Tables

If your environment has many sites, large amounts of shared data, and many users, you might want to set up one site as the *master site* for an intranet. Master sites are database sites that control some data for all other database sites on an intranet.

If you use a master site, certain tables can reside *only* on the master site database and are shared (read and written to through a SQL view) by other sites on the same intranet. No replication needs to occur for the shared tables, which can greatly improve system performance and simplify the setting up of replication rules.

After you set up a master site for an intranet and share tables between the sites on the intranet, the intranet is considered a "shared table" intranet. You can add new sites to a shared table intranet. Removing sites from a shared table intranet is more complex, because you must first unshare the shared tables (that is, recreate them) at the site being removed from the intranet. For more information about sharing and unsharing _all and user tables, see the online help.

For more information about setting up master sites and shared tables, see the online help and the *Multi-Site Implementation Guide*.

Chapter 2: Replication Examples

2

This chapter contains examples that may help you better understand how replication works.

Note: In this example, Infor10 ERP Business (SyteLine) is the application being used. The same principles, however, apply to other applications built on a Mongoose framework.

Transactional - Adding a New Site Group

Let's start with a simple example showing the steps and the forms used to set up replication between sites, as well as the data being replicated. Later examples only show the data being replicated.

In this example, your company has sites OH and MI. The sites are defined on the same intranet, and their databases are on the same SQL server. When OH creates a new site group, you want the new site group to also appear in MI. To do this, you will create a transactional replication rule.

Currently the **Site Groups** form, which mirrors the site_groups table, at OH contains this group:

The **Site Groups** form at MI contains this group:

Set Up Replication

Make Sure the Sites Are Linked

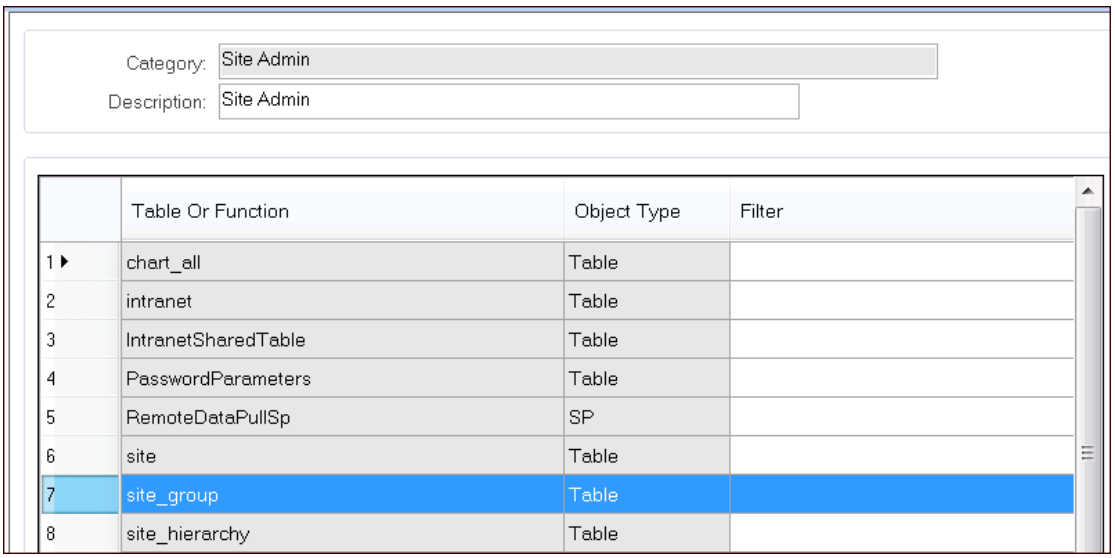
In the **Sites** form on the OH site, make sure the OH and MI sites are linked:

If they are not linked, see the *Multi-Site Implementation Guide* for information about how to set up linking.

Find the Category

First, you must determine which replication category contains the table you want to affect. In this case, the site groups are stored in the site_group table.

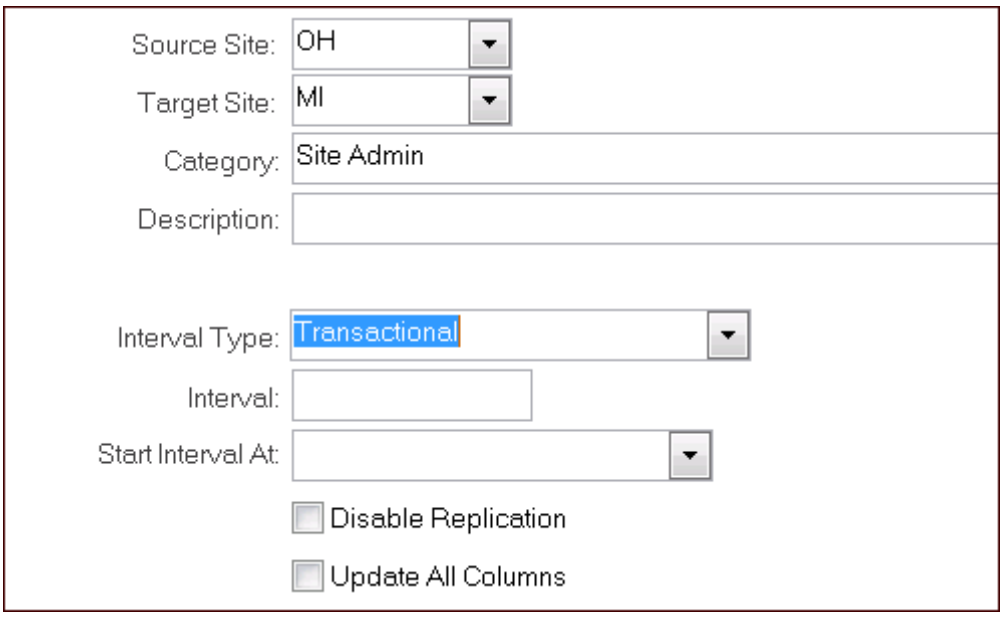
In the **Replication Categories** form, you can see that the Site Admin category contains the site_group table:



The site_group table is a base table—meaning that it is not an _all table. When a base table is replicated, the rows in the table are the same at the source and target sites. (In an _all table there are separate rows for different sites, as shown on page 9.)

Add a Rule

In the **Replication Rules** form on the OH site, create a rule from OH to MI that replicates the **Site Admin** category:



Regenerate the Triggers

Then, to reset the table triggers on the OH database, on the **Replication Management** form, click **Regenerate Replication Triggers**.

Add a New Site Group at OH

Now that your replication rule is set up and triggers are regenerated, you are ready to add the new site group FIN at OH. It has two member records, OH and MI.

Site Groups

	Multi-Site Group	Site	Site Name
1	OH	OH	Ohio
2	FIN	OH	Ohio
3	FIN	MI	Michigan

Check the Site Groups at MI

The new site group record should immediately appear in the MI site, after a form refresh:

Site Groups

	Multi-Site Group	Site	Site Name
1	MI	MI	Michigan
2	FIN	OH	Ohio
3	FIN	MI	Michigan

Notice that the original OH Site Group record was not replicated. Existing records at the source site are not replicated; only newly added or updated records are replicated.

Transactional – Adding an Order

Our next example is more complex.

A company has two sites, CAL and GA. The sites are defined on the same intranet and their databases are on the same SQL server. They are set up as "linked" in the **Sites** form.

A transactional replication rule is set up from CAL to GA for the **Centralized Order Entry** category, which contains the customer order _all (co_all) table and customer order line _all (coitem_all) table, along with many other tables and stored procedures.

The following data transfer occurs:

1 A user in the CAL site creates an order with one line using Ship Site CAL. The data is saved to the co and coitem tables. This data is then copied to the co_all and coitem_all tables in the CAL site, through triggers in the co and coitem tables.

Site	co Table	coitem Table	co_all Table	coitem_all Table
CAL	Co_Num=A0000001 Orig_Site=CAL	Line1 = Item X Ship_Site=CAL	Co_Num=A0000001 Orig_Site=CAL Site_Ref=CAL	Line1 = Item X Ship_Site=CAL Site_Ref=CAL
GA				

2 As soon as the _all tables are updated in the CAL site, triggers on the _all table in the CAL table run, and they execute a stored procedure to populate _all from the CAL database to the GA database. The base tables are not updated in the GA database.

Site	co Table	coitem Table	co_all Table	coitem_all Table
CAL	Co_Num=A0000001 Orig_Site=CAL	Line1 = Item X Ship_Site=CAL	Co_Num=A0000001 Orig_Site=CAL Site_Ref=CAL	Line1 = Item X Ship_Site=CAL Site_Ref=CAL
GA			Co_Num=A0000001 Orig_Site=CAL Site_Ref=CAL	Line1 = Item X Ship_Site=CAL Site_Ref=CAL

3 The user in the CAL site adds a second order line, using **Ship Site GA**. The data is saved to the CAL site's coitem table. The system then calls remote methods in the CAL site to replicate in the GA site the following data:

- The customer (if not found in the GA site)
- The order from the co table
- The order line (line 2 only) from the coitem table

This data is copied to the coinciding _All tables in both sites through trigger code in the co and coitem tables. This time, the base tables are updated in the GA database because that site will be shipping order line 2.

Site	co Table	coitem Table	co_all Table	coitem_all Table
CAL	Co_Num=A0000001 Orig_Site=CAL	Line1 = Item X Ship_Site=CAL Line2 = Item Y Ship_Site=GA	Co_Num=A0000001 Orig_Site=CAL Site_Ref=CAL	Line1 = Item X Ship_Site=CAL Site_Ref=CAL Line2 = Item Y Ship_Site=GA Site_Ref = GA Line2 = Item Y Ship_Site=GA Site_Ref=CAL
GA	Co_Num=A0000001 Orig_Site=CAL	Line2 = Item Y Ship_Site=GA	Co_Num=A0000001 Orig_Site=CAL Site_Ref=CAL Co_Num=A0000001 Orig_Site=CAL Site_Ref=GA	Line1 = Item X Ship_Site=CAL Site_Ref=CAL Line2 = Item Y Ship_Site=GA Site_Ref = GA

4 The system replicates the data through trigger code in the coitem_all table to both sites.

Site	co Table	coitem Table	co_all Table	coitem_all Table
CAL	Co_Num=A0000001 Orig_Site=CAL	Line1 = Item X Ship_Site=CAL Line2 = Item Y Ship_Site=GA	Co_Num=A0000001 Orig_Site=CAL Site_Ref=CAL Co_Num=A0000001 Orig_Site=CAL Site_Ref=GA	Line1 = Item X Ship_Site=CAL Site_Ref=CAL Line2 = Item Y Ship_Site=GA Site_Ref = GA Line2 = Item Y Ship_Site=GA Site_Ref=CAL
GA	Co_Num=A0000001 Orig_Site=CAL	Line2 = Item Y Ship_Site=GA	Co_Num=A0000001 Orig_Site=CAL Site_Ref=CAL Co_Num=A0000001 Orig_Site=CAL Site_Ref=GA	Line1 = Item X Ship_Site=CAL Site_Ref=CAL Line2 = Item Y Ship_Site=GA Site_Ref = GA Line2 = Item Y Ship_Site=GA Site_Ref=CAL

Note that, in this example, if there had been a validation error anywhere along the way, the database transaction would have been rolled back, undoing all changes at both sites. However, in this case the update was successful, so the whole transaction is committed, including updates to both sites.

Non-Transactional - Adding an Order

A company has three sites: OHIO, MICH, and CALIF. OHIO and MICH are on the same intranet, MIDWEST; CALIF is on a separate intranet, PACIFIC. A non-transactional replication rule exists from OHIO to target site CALIF for the Centralized Order Entry category, with an interval of "Immediate." A similar rule exists from OHIO to target site MICH.

An OHIO user updates and saves an order line record in the OHIO database. When the record is saved, a SQL trigger copies the data to the coitem_all table. (The coitem_all table is included in the Centralized Order Entry category.)

Since order entry data is to be shared between sites, the system defined a replication trigger on the coitem_all table. When the order line data is copied to the coitem_all table in OHIO, the replication trigger fires. The replication trigger’s job is to take the updated data and initiate the process of sending that data to the sites that need it. MICH and CALIF both need the order line update. The replication trigger writes the updated data to a set of staging tables; separate rows are created for the data that needs to go to the MICH site and the data that needs to go to the CALIF site.

On the utility server, a set of *replication services* run. These are processes (which run as Windows services) that are responsible for packaging/delivering and receiving/processing replication updates.

Each intranet has its own utility server, and the replication services on that utility server handle the replication processing for all sites within that intranet.

There are two replication services:

- **Infor Framework Replicator** is responsible for polling its intranet's sites' replication staging tables and packaging the data updates found in those tables into XML documents. These XML documents are then forwarded to the second service, Replication Queue Listener.
- **Infor Framework Replication Queue Listener** has two jobs: delivering the replication XML documents to remote intranets, and processing incoming replication XML documents from remote intranets. The Replication Queue Listener has two separate queues, one for outgoing data deliveries and one for incoming data.

If the replication XML document is to be delivered to a site on a remote intranet, Replicator will drop the document in Replication Queue Listener's outgoing queue; if the document is to be delivered to a site on the local intranet, Replicator will drop the document in Replication Queue Listener's incoming queue. In our example, the replication update to CALIF will be placed in the MIDWEST Replication Queue Listener's *outgoing* queue, and the replication update to MICH will be placed in the MIDWEST Replication Queue Listener's *incoming* queue.

Delivery of replication XML documents to the remote PACIFIC intranet by the Replication Queue Listener is performed via HTTP. The replication XML document is delivered to the PACIFIC intranet's URL (as defined in the data found in the OHIO site's **Intranets** form) via an HTTP POST operation. The ASP page that receives this POST on the PACIFIC intranet places the posted XML document into the PACIFIC Replication Queue Listener's incoming queue. The PACIFIC Replication Queue Listener picks up the XML document and imports the incoming coitem_all data into the CALIF site's database.

The MIDWEST intranet's Replication Queue Listener also accepts the incoming coitem_all update and creates or updates the record in the MICH site's database.

For more information about the services and queues mentioned in this example, see Chapter 4, "Replication Behind the Scenes."

For more information about the XML documents used in replication, see *Integrating IDOs with External Applications*.

Master Site - Sharing Product Codes

A company has 30 sites on one intranet, including MI and OH. OH has been designated as the master site for the intranet. The system administrator has set up the **Intranet Shared Tables** form so that the _all tables associated with the Inventory/Transfers category are shared. One of these shared tables is prodcode_all.

A user at MI creates and saves a new product code record. (Although the MI site database is set up with a prodcode_all view instead of a table, users still have the same record creation and deletion abilities they would have if the table was local.) Since there is no prodcode_all table at MI, the local (MI) prodcode table has a view that forwards the data directly to a table at OH.

The shared information for the new product code is then available at all sites on the intranet. No replication of product codes occurs in this scenario.

Chapter 3: Forms Used in Replication

3

Use the forms described in this chapter to set up replication. This chapter focuses on the information needed for the replication fields on these forms, and how that information is used by the system.

System and Site Forms

The forms in this group are used to set up your system for replication.

Intranets Form

Use the **Intranets** form to define the intranets that will group sites coexisting on a high speed network. For example, if you are running sites on the same LAN, define them in SyteLine to use the same intranet so that you can use transactional replication. Groups of databases should not be defined in the same intranet if they are not on the same version of your application, because transactional replication does not work between different versions.

You define the intranets on this form and then use the **Sites** form (page 26) to specify which intranet each site uses.

Fields Used for Non-Transactional Replication

If you are using non-transactional replication, use these fields on the **Intranets** form to define connection information that is used in data requests to and from other sites:

- **External**
- **Transport**

This connection information is not necessary for transactional replication.

For more information on these fields, see the online help.

HTTP Fields

- **Queue Server** – Enter the name of the MSMQ server for the intranet. If left blank, this field defaults to the utility server running this application, and the private queue area. If you have multiple intranets on different servers, include the utility server machine name in the path (for example, *machinename*\PRIVATE$).
- **Private Queue** – We recommend that you select this field, since queues in non-transactional replication are used only by Mongoose Framework services on the same machine. Setting up public MSMQ instances requires further ActiveDirectory setup, which is not required by the Mongoose Framework services.
- **URL** – This is the Web address to which the XML request/response documents will be posted. When replication occurs between sites on different intranets, XML documents are routed to this URL for the target site. It can also be used by system integrators for programmatic access to a site on this intranet.

  This URL could be an ASP page on the external system that receives and processes the XML from the message queue. The processing done at this URL is up to you; for example, an ASP page might map the data into the proper format for the external system, or it might write the XML documents to a location on its server for later processing.

  In many cases, this field provides the address of the default InboundQueue.asp in the virtual directory that was installed with the web server components.
- **Direct IDO URL** – This is the Web address of the synchronous XML interface to this intranet. It is used primarily by application clients which are configured to connect via the Internet. It can also be used by system integrators for programmatic access to a site on this intranet.

  If you are using this field for Mongoose Framework access, it should contain the address of the Configuration Server, which is deployed on your utility server when you include the Web server feature of the utility server setup. For example, if your machine name is UTILSERVER, then the Direct IDO URL might look like: **http://UTILSERVER/IDORequestService/ConfigServer.aspx**

  Or, if you want to use secured HTTP—a good idea if this server will be open to the internet for HTTP traffic—use the format: **https://UTILSERVER/IDORequestService/ConfigServer.aspx**

  ConfigServer.aspx processes only GetConfigurations requests. As part of the response, it returns a URL to the RequestServer.aspx. That URL is where all additional requests should be sent. For external programs integrating by means of IDORequest XMLs to a single utility server, the caller could skip the GetConfigurations call and post requests directly to the RequestServer. If you are using this field for that case, you could specify the address of the RequestServer.aspx here; for example: **http://UTILSERVER/IDORequestService/RequestServer.aspx**

Master Site

If the selected intranet is to contain shared _all or user tables, specify the master site where the tables are to reside. To add or change a value in this field:

- You must be logged into the site that you want to specify as the master site.

- This site must be defined as being within the selected intranet. Live links must be set up (on the **Sites** form) between this site and the other sites in the intranet.
- No table within the intranet is currently shared.

After you define the master site for an intranet on the master site's **Intranets** form, the master site name displays, but cannot be changed, on the **Intranets** form for all other sites in that intranet. (The master site name displays at the other sites only after you set up replication of the Site Admin category from the master site to the other sites in the intranet.)

System Types Form

System types are used mostly with non-transactional replication. Define your system types on this form. The system type should distinguish the type or version of application running on the sites to which it applies. For example, if you have several sites running different versions of your ERP application and another site running a field service application that will be integrated with the ERP application, you might want to have system types like these: ERP705, ERP800, or FieldService.

How System Types Are Used in XSL Transformation Files

The system type is used by the replication infrastructure to provide XSL transformations (if required) on data being replicated between sites. When defining a site on the **Sites** form, you can specify the site's system type.

During non-transactional replication, SyteLine looks in the source site's utility server for a file named

[*source_site_system_type*][*object_name*][*target_site_system_type*].xsl

where:

- *source_site_system_type* is the system type specified in the **Sites** form for the site where the data originated.
- *target_site_system_type* is the system type specified in the **Sites** form for the site to which the data will be imported.
- *object_name* is usually the name of an object in a replication category.

This XSL file resides on the utility server in an XSL subfolder under the replication directory. The replication directory is specified in the Service Configuration Manager’s **Replication** tab. If the source site system type and the target site system type are the same, the system does not look for a transformation file to apply.

(XSL files used with objects in the EXTFIN replication category do not follow this naming convention exactly; the object_name part of their XSL filenames must match the hard-coded name in the XML's UpdateCollection request.)

Example

Replication for an application creates an XML document that replicates Chart of Accounts (chart_all) data from a source site whose system type is set to "APP800" to a target site whose system type is "BrandX". If an XSL file exists with a name matching the current source-object-target replication process (that is, a file named APP800chart_allBrandX.xsl), the XML data is transformed using that XSL file before being sent to the target site. If there is no XSL file with that name, the data is not transformed.

Sites Form

Use the **Sites** form to define your sites and to determine the relationships between the sites. This is also where you would disable replication temporarily, if necessary.

When you first open the **Sites** form at a site, you should see a record for each site that was defined as being linked with the local site through the **Link Multi-Site Databases** option of the **SyteLine Configuration Wizard**. You can add records for other sites as needed.

System Info Tab

These fields on the **System Info** tab are used in replication:

- **System Type** – Used in non-transactional replication. See “System Types Form” on page 25.
- **Intranet Name** – If you are using transactional replication rules for sites, the sites generally should be on the same intranet.
- **Message Bus Logical ID** – Enter the string that the Infor10 ION system is to use as the logical ID, the identifier for the site. This is the identifier by which this site is recognized in all transactions involving ION.

  Note: Only *one* site should be set up to be the ION bus site. All other sites should not have this field defined.

Site User Map Tab

The fields on this tab set up the user name under which the "From site" communicates with the "To site."

These fields on the **Site User Map** tab are used in non-transactional replication.

- **From Site**
- **User Name**

Note: The specified user name, and that user’s password, must be added to the User table at the "From site." The local site administrator should assign this user the appropriate permissions to allow replication data to complete the transaction. This protects the local site from another site inappropriately adding or modifying data on the local site without permission.

For more information about these fields, see the online help.

Link Info Tab

Use the fields on the **Link Info** tab to:

- Disable replication from the local site to another site (the **To Site**).
- Set up or view information about linked servers.

The "linked servers" relationship is a SQL Server concept where SQL code from one site can run on the other site in real-time.

If you are performing transactional replication between sites on different database server machines, the server name is verified against the list of linked servers in the current site's SQL Server master database. If you get a verification error, you must manually create a linked server definition in the SQL Server master database of both the current site and the "To site." You can create a linked server definition using SQL Server tools.

If a transactional replication rule exists that matches the **To Site** field shown in the current record, this linked server name cannot be deleted.

For information about the fields on this tab, see the online help.

Site Groups Form

You might want to use the **Site Groups** form to segregate your sites into different groups for reporting and cross-site data viewing. There is a default multi-site group that is used as the site group (parms.sitegroup) if you do not provide one. In that case, all sites are associated with the default group.

Replication Categories and Rules Forms

Use these forms to set up replication categories and rules for transferring data directly to or from other system databases.

Once you have defined the source site, target site, categories (high-level groupings of database tables, stored procedures, or XML documents), and time frames when replication will occur, you can write your replication rules, based on the idea: "Any time this data changes in this place, replicate it to this place." You then regenerate the replication triggers to put the replication process into play.

Replication happens chronologically, based on the order things happened in the source system. For example, an order line cannot be added until after the order is added. So, when looking at the list of tables in a category, the replication process changes data from the listed tables in the proper order—it does not just run down the list of tables alphabetically to determine what is different.

Replication Categories Form

Use the **Replication Categories** form to specify the following and to group them into categories:

- Tables that should be replicated
- Stored procedures that should be run at the target site
- XML documents that should be sent to the target site

See “Replication Categories” on page 29 for more information about the default categories.

For more information about the fields on this form, see the online help.

Stored Procedures in Categories

Replication categories can include stored procedures that handle transfers of information that are more complex than just moving data from one table to another. As an example, consider an order line where the ship site is not the local site. If the user at the source site changes the ship site on an order to a different site, then information other than just the site name must be created or updated on the target site. So the replication rule runs a stored procedure on the replication site that inserts the order, but it inserts only *some* of the order lines (those that apply to the target site) on the target site's database.

Often, stored procedures listed in the categories are launched by a trigger in the developer code (for example, during creation of an order line for another site.) The code trigger calls one of these APIs:

- RemoteMethodCallSp, which passes a specified target site
  OR
- RemoteMethodforReplicationTargetsSp, which sets the target site to all sites

Methods and stored procedures can also be launched directly from a form.

If a rule exists for a replication category that includes a stored procedure, the rule determines whether replication between the source and target sites is transactional or non-transactional. When a transaction that triggers the rule is performed at the source site, the stored procedure runs at the target site, either immediately after the transaction, or after the interval specified in the rule.

XML Documents in Categories

When developer code calls SubmitReplicationXMLSp, it passes a target site. The related category and rule determine whether that site has the specified XML document enabled. The system returns an error if the site is disabled for replication or if there is no rule defined for the site/category.

There is no "transactional" option for XML documents. All replication through XML is non-transactional.

Replication Rules Form

Once your categories are set up, use the **Replication Rules** form to assign the sites where each category's data comes from and where it will be sent, and the intervals at which the data will be replicated.

Some notes about this form:

- If you want data to move both ways for a replication category/rule, the rule must exist at both sites.
- A site uses only those replication rules where the **Source Site** equals the site value specified in the **System Parameters** form (that is, the local site).
- If the **Interval Type** is anything other than **Transactional** or **Immediate**, specify the interval and the start date. Intervals are honored at run-time, which means you can change interval times without having to regenerate the triggers. (For information on regenerating triggers, see "Replication Management Form" on page 31).
- Immediate rules behave just like transactional rules, except they are sent through the system queues, while the transactional rules are updated directly into the target database through trigger code using linked servers. Remember that "Immediate" does not mean the information is replicated immediately; rather, it means that it is placed on the replication queue immediately. Also, for transactional replication, any error at the target site rolls the entire transaction back; for immediate replication, the local site update is committed regardless.
- You can select the **Disable Replication** check box to temporarily disable a specific rule for replication. This does not require a regeneration of the replication triggers to take effect. This is a good way to quickly and temporarily disable the replication from one site to another. However, the data that is processed at the source site during the down time is not sent to the target site's replication queue; it is simply ignored. So then, when you re-establish communication with the target site, you must use the **Manual Replication Utility** to synchronize the data.
- If you determine that a target site that is connected through transactional replication is down, you could change the **Interval Type** to one of the delayed intervals (minutes or hours). As soon as you save the **Replication Rules** record, any subsequent updates on the source site are recorded into XML documents and placed on queues—no regeneration of triggers is required.
- The **Update All Columns** applies only to non-transactional replication. In non-transactional replication, the system breaks tables up into "chunks" of columns, so sending all columns can mean sending more records. Select this field if you want every column to be updated on the target site even if it was not changed by a user interaction on the local site. In most cases, this will cause a large number of unnecessary records to be generated and sent across the network, which can degrade system performance. If the field is cleared, replication occurs only for the columns that have been changed. This is the recommended setting.

Replication Document Forms

The replication document forms are covered in Chapter 5, “Replication Using Infor BODs:"

In addition, these forms are discussed in Appendix B, “Troubleshooting:"

- Replication Errors (see page 141)
- Find Replication Setup Issue (see page 142)

Other Forms that Can Affect Replication

Intranet Shared User Tables Form

Use the **Intranet Shared User Tables** form to share user maintenance tables so that they reside only at the master site for this intranet. All other sites on this intranet will have SQL views into the master site's tables, and they can add, update, and delete records through the views. For more information, see the online help for setting up shared user tables.

You can choose to share all user tables, or to share all tables except the AccountAuthorizations and UserGroupMap tables, which will then be maintained locally.

To make changes on this form, you must be logged into the selected intranet's master site. From other sites, you can view, but not change, the shared tables configuration, as long as the master site is set up to replicate the site administration category to the other sites.

Sharing Tables

In this form, select **Set up shared user tables** and click **Process**.

During processing, the user tables listed in the top grid are removed from all sites on the intranet except the master site. Views to the table on the master site are created at each using site. In addition, references to user ID or group ID columns in application-specific tables, shown in the bottom grid, are also updated.

Unsharing a Table

To "unshare" the tables, select **Set up per site user tables** and click **Process**. (The field label changes after tables are shared.) During processing, the views at the using sites are dropped, and the user tables are rebuilt at each of the using sites on the intranet.

For details on how the system handles sharing and unsharing, see “Shared Tables” on page 55.

Intranet Shared Tables Form

Use the **Intranet Shared Tables** form to choose the _all tables that you want to share—that is, the tables that should reside only at the master site for this intranet. All other sites on this intranet will have SQL views into the master site's table, and they can add, update, and delete records through the views. For more information, see the online help for setting up multi-site shared tables.

To make changes on this form, you must be logged into the selected intranet's master site. From other sites, you can view, but not change, the shared tables configuration, as long as the master site is set up to replicate the site administration category to the other sites.

Replication Management Form

The **Replication Management** form contains links to most of the replication tasks, and buttons to handle replication trigger and view updates.

Regenerate Replication Triggers

Use the **Regenerate Replication Triggers** button to put any changes to your replication setup into play. For example, any time you change a rule or category, you must regenerate the triggers.

Note: Because clicking this button drops and re-creates the T-SQL trigger code in the database, no other users should be in the database (logged into the system) when the administrator does this. The regeneration of triggers must be done separately on each site.

For each site used in a transactional replication rule, the regeneration process checks that "linked servers" are set up before regenerating the replication triggers. If this validation fails for any linked server, an error message displays and the regeneration cannot continue until the linked server is set up.

Regenerate Views to Master Site

To realign the SQL views at all user sites in a shared tables intranet with the corresponding _all tables at the master site, click **Regenerate Views to Master Site**. You must be logged into the master site to use this button.

Note: Because clicking this button drops and re-creates T-SQL code in the database, no other users should be in the database (logged into the system) when the administrator does this.

Manual Replication Utility Form

Use the **Manual Replication Utility** form for data initialization or resynchronization of data between sites (for example, if the target site has been offline for some time, or when data already exists in a site that has just been added to a multi-site system).

Most of the fields are self-explanatory. Select **Local Site Data Only** if you want to send only information about the local site and not all sites.

Update _All Tables Form

Caution: Use this form with great care.

The **Update _All Tables** form is available to expert users and can be used to repopulate or truncate the _all tables at the local site. Use it when the _all tables do not properly reflect the data in the corresponding base table, generally because some type of corruption happened. The form resides in the Master Explorer but not in any Master Explorer subfolder.

Some notes about this form:

- If the base table is large and/or replication is occurring (that is, the **Disable Replication** check box is not selected), this utility can take a long time to process.
- You can query only _all tables that have a corresponding base table and that are not tmp* tables.
- The two **Delete** buttons use the site field to delete the rows (records) that either match or do not match the specified site in the selected _all tables.
- When the **Disable Replication** check box is selected, replication of _all data does not occur as it is deleted and readded. The benefit is that performance is better if only the local site has bad data. The downside is that the other sites do not get the changes you are making in the local _all table.
- Any failures are shown in the message column when the operation completes. All tables are handled separately, so one failure does not roll back all the other tables.
- A message indicates how many tables were successfully processed out of how many were attempted. It displays an error icon for any that failed.

Truncate Tables

The **Truncate Tables** button deletes all the data in the selected _all tables. A truncate action does not fire database triggers or do SQL Server logging truncation, so it is the fastest way to delete all the rows in a table.

If a master site is defined and a non-master site on the same intranet is being processed, clicking the **Truncate Tables** button only deletes rows for the current site from the view. In the master site, where the base table still exists, the truncate action is still performed.

Repopulate Tables

The **Repopulate Tables** button on the **Update _All Tables** form repopulates the _all tables with data from the local site. You might repopulate _all tables during initial setup of the system, or you might do it because the _all tables have bad data for whatever reason. You need not truncate before you repopulate. When you click the repopulate button, all data is deleted in the selected _all tables before it is repopulated.

Repopulation is done in the local site; whether the repopulation is also done in other sites depends on whether you have the **Disable Replication** option selected and whether there are rules for replication of those tables to the other sites. If you are using a view instead of a table, the **Disable Replication** check box and rules become irrelevant.

Example 1: Sites A and B Sharing Item_all Data

If the item_all table is corrupt in a system with sites A and B, who share item_all data through a replication rule, you could use the truncate button this way:

1 In the **Update _All Tables** form at Site A, select the item_all table and click **Truncate Tables**. This truncates the item_all table in Site A.

2 In the **Update _All Tables** form at Site B, select the item_all table and click **Truncate Tables**. This truncates the item_all table in Site B.

3 In Site A, select the item_all table and click **Repopulate Tables** (leave the existing replication rule enabled). This adds all of site A's item_all records in both sites A and B.

4 In Site B, select item_all and click **Repopulate Tables** (leave the existing replication rule enabled). This adds all of site B's item_all records in both sites A and B.

In the above example, either site can have bad data, or both sites can have bad data. The item_all tables in both sites are repopulated entirely from scratch.

Example 2: Truncating Tables with Master Site and Views

Two linked sites, OH and MI, are on the same intranet. OH is the master site. The whse_all table is shared - so in the OH_App database, whse_all is a table, while in the MI_App database, it is a view pointing at the table in the OH_App database. If you log into MI and truncate whse_all, the system sees it as a view and deletes whse_all where site_ref = 'MI'. The result set for whse_all would change from this:

Site_ref	Whse Name
MI DIS1	Distribution One
MI DIST	Distribution Center
MI MAIN	Progressive Cycles
MI TEST	NULL
OHIO DIS1	Distribution One
OHIO DIST	Distribution Center

Site_ref	Whse Name
OHIO MAIN	Progressive Cycles
OHIO ZZZZ	TEST

to this:

Site_ref	Whse Name
OHIO DIS1	Distribution One **Note:**
OHIO DIST	Distribution Center
OHIO MAIN	Progressive Cycles
OHIO ZZZZ	TEST

However, if you log into the OH site, where the whse_all table exists as a table, and you click the **Truncate Tables** button, a truncate is performed and there are no rows left in the whse_all table.

If you want to completely repopulate the whse_all table from the base whse tables, the fastest way is to truncate whse_all in the master site and use the **Include All Sites in Intranet** check box (at the master site) to repopulate in every other site in the intranet.

Include All Sites in Intranet

Select **Include All Sites in Intranet** if you are working at a master site, and some _all tables at your master site and the corresponding _all views at the other sites on the shared intranet need to be truncated and/or repopulated. Then click either **Truncate Tables** or **Repopulate Tables**.

The selected _all tables are then truncated/repopulated at the master site, and a remote method call is run to truncate/repopulate the views at the other sites on the intranet. The changes made at the other sites are then replicated back to the master site so that the tables at the master site are populated correctly.

Chapter 4: Replication Behind the Scenes

4

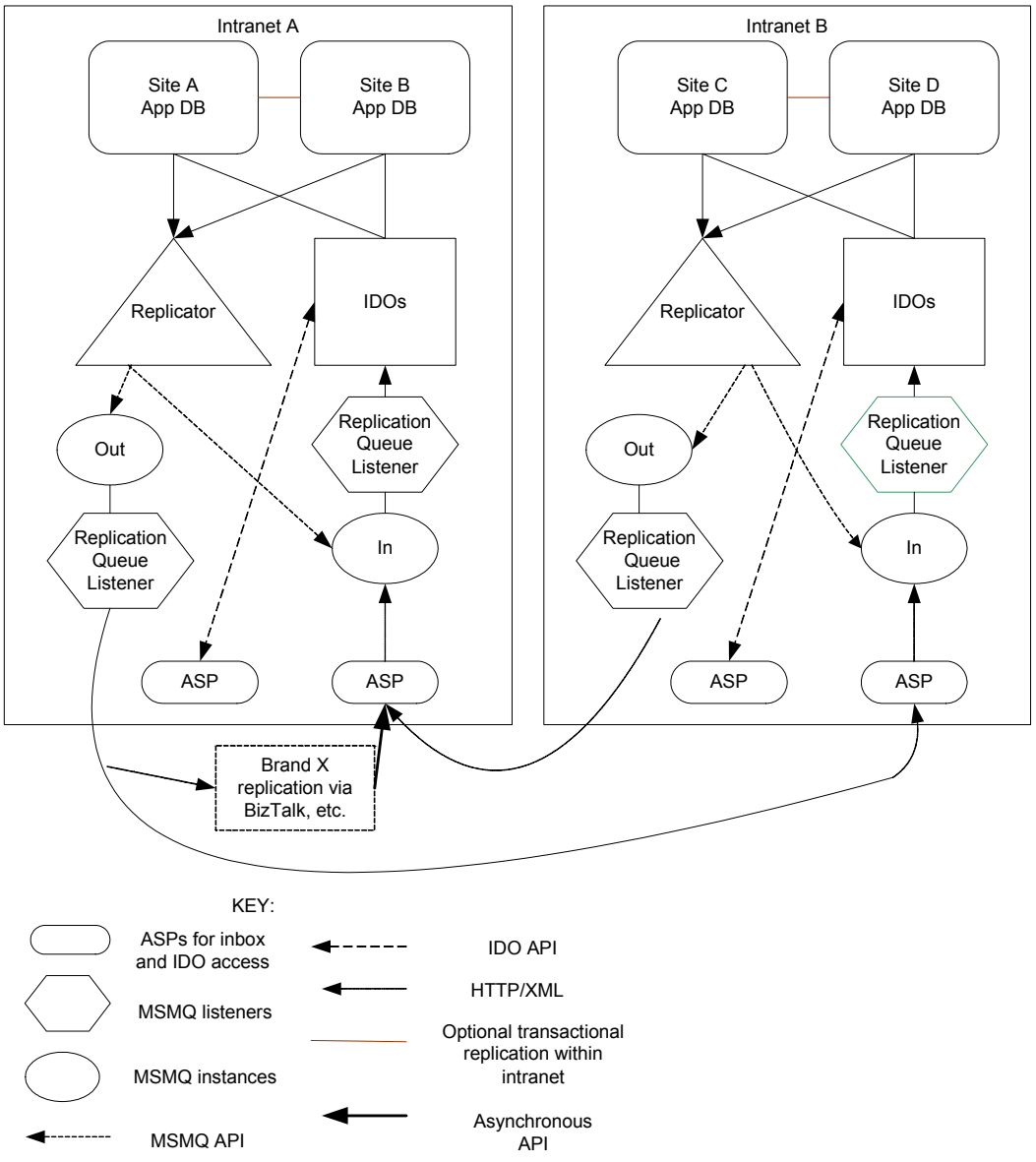
This chapter provides information about what happens during the replication process.

Additional behind-the-scenes information about replication using ION and business object documents is included in Chapter 5, “Replication Using Infor BODs."

Overview of Processes and Tools

This diagram provides an overview of the processes and tools used internally during replication.

Many of the processes and tools shown here apply only to non-transactional replication. The distinctions between transactional and non-transactional processes are made clearer in the Example Flows starting on page 51. BOD replication is covered in Chapter 6.



Replication Services

If your system is using any non-transactional replication, the following Windows services were installed and set up on the utility server at installation. They should start automatically.

- **Infor Framework Replicator** (Replicator.exe) – This service monitors all the sites on its intranet, looking at data that has been staged to replication tables due to non-transactional replication rules. When it finds data that is ready to send, it extracts the data from the staging tables, formats XML documents in the IDO Request schema with the data, and posts the documents to the appropriate queue. Documents bound for sites on the same intranet are placed directly into that intranet's inbound queue. Documents bound for sites on a different intranet are placed on the outbound queue, for posting across the internet to the target intranet's URL.

  For each intranet, the Replicator must have a "bootstrap" site configuration, which is specified with the Configuration Wizard on the utility server. This site must be set up through the **Sites** form's **User Map** tab so that it communicates with the other sites on its intranet. From the bootstrap site, the Replicator can query a list of all the other sites on its intranet and connect to those sites to monitor them. For more information about how to set this up, see the *Multi-Site Implementation Guide*.
- **Infor Framework Replication Queue Listener** (ReplQListener.exe) – This service monitors the inbound and outbound Microsoft Message Queues (MSMQs). XMLs on the outbound MSMQ are posted to the URL for the target intranet. XMLs on the inbound MSMQ are dispatched into the target site. The Replication Queue Listener also performs the appropriate XSL transformation on the XML document when transferring documents from an outbound queue to another intranet. The transformation style sheet that is used depends on the source site's System Type, the collection or table name, and the target site's System Type.

MSMQ Instances

MSMQ (Microsoft Message Queue) instances are necessary only when connecting sites through non-transactional replication categories and rules. Each intranet is defined to have the following MSMQ instances:

- **Inbound Queue** – Receives requests from either the Replicator or an ASP page. The Replication Queue Listener, which is monitoring the queue, directs the requests to the IDO layer for placement into the target database.
- **Outbound Queue** - Stores requests for outbound XML documents until the XML is successfully sent to the target intranet or a fatal error is encountered.

Active Server Page

Each intranet also has an Active Server Page (ASP) that provides the Internet access to the inbound queue.

Triggers

The replication feature is driven by table triggers. SyteLine tables included in a replication rule's replication category have special auto-generated replication triggers in addition to their standard application triggers. After you configure replication operations through the replication forms (see Chapter 3, “Forms Used in Replication"), you must click **Regenerate Replication Triggers** on the **Replication Management** form. This causes the toolset to regenerate the necessary replication triggers associated with the tables to be replicated. It first drops all existing replication trigger code *for the local site only* and then recreates the replication triggers based on the updated information you built in to the **Replication Categories** and **Replication Rules** forms.

Note: Regenerating the triggers should be done at each site. It should be performed by the system administrator only when no other users are connected to the database.

> Trigger code is not generated for table replication objects that are actually views in this local source site, or in the target site. For more information, see “Shared Tables” on page 55.

Replication triggers are created only for tables associated with replication categories and where rules exist that contain those categories. (Triggers are generated even if the rules are currently disabled.) The replication triggers for each table are:

- *TableName*IupReplicate – Handles INSERT and UPDATE operations
- *TableName*DelReplicate – Handles DELETE operations

Triggers and Transactional Replication

If the replication interval type (assigned in the **Replication Rules** form) is **Transactional**, the triggers call similarly named stored procedures to immediately perform the requested operations at the remote site. The *TableName*IupReplicate trigger calls ReplIup_*TableName_DestinationSite*Sp to perform INSERT/UPDATE operations. The *TableName*DelReplicate trigger calls ReplDel_*TableName_DestinationSite*Sp to perform DELETE operations.

To call the stored procedures from the triggers, SyteLine stores primary key information in a separate table and uses that information in the called stored procedures. For all insert or update operations, the table tmp_TrackRows_RP_tablename is used. It contains two columns: the unique session ID for the calling SQL code, and the RowPointer value from the base table, which uniquely identifies a row in the base table.

Because delete triggers fire after the delete has taken place, storing a RowPointer does no good. Since every table has a different primary key, a separate table is generated at the same time as the replication triggers, to store the session ID, site reference, and primary keys. These tables are named tmp_TrackRows_*TableName*.

Triggers and Non-Transactional Replication

If the assigned replication interval type is **Non-Transactional** (Immediate or some defined interval), these triggers place the data in a staging area for later replication across sites. The staging area consists of these tables:

- *ReplicatedRows3* – A database table that stores general data about the replication request. It has the following columns:
  - **ObjectName** – The name of the table, method, or XML document being replicated, as found in **Replication Objects**.
  - **ObjectType** – Type is table, method, or XML document.
  - **ActualObjectName** – The actual name of the table. This can be used to rename the object being replicated, if necessary.
  - **ActualObjectType** – The type of the actual object.
  - **UpdateAllColumns** – Whether the replication rule indicates that all columns are to be updated.
  - **RefRowPointer** – For table objects, the RowPointer being referenced.
  - **OperationType** – Indicates what type of operation is to be performed:
    - **1** = Insert
    - **2** = Update
    - **3** = Delete
    - **4** = Method call
  - **OperationNumber** – An incrementing integer used to ensure the data is replicated to the remote site in the same order it was created at the original site (to preserve data integrity).
  - **ToSite** – Site to which the data is to be sent.
  - **ProcessingPtr** – Unique ID value used by the process that actually moves the data to an outbound queue. This prevents other processes from attempting to transfer the same row.
  - **WorkflowList** – Currently unused.
  - **RowPointer** – Unique ID for the current record in this table.
  - **CompressionPtr** – Not used.
- *ShadowValues* – The table that contains the actual data being replicated. The ShadowValues table contains these columns:
  - **OperationNumber** – The **OperationNumber** and **RowPointer** join this table to the ReplicatedRows3 table, which contains all other needed information.
  - **LineNum** – Since there are sometimes more than 55 columns in the base table and old and new rows, multiple lines are used.
  - **OldNew** – Indicates whether the record contains the new values or the old values. For insert, only new values are stored. For update, both old and new values are stored. For delete, only old values are stored.
  - **Value*n*** – Each of the Value*n* columns contains the value of the column being replicated. This is an ntext column (blob). Because the text column was not originally used, notes are copied using a separate mechanism. The blob columns have more overhead associated with them,

so the values default to 1000 bytes in place if they are small enough. This is accomplished via the following T-SQL system stored procedure call:

```
sp_tableoption N'ShadowValues', 'text in row', '1000'
```

- **Name*n*** – Each Name*n* column contains the name of the replicated column.
- **RowPointer** – See above.

A single SELECT statement is used to retrieve information out of the ShadowValues table by the Replicator, which then moves the data to the outbound or inbound queue and creates the necessary MSMQ messages. Those messages are processed by the Replication Queue Listener.

Other application database tables used by the replication process include:

- *ReplicatedRowsErrors* – Contains rows that failed to load on inbound replication. This failure could occur because of something like a table constraint failure (primary keys, check constraints, foreign keys).
- *ShadowValuesErrors* – Contains the actual bad rows of data.
- *ReplicationOperationCounter* – Increments the unique operation counter.

Disabling Triggers

All existing base triggers should have a call to the dbo.SkipBaseTrigger() function at the top of the code. If the function returns a value of **1**, the trigger code should exit. This allows inbound replicated data to avoid the validations and processing associated with the base table's trigger. The way some triggers work makes it impossible to allow them to fire. For example, a base table trigger might insert into another table (table X), which would then collide with the data being replicated for table X. Since tables are grouped by categories, if a trigger changes other pertinent data in another table, that table should be in the same category as the base table for the trigger.

The generated trigger logic calls the dbo.SkipAllUpdate() function if the base table has an associated _all table that is maintained within the same trigger. This allows the _all table to be populated if desired, even though the rest of the base trigger logic does not fire.

Custom Triggers

For information about specifying and generating custom table triggers, see the online help topic on "Maintaining Application Schema Metadata."

Stored Procedures and Functions Used in Replication

This section contains a brief description of each stored procedure and function that plays a role in replication.

BuildWhereClauseSp

This routine takes an input set of column names and values and builds a where clause to return to the caller. If the @TableAlias is not provided, no table alias is used. All values are wrapped in quotes to simplify the logic, allowing implicit type conversions to occur. All values except the first can be NULL in the where clause.

ClearReplicateObjectsSp

This routine cleans out all the replication rows associated with a particular processing pointer value. If the @OperationNumber is passed as anything but NULL, then a failure has occurred. All records before that number are removed and all records after that row are marked as not being processed.

DelayedReplication

This function returns a **1** if delayed replication from the source site to the target site is currently enabled.

DeleteRemoteNotesSp

This routine is meant to be run at a remote site. For the notes pertaining to the single row of the input @TableName specified by the @ToRowPointer or @ToWhereClause if @ToRowPointer is null, this routine:

- Deletes any specific notes tied to that record.
- Deletes any ObjectNotes records tied to that record that point to specific notes, user notes, or system notes.
- Deletes any NotesSiteMap records that tie from the @FromSite to any of the specific notes deleted.

Reusable notes (user and system) are not removed.

DeleteReplDataSp

This procedure is a TSQL procedure in SQL 2000. In SQL 2005 it is a SQLCLR procedure that runs in its own connection and transaction, thus avoiding a loopback error. It deletes temporary data at the remote site.

DropReplicationTriggersSp

This routine drops all of the triggers in the current database that have the 'Replicate' suffix in the name. If a particular table name is passed, then only the replication triggers on that table are dropped. The staging tables and generated stored procedures which are used by the replication triggers are dropped as well.

GetReplicateObjectsSp

The SetReplicateObjectsSp procedure marks all the ReplicatedRows3 table rows with a "ptr" value. That "ptr" value is then used by this routine to retrieve all the replicated row information. Separate procedures are used because methods that return result sets in object studio can not also return individual output parameters, and the @ProcessingPtr will be needed for later cleanup. This routine also finds columns with values that have not changed and removes them. If no columns changed, the routine deletes the entire update operation before sending the list of rows to the service.

GetReplicationCounterSp

This routine retrieves the next operation counter for use in replication. It is important that this code is both fast and non-blocking from a lock perspective.

GetReplicationOnSp

This routine determines whether delayed or transactional replication is turned on for a particular object from a site to a site. It combines the functionality of the TransactionalReplication and DelayedReplication functions to reduce the total processing time.

GetReplicationToSiteInfoSp

This routine returns the following information: a target site for the current site based on replication rules, the name of the MSMQ to be used for outbound replication from the local site to the target site, the name of the user specified for replications from the localsite to the target site, from the SiteUserMap table, and the encrypted password for the user. If the queue name is NULL, it indicates that the intranet was not properly specified for the local site. If the user name/password values are NULL, then no replication user has been set in the SiteUserMap table for the local site to the target site.

InitRemoteServerSp

This routine is used to set up for a remote procedure operation, insert, update, delete, or stored procedure call. The replicating flag indicates whether this routine wants the triggers to fire in the remote server.

IsObjectReplicatedSp

Use this procedure to return a flag that specifies whether a given object is set up for replication to any sites. The **Enabled Flag** (**0**/**1**) is returned as an output parameter. The stored procedure itself always returns success.

ListSiteAllTablesSp

This routine lists all of the _all tables or views for the current site.

LoadReplicatedNotesSp

This routine takes a row out of the NotesContentShadow table and updates the matching system, user, or specific note. If the note does not exist, as specified by the ReusableNotesSiteMap table, then it is created and a mapping between the origin site and this site for the note token is created in the ReusableNotesSiteMap table.

MakeRemoteObjectNotesSp

This routine is used to create an ObjectNotes record. It is called at the remote site and assumes the system, user, or specific notes records have already been created before it is called. The NotesSiteMap table records, which map when notes were last copied from a particular site, must also be up-to-date before this routine is called. If @ToRowPointer is null, then @ToWhereClause will be used to get the rowpointer.

ManualReplicationRunning

If no site is passed in, this function returns **1** if manual replication is underway at all. If a site is passed in, then it returns **1** only if manual replication is running to that site.

ManualReplicationSiteVar

This function returns the value of the session variable that indicates whether the input site is running for manual replication.

PullReplDataSp

This procedure is a TSQL procedure in SQL 2000. In SQL 2005 it is a SQLCLR procedure that runs in its own connection and transaction, thus avoiding a loopback error. It populates temporary data at the remote site.

RemoteInfobarSaveSp

This routine saves the passed in infobar into a session variable that will be retrieved by the ResetRemoteServerSp routine. This routine should be called by any procedures that are called remotely and that have an @Infobar output parameter.

If the Infobar contains a message, RemoteInfobarSaveSp raises an exception from the database. This is how the error message gets back to the IDO layer without it knowing what parameter in a method call was the error message return.

RemoteMethodCallSp

This routine is used to call a stored procedure at a remote site. The name of the site, the name of the stored procedure, and the parameter values must be passed. For delayed replication, a success status merely indicates the call was successfully saved for later processing. For transactional replication, a call directly to the remote server is made. The remotely called stored procedure is expected to use the RemoteInfobarSaveSp routine to return any error message. If the remotely called stored procedure is not found, RemoteMethodCallSp returns an error message indicating an improper setup of replication rules.

If the **To Site** is the same as the local site, this routine simply executes the stored procedure locally. If the **To Site** is transactionally linked to the local site, the stored procedure is called on that server. If the **To Site** is not transactionally linked to the local site, data is stored in the ReplicatedRows3 and ShadowValues tables to save the method and parameters for later replication through XML documents.

RemoteMethodForReplicationTargetsSp

This routine calls the RemoteMethodCallSp routine for the input method for every target site currently defined in the replication rules.

RemoteReplSpCreateCodeSp

This function generates the SQL code that creates a remote server stored procedure for pulling data to that site from this site.

RemoteReplSpCreatePKCodeSp

This function produces the code necessary to create a stored procedure on a linked SQL Server that pulls data from the calling server. It pulls into a table containing only primary key information used by transactional replication.

RemoteReplSpName

This function returns the name of a transactional replication "pull data" remote stored procedure. There are two, one for primary keys only, and one for all columns in a table.

RemoteReplTableCreateCodeSp

This function creates the code that creates a cross-server temporary table on another SQL server. If @PKOnly = 0, then use all the base table columns plus a SessionID column. Otherwise, use only the primary key column.

RemoteReplTableName

This returns a replication table name to be created at a remote site on another SQL Server. It may or may not be a primary key only table, as specified by the @PK flag.

ReplColumnLengthSp

This determines the length of a column and is used by non-transactional error forms.

ReplicationObjectValidSp

This validates that the input table or stored procedure actually exists in the database.

ReplicateOneNoteSp

The NotesContentShadow table is populated with the appropriate system, user, or specific note information and then a remote method is called which takes that shadow record at the remote site and populate the appropriate notes record there.

ReplicationColumn

This function takes in a column name and the base datatype and returns a string of code to use for that column. The code returned might be just the column name, or it might be the column name wrapped by some conversion function.

ReplicationDateTime

This routine converts a datetime value to the format required for replicated datetime values. It might need to go to XML format, which is convert style 126, at a later time. It also handles the time zone change between sites.

ReplicationDisabled

This function returns a **1** if the replication from the source site to the target site is currently disabled. Otherwise, a **0** indicates some type of delayed replication interval. If the manual replication utility is running, then the replication for all sites which are not being replicated to are disabled.

ReplicationFilters

This routine returns the filter clause to use for replication. Since a table can be in different categories with different filters, the output filter is an OR of all the category filters.

ReplicationOnObjectSp

This routine is used on the **Find Replication Setup Issue** form. It checks for replication database setup problems that would prevent replication from the local site to the entered site, object, and object type. For more information, see “Find Replication Setup Issue” on page 142.

ReplicationShadowInsert

This internal routine is used to generate the code that inserts into the ShadowInsert table for delayed replications. When dealing with views, if none of the columns specified in a view are updated, this function skips adding any records.

ReplicationTriggerDelCodeSp

This routine generates a result set containing the code lines to replicate a deletion on the input table name. If there are remote site link server and database names available, it generates direct transactional delete statements to remote servers. It outputs multiple object creates, separated by a "GO" on a line by itself. The objects are a delete replication trigger and a stored procedure for each site linked transactionally, and a table containing the primary key values for the input table.

ReplicationTriggerDelCode2005Sp

This is a SQL 2005 version of ReplicationTriggerDelCodeSp, with changes made to handle cross-server replication differently to avoid loopback errors.

ReplicationTriggerIupCodeSp

This routine generates a result set containing the code lines to replicate insert/update on the input table name. If there are remote site link server and database names available, it generates direct transactional DML statements to remote servers. It outputs multiple object creates, separated by a

"GO" on a line by itself. The objects are an insert/update replication trigger for the input table and a stored procedure for each site linked transactionally.

If a view is used, then the routine checks for delayed replication to see if any of the columns in the view were updated. If not, then it skips the entire set of code to that target site.

ReplicationTriggerIupCode2005Sp

This is a SQL 2005 version of ReplicationTriggerIupCodeSp, with changes made to handle cross-server replication differently to avoid loopback errors.

ReplicationTriggerWhere

This routine makes the primary key "where" clause for an input table name. The table abbreviations used vary, so they are passed into this routine. If no primary key exists, then the RowPointer column is used. The string REPLACESPACE@ is put into the where clause so that appropriate indentation can be set in by the caller.

ReplicationUpdateAll

This function returns a **1** if the update all columns flag is set for the input source site, target site, and object.

ReplKeysTableName

This routine returns a string containing the name of the replication keys table associated with the input table.

ReplLabelSp

This routine returns either the name of the column if it is a table or the name of the parameter if it is a method (stored procedure). For procedures, if the @Infobar parameter is found to be an input only parameter, the name is changed to @InInfobar so that the resubmittal program knows not to make this an output parameter.

ReplTriggerProcName

This routine returns the name of the generated stored procedure used to copy data in a live link from an insert or update operation to another site.

ReplTriggerTemp

This routine returns a string containing a create table statement. The table is TrackRows_*TableName* and it contains all the primary key columns of the input table, as well as a **SessionID** column.

ResetRemoteServerSp

When this routine runs, the session information on this (the remote server for an operation) is wiped out. It also retrieves the infobar session variable value, which might or might not have been set.

SaveReplicateErrorOldSp

This routine saves the old data for a failed delayed inbound replication. The @OperationNumber is returned to the caller by the SaveReplicateErrorSp routine and then passed into this routine. Only updates have old values.

SaveReplicateErrorSp

This routine saves the data for a failed delayed inbound replication. The @OperationNumber is returned to the caller to be used with the SaveReplicateErrorOldSp to save the old values, if needed for an update operation. Method calls, inserts, and deletes do not require the old row information.

SetReplicateObjectsSp

If no @ToSite is passed, this routine gets the data for all to sites from the current site. The ReplicationRows3 table contains a list of objects needing to be replicated. A @ProcessingPtr value is generated in this routine and returned to the caller as well as being set in the ReplicatedRows3 rows that need to be shipped out for replication.

SkipBaseTrigger

This function returns a **1** to indicate that base trigger processing should be skipped or a **0** to indicate that it should not be skipped.

SkipAllReplicate

This function returns a **1** to indicate that _all replicating should be skipped or a **0** to indicate that it should not be skipped.

SkipAllUpdate

This function returns a **1** to indicate that _all copying should be skipped or a **0** to indicate that it should not be skipped.

SkipRemoteUpdate

This function returns a **1** to indicate that the call to remote method call should be skipped or a **0** to indicate that it should not be skipped.

SkipReplicatingTrigger

This function returns a **1** to indicate that trigger processing should be skipped or a **0** to indicate that it should not be skipped. If replication data is being imported into a server, then the Skip trigger flag should have the 1 bit set.

TransactionalReplication

This function returns a **1** if the replication from the source site to the target site is currently transactional. Otherwise, a **0** indicates some type of delayed replication interval. If a NULL @TargetSite is passed, see if transactional replication is turned on for any target site.

TransferNotesToSiteSp

This routine allows all the notes for a particular row of a table to be copied to a remote site. The RowPointer of the local site table is used to see what notes are tied to the record locally. Additionally, a "to where" clause which will retrieve that same record at the remote system is also needed. The equivalent record at a remote site may not have the same RowPointer value as the one on the local site, which is why a where clause is used.

If the ToRowPointer is known, it can be used instead of the ToWhereClause. While the local record could be retrieved using this same where clause, the RowPointer is more efficient, hence both are required by this routine. The BuildWhereClauseSp routine can be used to build a where clause. The @DeleteFirst flag indicates whether the existing notes for the remote record should be removed before copying the notes from the current site.

Here is an example call:

```
EXEC @Severity = BuildWhereClauseSp
@WhereClause = @WhereClause OUTPUT
, @Key1Name = 'item'
, @Key1Value = @ItemItem
```

```
EXEC @Severity = TransferNotesToSiteSp
 @ToSite = @RemoteSite
, @TableName = 'item'
, @RowPointer = @ItemRowPointer
, @ToWhereClause = @WhereClause
, @ToRowPointer = NULL
, @DeleteFirst = 1
, @Infobar OUTPUT
```

If you are at the remote site and have the RowPointer for the record at both sites, just call TransferNotesToSiteSp from the remote site back to the original site using RemoteMethodCallSp.

UpdateAllTablesSp

This procedure is used by the **Update _All Tables** form. It does the actual work of updating a particular table. @OperationType is set to one of the following:

1 – Repopulate the table with the current site data.

2 – Truncate the table.

3 – Delete records for the input site.

4 – Delete records which are not for the input site.

ValidateObjectReplicatedSp

Use this procedure to validate that a given object is set up for replication to a given site. If not, the procedure returns failure and an appropriate error message.

Stored Procedures Created by Replication Regeneration

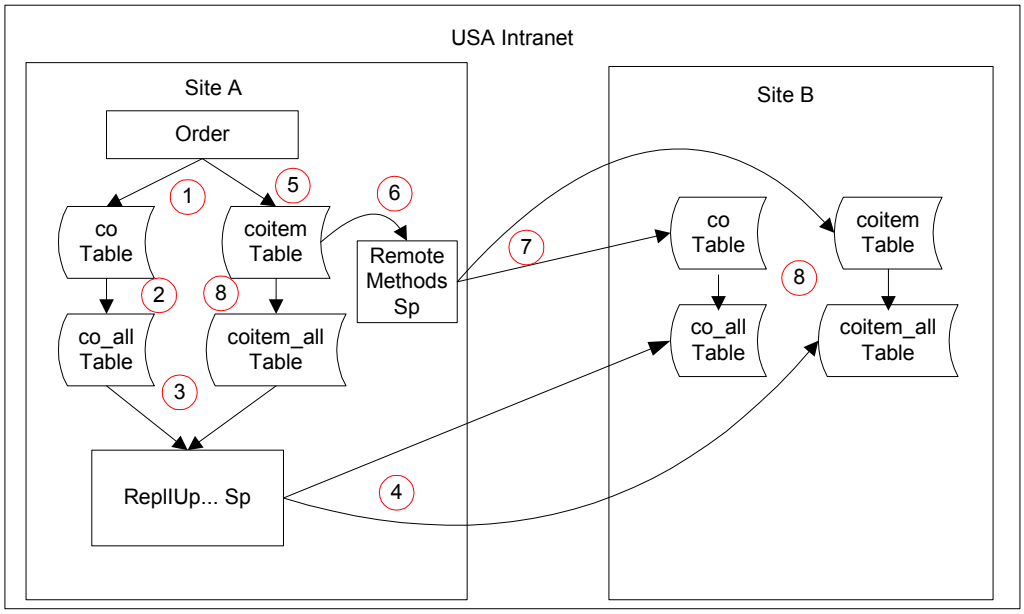
After you create categories and rules and click the **Regenerate Replication Triggers** button on the **Replication Management** form, the generation process runs ReplicationTriggerDelCodeSp and ReplicationTriggerIupCodeSp (see descriptions in the previous section). For each table that is included in a replication category and rule, these routines look up the table's schema on the current database and then create the replication table triggers and the following two stored procedures:

- ReplIup_*TableName_DestinationSite*Sp
- ReplDel_*TableName_DestinationSite*Sp

These newly created stored procedures use table-specific INSERT or UPDATE statements to actually perform the replication.

Example Flows

Transactional Replication - Example Flow



1 A user at Site A creates an order with one line using Ship Site A. The record is saved to the **co** and **coitem** tables in Site A.

2 The order data is copied to the coinciding _ALL tables in Site A through trigger code in the **co** and **coitem** tables.

3 The **Site A _ALL** tables' trigger code calls the ReplIUp... stored procedure.

4 The stored procedure immediately updates the Site B _ALL tables.

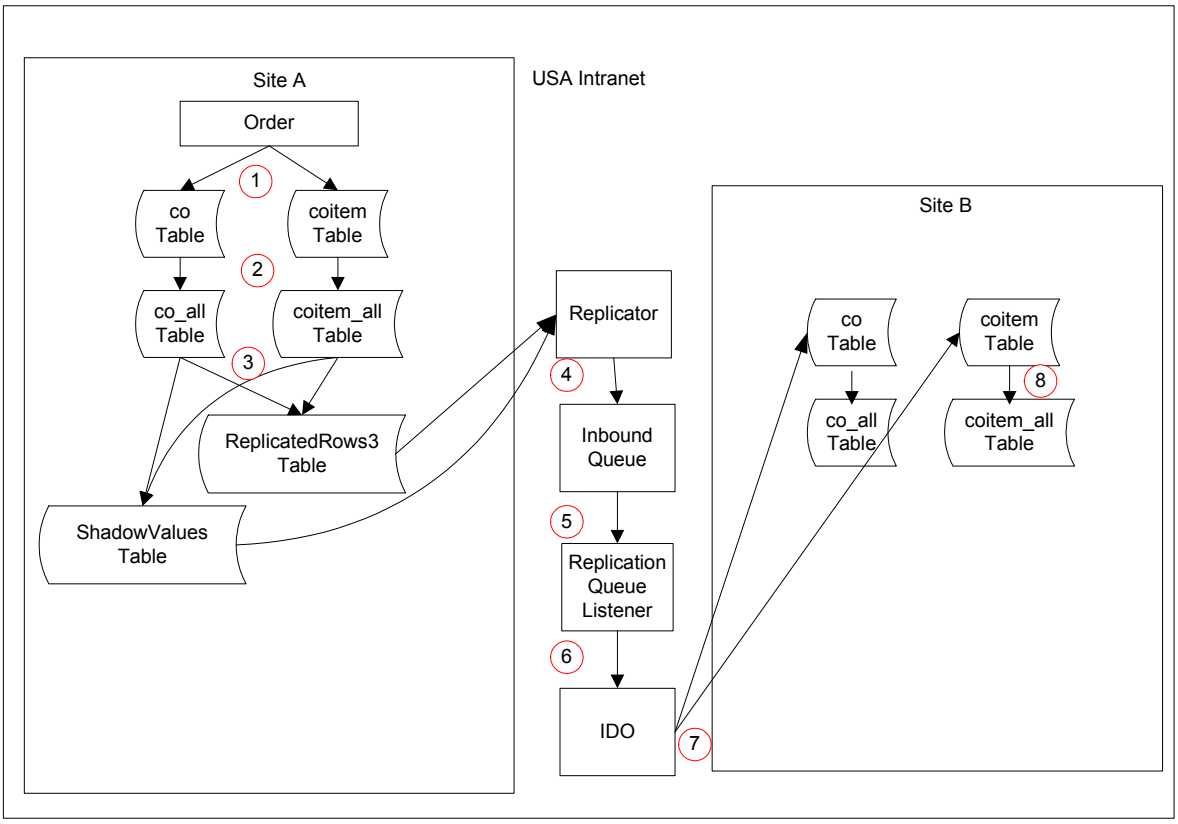
5 A user at Site A adds a second line to the order, using Ship Site B. The record is saved to the **coitem** table.

6 The system calls the RemoteMethods stored procedure in the source site.

7 The stored procedure creates in Site B the customer row (if not existing in Site B), the order row, and the coitem row (line 2 only), according to business rules.

8 The triggers in the base tables copy the data to the coinciding _ALL tables.

Non-Transactional Replication Over Same Intranet - Example Flow



1 A user at Site A creates an order with two lines, one shipped from Site A and one from Site B. The record is saved in the **co** and **coitem** tables.

2 This data is immediately copied to the corresponding _All tables.

3 Categories and rules for this type of insert/update are already in place, and set for delayed replication. The trigger on the _All tables sends the request to the ReplicatedRows3 and ShadowValues tables.

4 The Replicator picks up the request from ShadowValues and checks the rules and categories. It determines that the request is for Site B on the same intranet. Depending on the rules, it may put a time/date stamp on the request, for later replication. It sends the request to the Inbound Queue.

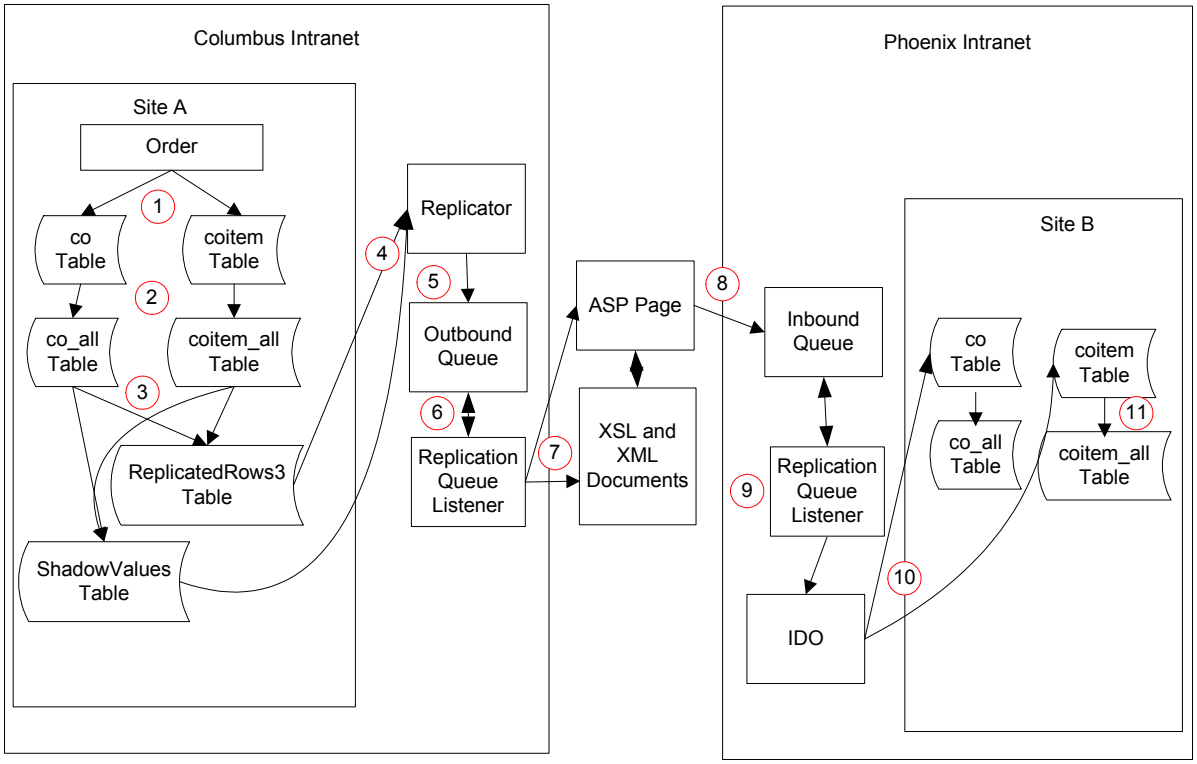
5 Once the Inbound Queue accepts the request from the Replicator, the service removes the appropriate data rows from the ReplicationRow3 and ShadowValues tables.

6 The Replication Queue Listener is waiting for the next request. It finds one in the Inbound Queue, so it executes the request based on the content of the document. (The Replication Queue Listener actually makes the IDO call to do the operation in the target site.) The contents of the document also show which site is the target; this becomes the site configuration name. The configuration name must *exactly* match the site name defined in the **Sites** form.

7 The IDO routes the contents of the document to the appropriate site database (the login is already established). The update from Site A is completed at Site B.

8 This data is immediately copied to the corresponding _All tables.

Non-Transactional Replication Across Different Intranets - Example Flow



1 A user at Site A creates an order with two lines, one shipped from Site A and one from Site B. The record is saved in the co and coitem tables.

2 This data is immediately copied to the corresponding _All tables.

3 Categories and rules for this type of insert/update are already in place, and set for delayed replication. The trigger on the _All tables sends the request to the ReplicatedRows3 and ShadowValues tables.

4 The Replicator picks up the request from ShadowValues and checks the rules and categories. It determines that the request is for Site B on a different intranet. It prepares an XSL document for a later transfer to an ASP page, and it puts a time/date stamp on the document. It also builds the XML document.

5 It sends the documents to the Outbound Queue. Once the Outbound Queue accepts the request from the Replicator, the service removes the appropriate data rows from the ReplicationRow3 and ShadowValues tables.

6 When the document is sent to the Outbound Queue, the Replication Queue Listener picks up the request.

7 The Replication Queue Listener sends the document to the target intranet and site's ASP page, using HTTP protocol.

8 The inbound ASP page places the XML into the target's Inbound Queue.

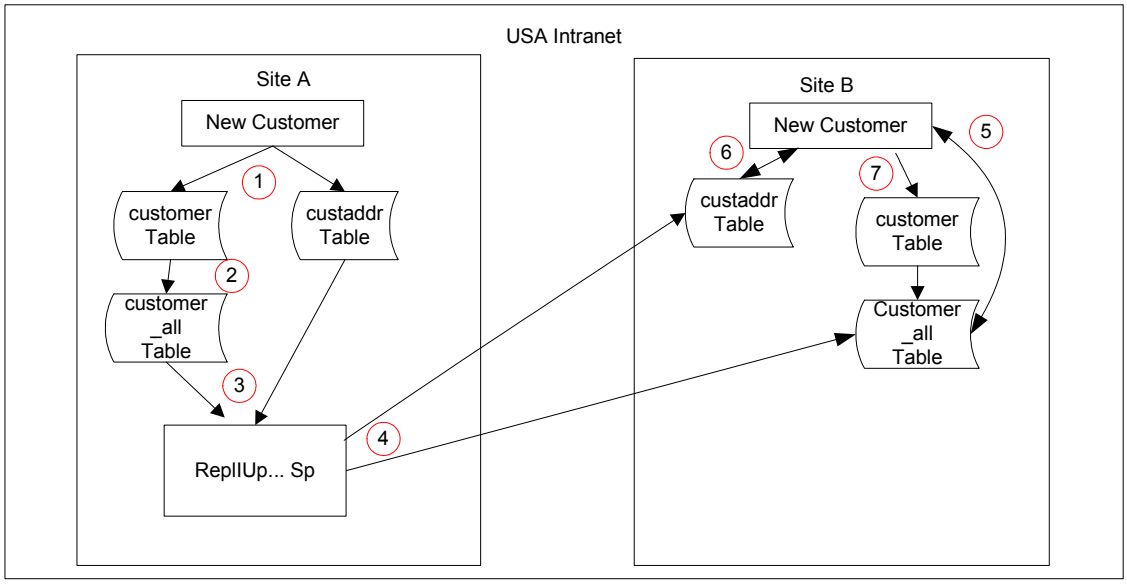
9 The Replication Queue Listener on the target intranet is waiting for the next request. It finds one in the inbound queue, so it executes the request based on the content of the document. The contents of the XML document also indicates which site is the target, and the user name in the document (the designated user specified on the **Sites** form as the source site for this site-pair) becomes the IDO login name.

10 The IDO routes the contents of the document to the appropriate site database (the login is already established). The update from Site A is completed at Site B.

11 This data is immediately copied to the corresponding _All tables.

Transactional Replication - Adding a Customer

Replication of customer and vendor data is handled differently than replication of other types of data.



This example assumes that the customer and custaddr tables are replicated between sites A and B.

1 A user at Site A adds a customer X.

2 The data is copied to the customer_all table in Site A through trigger code in the customer table. The custaddr table does not have a corresponding _all table.

3 The customer_all and custaddr table's trigger code calls the RepIIUp... stored procedure.

4 The stored procedure immediately updates the Site B custaddr and customer_all tables.

5 A user in Site B wants to select the customer added in Site A. The **Customers** form at Site B shows only local customer records (those in the customer table at Site B). However, the user at Site B can add a "new" customer and see customer X listed in the **Customer number** field's drop-down list, which includes records from custaddr.

6 The user selects customer X, and some information for customer X from the custaddr table is filled in. The rest of the customer data must be supplied by the user at Site B.

7 The user saves the new record, which adds customer X to the customer table at Site B. This also updates the customer_all table at Site B.

8 (Not shown on the flowchart) Any information that is changed on the custaddr table at Site B is replicated back to Site A. Also, a new customer_all record with site_ref = Site B is created at Site A.

When SQL Server Shuts Down

When Microsoft SQL Server is shut down, the Replicator is unable to perform its SLShadowValues polling loop since it cannot access the database. After a failure, the Replicator service attempts to reconnect with the SQL server.

Shared Tables

How It Works

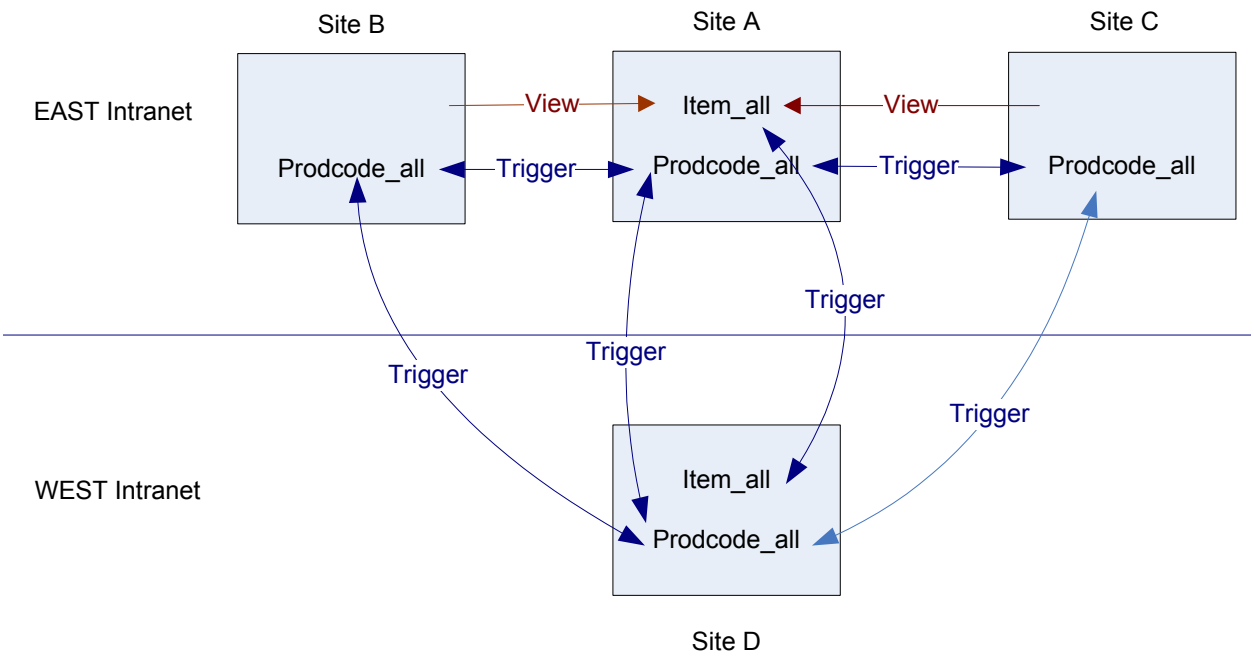
When an _all table is stored at a master site, the "using" sites:

- Do not have an actual _all table in their databases
- Use (read, maintain, etc.) the master site's _all table through views
- Have no replication trigger source generated for that _all table

The master site for an _all table has no replication trigger source generated for that _all table to the other sites in its Intranet.

Example

You have three sites in the East intranet (A, B, and C), and another site (D) in the West intranet. Site A is the master site in the East intranet for the shared **item_all** table. No other tables are shared in the East intranet. Data in the **prodcode_all** table is replicated between all four sites. Site D requires replication between its item_all data and the item_all data from all sites in the East intranet.



Item_all: Sites B and C have views into Site A's item_all table. No replication triggers are set up for the item_all table between A, B, or C. However:

- Site A has replication triggers to replicate the item_all table to site D.
- Site D has replication triggers to replicate the item_all table to site A (the master site of the East intranet, which holds all the item_all data for the intranet).

Prodcode_all: Replication triggers for prodcode_all data are set up between each of the sites:

- Site A has replication triggers to replicate the prodcode_all table to sites B, C, and D.
- Site B has replication triggers to replicate the prodcode_all table to sites A, C, and D.
- Site C has replication triggers to replicate the prodcode_all table to sites A, B, and D.
- Site D has replication triggers to replicate the prodcode_all table to sites A, B, and C.

Details

The following sections describe what happens within the system when you perform tasks relating to an intranet with a master site and shared tables:

- Setting up a master site
- Sharing an _All table
- Unsharing a shared _All table
- Sharing user tables
- Unsharing user tables
- Adding a new site to a shared intranet

- Regenerating replication triggers
- Regenerating views to the master site
- Maintaining data for multiple sites from the master site
- Setting up replication to sites in other intranets

In addition, the stored procedures used with shared tables are described here.

Setting Up a Master Site

If defined for an intranet, the master site database contains any shared _all tables. When you enter a master site on the **Intranets** form, the system validates whether all of the following are true:

- The selected site is in the currently selected intranet.
- The selected site is the site where you are logged in.
- Live links are set up between this site and the other sites in the intranet.

When a master site is specified for an intranet, the IntranetSharedTable table is populated in the master site's database with the list of sharable _all tables. The system adds records only for _all tables where triggers do not exist that are not replication triggers; for example, shipcode_all would not be included in the list of sharable tables, since it has a trigger to make data integrity type checks.

If the master site value for this intranet is later set to blank (deleted), the _all table records for that intranet in IntranetSharedTables are also deleted. Each record in the table contains the intranet name, table name, and flags indicating whether the table has been shared or processed. (The columns map to the fields on the **Intranet Shared Tables** form.)

If (as recommended) you have defined rules to replicate the system administration category from the master site to all the using sites in the intranet, those using sites will have a copy of the IntranetSharedTable table.

Sharing an _All Table

After you select and save the **Shared** setting for a currently unshared _all table on the **Intranet Shared Tables** form, click **Process** to start the process:

1 The system runs IntranetValMasterSiteLinksSp to validate the live link setup between the master site and the using sites of the intranet.

2 If the previous step produces no validation errors, the system runs IntranetProcessSharedTablesSp to get the appropriate table and view configuration setup by doing the following at each of the using sites:

   - Drop the _all table in this using site if it exists.
   - If the _all table does not exist in the master site, create it.
   - Create a view in the using site's database to the master site's _all table.

- Check whether the master site's _all table has data from the using sites' base table. If not, populate the master site _all table from the using site's base table.

3 If the previous step is successful, the system determines the list of using sites for this intranet and loops through their servers/databases. It calls the RegenerateReplicationTriggers method, passing the server and database names. The **Processing Step** field on the form indicates each site where trigger regeneration is currently being performed.

4 The system calls RegenerateReplicationTriggers again, passing a NULL value for the server and database name parameters.

This regenerates this master site's replication triggers.

Unsharing a Shared _All Table

After you unselect the **Shared** setting for a currently shared _all table on the **Intranet Shared Tables** form, click **Process** to start the process:

1 The system runs IntranetValMasterSiteLinksSp to validate live link setup between the master site and the using sites of the intranet.

2 If the previous step produces no validation errors, the system runs IntranetProcessSharedTablesSp to get the appropriate table and view configuration setup by doing the following at each of the using sites:

- Drop the _all view in this using site.
- Create the _all table in the using site's database, using the table definition from the master site.

  The system uses TableScriptSp to create the script for the table definition from the master site. It stores this information in variables and calls IntranetRemoteScriptSp to execute the script that populates the _all table in the using site.

3 If the previous step is successful, the system determines the list of using sites for this intranet and loops through their servers/databases. It calls the RegenerateReplicationTriggers method, passing the server and database names. The **Processing Step** field on the form indicates each site where trigger regeneration is currently being performed.

4 The system calls RegenerateReplicationTriggers again, passing a NULL value for the server and database name parameters. This regenerates this master site's replication triggers.

Sharing User Tables

After you select **Set up shared user tables** on the **Intranet Shared User Tables** form, click **Process** to start the process:

1 The system runs IntranetValMasterSiteLinksSp to validate live link setup between the master site and the using sites of the intranet.

2 If the previous step produces no validation errors, the system runs IntranetProcessSharedUserTablesSp to set up the table and view configuration by doing the following at each of the non-master sites, for all of the sharing user tables:

- Validate that the UserNames and GroupNames tables exist in the site application database. If not, the site is already sharing the user tables and processing can continue to the next site. If tables are already shared, make an additional check to see if UserGroupMaps and/or AccountAuthorizations tables have been selected as "not shared." If so, copy those tables back to the each non-master site if they are not already there.
- Copy any records in the UserNames table to the master site's UserNames table where the UserName does not exist in the master site. If a UserId does not exist in the master site, the system generates a new UserId in the master site.
- Copy any records in the GroupNames table to the master site's GroupNames table where the GroupName does not exist in the master site. If a GroupId does not exist in the master site, the system generates a new GroupId in the master site.
- Create temporary tables #GroupIdMap and #UserIdMap to contain old and new GroupId and UserId values for GroupNames and UserNames records that were copied to the master site.
- Update UserId and/or GroupId values in the non-master site on all tables specified in the lower grid where the UserId or GroupId column value was updated in the master site.
- Copy any records from each shared user table to the matching user table at the master site, where the Primary Key values do not already exist in the master site's table.
- Remove the shared user tables from the non-master sites in the intranet and set up SQL views from those sites into the master site.

3 If the previous step is successful, the system determines the list of using sites for this intranet and loops through their servers/databases. It calls the RegenerateReplicationTriggers method, passing the server and database names. The **Processing Step** field on the form indicates each site where trigger regeneration is currently being performed.

4 The system calls RegenerateReplicationTriggers again, passing a NULL value for the server and database name parameters. This regenerates this master site’s replication triggers.

Unsharing User Tables

After you select **Set up per site user tables** on the **Intranet Shared User Tables** form, click **Process** to start the process:

1 The system runs IntranetValMasterSiteLinksSp to validate live link setup between the master site and the using sites of the intranet.

2 If the previous step produces no validation errors, the system runs IntranetProcessSharedUserTablesSp to do the following for each of the shared user tables at each of the non-master sites:

- Drop the view for the shared table.
- Create the user tables and associated triggers.

- Copy the data from the master site back to the non-master sites. For the UserNames table, the system copies UserId values into the non-master site's UserName table as-is, so that other records that reference the UserIds do not need to be adjusted.
- Copy [table-name]Iup and [table-name]Del triggers, if the exist in the master site, to the non-master site.

3 If the previous step is successful, the system determines the list of using sites for this intranet and loops through their servers/databases. It calls the RegenerateReplicationTriggers method, passing the server and database names. The **Processing Step** field on the form indicates each site where trigger regeneration is currently being performed.

4 The system calls RegenerateReplicationTriggers again, passing a NULL value for the server and database name parameters. This regenerates this master site’s replication triggers.

Adding a New Site to a Shared Intranet

When you change the intranet of a site, the system first determines whether the selected intranet has shared _all tables. If it does, the system:

1 Validates that the user is logged into the site whose intranet value is being changed. (If not, the user cannot change the site value.)

2 Drops this site's tables that are shared in its new intranet.

3 On this site, adds views to the new master site for tables that are shared in its new intranet.

4 Populates the master site's _all table(s) with data from the base table at this site, if the data does not already exist at the master site.

5 Runs the Regenerate Replication Triggers function at all sites in the new intranet.

If the new intranet is sharing an _all table, this site looks at different data than it had prior to changing its intranet value.

In order to get replication triggers properly defined, and data properly replicated, you might need to also perform steps at other sites in this site’s old/new intranets. For example, re-examine replication categories and rules that replicate data to and from this site and its old/new master sites, and run **Regenerate Replication Triggers** if you change any of the information.

Shared user tables are not addressed automatically by the system when you add a new site. When you are already sharing user tables, and you add a new site to the current master site's intranet, you must set up the new site to share user tables, as described in the online help.

Regenerating Replication Triggers

The RegenerateReplicationTriggers method includes parameters to pass a server name and database name. If this information is provided, the method uses the server and database name to build the SQL strings and regenerate the replication triggers at that site and database (that is, remotely).

The logic takes table sharing into account:

- If a using site on an intranet contains a view to a shared _all or user table, no replication triggers should be generated for that table with the using site as a source site to any other site as a target site.
- If a site has a replication category for a table that exists in its database, replication triggers for that table should only be created to target sites that are not using sites of that table. That is, there should be no replication triggers from the master site to the using site for shared _all or user tables. Also, there should be no replication triggers from sites in other intranets to using sites for tables that are shared on the using site's intranet.

Regenerating Views to the Master Site

The **Regenerate Views to Master Site** button on the **Replication Management** form synchronizes the using sites' view definition with the master site's table definition for any shared _all or user tables. This button is enabled when the site where the user is logged in is the master site for the intranet, and at least one table is shared in that intranet (an IntranetSharedTables record exists with Shared = 1 and Processed = 1, *or* Shared = 0 and Processed = 0).

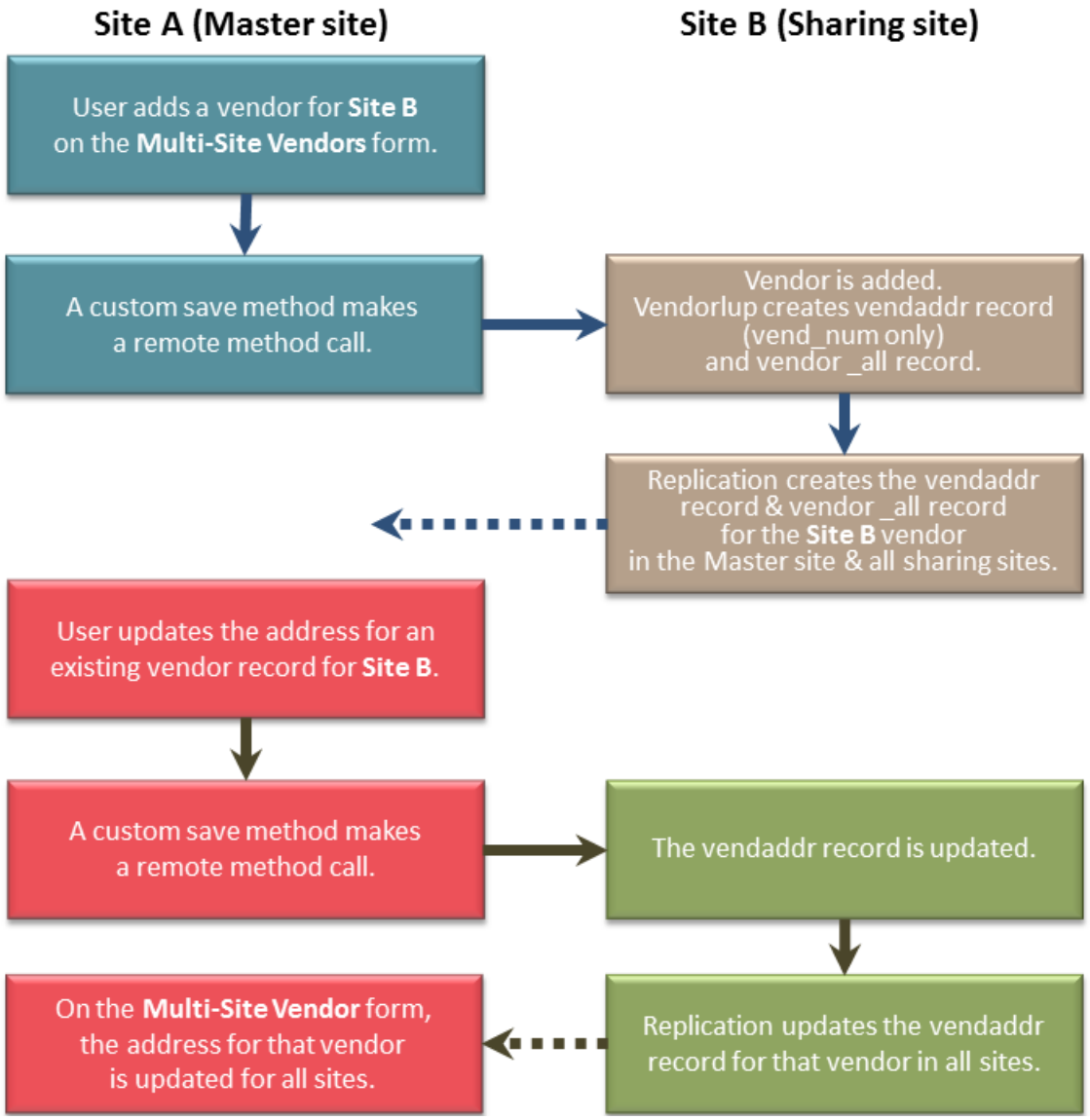
When you click this button, the application runs the RecreateUsingSiteViewsSp method described on page 64 and returns a success or failure message.

Maintaining Customer, Vendor or Item Data for Other Sites in the Intranet

Special forms are available only at the master site that allow you to add and maintain some vendor, customer or item data for other sites on that intranet. This section shows examples for vendor data only.

Vendor data includes the vendor_all table/view (storing information that is not typically shared among sites), as well as the vendaddr tables (storing information that is shared among all sites). The vendaddr base table is replicated directly from the base table in the source site to the base table in the target site. In a master site, vendor_all is a view at the sharing sites (assuming that vendor_all is set up as a shared table), but vendaddr is an actual table that contains the same data at the master site and all sharing sites.

The process for adding and updating vendor data between the master site and sharing sites is similar to this:



Setting up Replication to Sites in Other Intranets

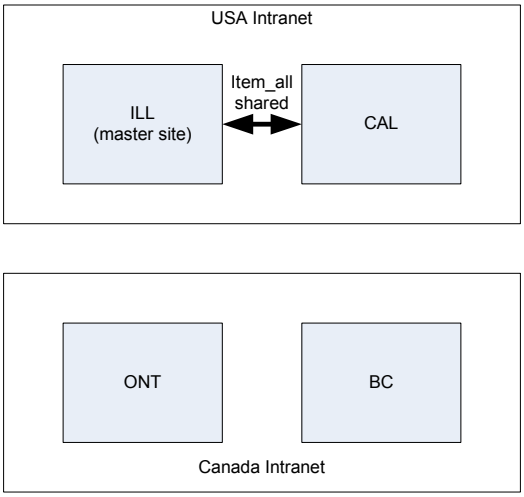
If there are other intranets with sites that want to replicate (not share) _all table data to/from sites in the sharing intranet:

- For tables that are shared, set up replication categories/rules between the master site and the sites on the other intranets. (You probably will also need to set up the same categories/rules between the specific shared site and the sites on other intranets; see the following example.)

- For tables that are not shared, set up replication categories/rules between any/all of the sites in the shared intranet and the sites on the other intranets.

Example

In the setup shown below, the following are true:



- If CAL needs visibility into BC's item data, replication rules for a category containing item_all should be set up from BC to ILL.
- If BC needs visibility into CAL's item data, replication rules for a category containing item_all should be set up from ILL to BC.
- If BC and CAL interact in other ways, for example order entry of an item in BC and shipment of the item from CAL, you still need centralized order entry replication rules between CAL and BC, since additional tables and stored procedures in that category are needed to perform these functions. The item_all table, although it exists in the category, will not be replicated from CAL to BC.
- If ONT needs visibility into CAL's customer data (which is not a shared table), replication rules for a category containing customer_all should be set up from CAL to ONT.

Stored Procedures Used with Shared Tables

For more information about shared tables, see “Shared Tables” on page 55.

GetMasterSiteSp

When given a site name, this routine finds the intranet for that site and returns the master site from the Intranet record for that site.

IntranetProcessSharedTablesSp

This routine puts a new shared table configuration in place for the intranet when the user clicks **Process** on the **Intranet Shared Tables** form. Input parameters include the intranet name and table name.

IntranetProcessSharedUserTablesSp

This routine takes two input parameters, the intranet name and the name of a non-master site on the intranet. This routine handles setting up shared user tables or per site user tables for the specified non-master site and the master site on the specified intranet. This routine gets called for every non-master site on the intranet.

IntranetRemoteScriptSp

This routine allows a script to be run at the using site in an intranet with shared tables. This stored procedure is used by other stored procedures.

IntranetValMasterSiteLinksSp

This routine validates live link setup between the master site and the using sites of an intranet. Input parameters include the intranet name and the master site name.

RecreateUsingSiteViewSp

When given an intranet name, this routine gets the master site from the Intranet record, then gets the master site's Database Name value, executes IntranetValMasterSiteLinksSp, finds any shared _All tables at the master site, drops the same _All table in the using site, and recreates in using sites a view to the _all table at the master site.

ResetIntranetSharedUserTablesFromDefaultSp

This routine removes all records from the IntranetSharedUserTable table and populates it with records from IntranetSharedUserTableDefault.

UpdateMasterSiteAllTableSp

This routine updates a master site's _all table with records from a using site in an intranet with shared tables. Input parameters include the master site and the table name.

UserNamesRemoteUpdateSp

This routine adds code to verify that the UserNames table exists in the current site. If not, return and do nothing.

Chapter 5: Replication Using Infor BODs

5

Overview

SyteLine uses XML messaging to integrate with Infor applications and with some outside applications. It supports exchanging documents or messages among multiple applications.

The messaging architecture is based on the BOD Message Architecture, as defined in the Open Applications Group Integration Specifications (OAGIS). Data sent to many other Infor applications is sent as a Business Object Document (BOD). A BOD is an XML document composed of a verb, which identifies the action to take with the document, and a noun, which identifies the document content.

SyteLine replication logic is used to create BODs and place them in an outbox table.

Replication Document Definitions

Outbound Business Object Document (BOD) XMLs are created based on metadata that is defined in SyteLine forms. This metadata also defines the processing to be performed in SyteLine on receipt of inbound BODs.

BOD elements are often mapped to objects in tables or collections, although certain elements can be specified as literals or calculated values that are not tied to any table or collection. All of this is defined in the metadata.

- The **Replication Documents** form specifies the general structure and conventions used by the BOD.
- The **Replication Document Outbound Cross References** form specifies when and how to create a BOD triggered by an action taken in your application.
- The **Replication Document Inbound Cross References** form specifies what actions to take in your application when an inbound BOD is received.
- The **Replication Document Elements** and **Replication Document Attributes** forms define BOD elements (tags) and the attributes of those elements, and map the elements and attributes as required.

It is possible to create your own BODs or copy and then modify existing ones. You should not do this unless you are very familiar with SyteLine and with the processes and applications used in the Infor Service Oriented Architecture (SOA).

Defining Inbound and Outbound Bootstrap Sites for an Intranet

Inbound

In a multi-site environment, you designate one configuration whose application database is the entry point for BODs sent from other BOD-enabled applications for all sites on the intranet. (That is, the replication document inbox on the site is expected to receive inbound BODs for all sites on its intranet.) The Infor Framework Inbound Bus Service uses this site's replication document rules and site definitions for processing all incoming BODs. This site is designated through the **Inbound Bus Configuration** field in the Service Configuration Manager utility.

Outbound

In a multi-site environment, one site per intranet collects in its replication document outbox all outbound BODs generated by sites on that intranet. That site is designated through the **Configuration** field in the Service Configuration Manager utility.

BOD Loader Definition

In order to pick up BODs from the bootstrap site's outbox, the Infor BOD Loader must be configured so that its database resource is set to the site's application database.

Setting up Generation of Outbound BODs through Replication

Generating BODs

To create BODs, you must set it up like this:

1 Use the **Intranets** form to create a new intranet specifically for the transmission of BODs.

2 Use the **Sites** form to specify a logical ID for the local site.

Logical IDs are used to identify the originator and receiver of messages from Infor BOD-enabled applications as they travel through the system. (The logical ID is needed for both inbound and outbound BOD transmission.)

3 Use the **Sites** form to create a logical "site" for the intranet created in Step 1.

4 Use the **Replication Rules** form to create a new rule using the **ESB** category, with the target site set to the logical site you created in the previous step.

5 Regenerate replication triggers.

Detailed steps for this process are included in the online help.

The next time a user action triggers BOD generation, the BOD XML is created automatically and placed in the Replication Document Outbox. The user actions that trigger each BOD are listed in the **Replication Document Outbound Cross References** form.

Generating Specific BODs Using a Utility

In some cases (for example, when initializing an integration), you might need to manually generate certain BODs. Use the Replication Document Manual Request Utility to do this.

Processing Outbound BODs

These are the steps the system goes through to generate a BOD.

1 SyteLine uses its replication engine to generate every BOD.

Two separate processes can trigger BOD generation:

- A business event occurs in the system that triggers a remote method call;
  OR
- A change is made to a table that triggers an UpdateCollection request.

  Each of these processes is considered a BOD generation integration point. The user actions that trigger each BOD are listed in the **Replication Document Outbound Cross References** form.

  Example to illustrate the first process: A user runs the **Purchase Order Report**. As a part of the logic executed, a remote method call is made to the stored procedure TriggerPurchaseOrderSyncSp, which triggers BOD generation replication logic.

  The remote method call can pass parameters by way of the replication document forms. These are referred to by sequence; that is, P1 refers to the first parameter, P2 to the second

parameter, and so on. The following code snippet shows the two parameters passed by a remote method call to TriggerPurchaseOrderSyncSp:

```
EXEC @Severity = dbo.RemoteMethodForReplicationTargetsSp
 @IdoName = 'SL.ESBSLPos'
 , @MethodName = 'TriggerPurchaseOrderSyncSp'
 , @Infobar  = @Infobar OUTPUT
 , @Parm1Value = @PoPoNum
 , @Parm2Value = @ActionExpression
```

2 In the **Replication Categories** form, a default category called **ESB** must be created that contains all the defined remote method calls, or Object Names referred to by replication documents, for integration points to other BOD-enabled applications.

3 If a replication rule exists between a SyteLine site and an Infor ESB site where the category is set to **ESB**, then whenever a business event or table change with an integration point occurs at the SyteLine site, the system adds an entry in the Replication tables.

4 The Replicator picks up rows from the Replication Tables and places them in MSMQ.

5 ReplQListener picks up the rows from MSMQ and determines that these are rows bound for another BOD-enabled application. It then generates a BOD for the business event or table change. As mentioned above, parameters might be passed by the event.

In our example, the user runs the **Purchase Order Report**, which calls TriggerPurchaseOrderSyncSp. Since a rule exists for a category where this stored procedure is defined, the system builds a replication document. It finds a replication cross reference for PurchaseOrder.Sync where the Applies To method is TriggerPurchaseOrderSyncSp, so it retrieves the PurchaseOrder replication document template and builds the BOD XML document, using the noun and verb from the **Replication Document Outbound Cross Reference** form, plus element and attribute information defined in the **Replication Documents**, **Replication Document Elements**, and **Replication Document Attributes** forms.

Also in this example, the purchase order number is passed as the first parameter (P1). The replication document uses this information in a filter (PoNum=FP(P1)) so that the generated BOD contains data for only the specified purchase order.

6 ReplQueueListener places the generated BOD XML in the Replication Document Outbox on the bootstrap site, where the BOD can be found and processed.

Processing Inbound BODs

BODs from other applications are routed to a SyteLine instance. The target site is indicated in the "ToLogicalId" element of the BOD. The inbound BOD is placed in the Replication Document Inbox table at the bootstrap site for the intranet.

A Windows service, the Infor Framework Inbound Bus Service, monitors the inbox at the bootstrap site. When a BOD appears in the inbox, the service transforms it according to the rules defined for the inbound BOD in the replication document metadata at the bootstrap site. The service then executes

the resulting request in the target site. If the target site is **lid://default**, the request is executed in all BOD-enabled sites on the mater site's intranet.

Any trace and error messages are logged and viewable through the Log Monitor, and error messages are displayed in the Windows application event viewer.

For a sample and brief tutorial on how SyteLine integrates with ION to process inbound BODs, see Appendix A, “Tutorial: Processing Inbound BODs."

Chapter 6: Setting Up Custom Replication

6

Custom Replication Categories

If the standard replication categories do not meet your needs, you might need to set up your own categories, or modify the existing ones. If you plan to customize replication on your system, creating your own categories and rules, consider the following:

Caution: If you choose to create new categories, we strongly recommend that you get help from Infor Consulting Services. Determining all the relationships between tables and stored procedures is not a simple task.

- You must understand the relationships of the data you want to move between systems.

  You should determine whether a table you want to replicate has sibling tables or child tables (for example, the item table has child tables for item location, item/warehouse, and so on). The DataMap or the IDO editor forms in the application may be helpful in determining this.
- When designing replication categories/rules, think about the sequence of events—what you need to have happening on the target side, and in what order. (This determines what you include in your replication triggers.) Do you need to write stored procedures in addition to replicating table data?

_All Table Considerations

_All tables include the site as part of the key. Unlike many base tables, the _all tables do not include foreign keys to other tables in the system. So, when you create a new replication category, if you can include the _all tables instead of the base tables, you will be less likely to have problems with missing data.

Also, if you create a new module with a set of new tables and you plan to replicate some or all of the new data, you should think about using a base table containing the data for the local site, plus an _all table containing the data for all sites. This will simplify your replication process as explained above.

Be aware of the limitations of using _all tables for replication. There are no data validation triggers on _all tables. Not all data from the base table may be contained in the _all table. For example, the item table contains the local site's items and data about those items. The item_all table contains information only about the items that are available for all sites. So not everything about every item is replicated at the target site, since it takes data from item_all.

There might be some simple base tables (for example, some of the code tables) that can be replicated directly because they do not contain many foreign keys or dependencies.

Replicating Tables and Columns Added with the SQL Table and SQL Columns Forms

When you use the **SQL Tables**, **SQL Columns**, and **SQL Table Primary Key** forms to create custom tables, the new tables are set up with the appropriate columns, indexes, and triggers so that they are treated by the system just like any base table. If you set up the same custom table in multiple sites, you can include the table in replication categories and create replication rules between those sites. The table schema must be exactly the same at each site.

You can use the **SQL Columns** form to add custom columns to base tables. If the base table has an _all table, you can also add the new columns to the _all table in order to replicate the data. If you do this, add the columns to both the base table and the _all table in each site that is replicating the _all table data.

Adding UET Schema Changes to _All Tables

If you modify a base table schema using the User Extended Tables (UET) function, then the same UET columns/indexes will automatically be added to the corresponding _all table. Impacting the schema through the UET Impact Schema utility automatically regenerates the table triggers for the base table to populate the new column(s) in the corresponding _all table at the current site.

Then go to the **Replication Management** form and click the **Regenerate Replication Triggers** button. This recreates the appropriate replication triggers and stored procedures based on the latest table schema changes.

Note: Replication triggers must be regenerated any time UETs are changed, added, or deleted.

If an _All Table is Shared

If an _all table is shared in an intranet, the _all table resides on the master site, and UET fields should be changed there. After making the change, go to the **Replication Management** form on the master site, and click the **Regenerate Views to Master Site** button.

Copying UET Table Schema Changes

If one site contains UET modifications to a table, but the same table in another site does not contain those modifications, errors probably will be generated during replication of that table's data from one site to the other.

For that reason, before you replicate data between sites, you should copy any UET schema changes to all sites involved in the replication. A stored procedure, ExportUETClassSp, assists in handling this process. When you run ExportUETClassSp on Site A, you create a SQL script file that you can apply to Site B so that the table schemas on both sites are identical. The general steps to copy the UET schema changes are:

1 Disable replication at all sites.

2 At the site with UETs, run ExportUETClassSp, which creates several scripts.

   For instructions to run ExportUETClassSp, see the document *Modifying SyteLine*, the chapter on "Scripting UET Definitions." That chapter also includes the text used to create the stored procedure, which you can copy into an SQL query.

3 At each of the other (target) sites that will replicate data to/from the site with UETs, apply the scripts generated in step 2.

   Again, for instructions to run these generated scripts on the target sites, see the document *Modifying SyteLine*, the chapter on "Scripting UET Definitions."

4 At each target site, run the SyteLine activity **UET Impact Schema**.

5 Enable replication at all sites.

6 Regenerate replication triggers at all sites.

Maintaining Custom Schema Metadata

When you create a custom table or a custom column on a standard table, you can use the Application Schema Metadata forms to define metadata (for example, table triggers, alpha keys, and UET inheritors). You can also dump the metadata into SQL scripts that can then be loaded into other sites where the same table/column changes are being made. For more information, see the online help.

Setting Up Non-Transactional Replication for Sibling Tables or Subsets of Tables

In the **Replication Categories** form, you can set up insert, delete, and update operations from one or more sibling tables and send them through non-transactional replication to a single output table.The generated replication code uses columns in the categories grid to decide what table to send data to, and what view name to use for inserts.

The view must always contain the **RowPointer** and primary key columns.

When an update occurs to a base table, and some columns from that base table are included in a view defined in **Replication Categories**, the view is checked to see if any of the matching columns in

the view have been updated. If only updates to columns not in the view occur, then nothing is replicated.

Example of Combined Sibling Table View

To join the customer and custaddr tables together into a single SuperCustomer output table, do the following:

1 Make a view called **SuperCustomerView** that contains a **RowPointer** column tied to the custaddr.RowPointer.

2 Make a new category with the following 2 object category rows:

Grid Column	Object Row 1	Object Row 2
Object Name	customer	custaddr
Object Type	TABLE	TABLE
Filter		
Skip Insert	Yes	
Skip Update		
Skip Delete	Yes	
Skip Method		
To Object Name	SuperCustomer	SuperCustomer
Insert From View	SuperCustomerView	SuperCustomerView

The insert view name is needed on the customer object, even though no insert actually occurs on INSERT, because the view is used to determine which columns being updated need to be replicated.

After you regenerate the replication trigger code, the following scenarios would occur.

On an INSERT of customer:

- The customer record is inserted.
- The customerIup trigger inserts a custaddr record.
- The custaddrIupReplicate trigger inserts ShadowValues row to SuperCustomer and selects columns from SuperCustomerView.
- The customerIupReplicate trigger does not send an insert.

On an UPDATE of a customer:

- The customer record is updated.
- The customerIupReplicate trigger sends the update to SuperCustomer.

On an UPDATE of custaddr:

- The custaddr record is updated.
- The custaddrIupReplicate trigger sends the update to SuperCustomer.

On a DELETE of a customer:

- The customerDel trigger deletes the custaddr record.
- The custaddrDelReplicate trigger sends the deletion to SuperCustomer.
- The customerDelReplicate trigger does not send a deletion.

Executing Stored Procedures at Other Sites

To run a stored procedure at a remote site, you have these options, depending on your needs:

- Use RemoteObjectName to build a stored procedure name and then execute the stored procedure when:
  - You are guaranteed to be linked to the other site; that is, the other site's application database is either on the same SQL Server or is on a SQL Server that is linked to the current application database's SQL Server;

    AND
  - You want to make the call transactionally, that is, synchronously.

  If output parameters must be returned from the synchronous call before proceeding, use this option.
- Use RemoteMethodCallSp to execute a stored procedure when:
  - The two sites are connected using either transactional or non-transactional replication;

    AND
  - No output parameters, other than the status, need to be returned to the calling program.

  In this case, the stored procedure to be called must be included in a replication category, and a replication rule must exist from the calling site to the target site for that category.
- Use RemoteMethodforReplicationTargetsSp, which works just like RemoteMethodCallSp, to execute the stored procedure at all sites where a replication rule exists from the current site to the remote site for the category that contains the stored procedure.

RemoteObjectName

Your custom program can call the RemoteObjectName function, which takes as arguments the site and the name of the stored procedure to call. This function builds and then runs the stored procedure on the remote, linked site. Any input or output parameters are placed with the EXEC @SpName statement.

For example:

```
DECLARE @SpName SYSNAME
SET @SpName = dbo.RemoteObjectName (
'MI'
, 'mySp'
)
EXEC @SpName
```

The more detailed example below, from application code, shows the execution of CLM_ApsGetWaitSp at a site designated by the parameter @PSite. First RemoteObjectName validates whether the information required to build the full path to a stored procedure of that name on that site exists. If no, it returns a null value and the code must perform error handling. If yes, the function returns the path to the stored procedure, which the code then executes, passing the parameter @POrder to it:

```
SET @SProcName = dbo.RemoteObjectName(@PSite, 'CLM_ApsGetWaitSp')
  IF @SProcName IS NULL
   BEGIN
   SET @Severity = 16
   RETURN @Severity
  END
EXECUTE @Severity = @SprocName '', @POrder
```

When using the RemoteObjectName function, you need not set up a replication category or replication rules. However, in order for site A to run the stored procedure at site B, the **Sites** form at Site A must show the appropriate link information and application database name for the current site and for site B.

RemoteMethodCallSp

Your custom program can call the stored procedure RemoteMethodCallSp, which takes as arguments the name of the site, the name of the stored procedure at the remote site, and any parameter values to pass to the stored procedure.

For example:

```
EXEC @Severity = RemoteMethodCallSp
  @Site = @ToSite
, @IdoName = N'SL.SLObjectNotes'
, @MethodName = N'LoadReplicatedNotesSp'
, @Infobar = @Infobar OUTPUT
, @Parm1Value = @RowPointer
```

For the calling site to run the stored procedure at the target site, the stored procedure must be included in a replication category that is used in a replication rule from the calling site to the target site.

A transaction can trigger your custom code, or the code can be built into a form. In either case, your custom code calls RemoteMethodCallSp to queue the stored procedure for execution at the remote site. Control then returns to the calling program.

If the replication category containing the stored procedure is included in a transactional rule, the stored procedure runs immediately on the remote site. The remotely called stored procedure is expected to use the RemoteInfobarSaveSp routine to return any error message. If the remotely called stored procedure is not found, RemoteMethodCallSp returns an error message indicating an improper setup of replication rules.

For non-transactional rules, the request is stored in the calling system as an XML request and is sent to the remote system after the interval specified in the rule. The request runs the stored procedure on

the remote system. In this case, a success status returned from RemoteMethodCallSp merely indicates the call was successfully saved for later processing.

If the **To Site** on the replication rule is the same as the local site, RemoteMethodCallSp runs the stored procedure locally.

Appendix A: Tutorial: Processing Inbound BODs

A

This appendix is a tutorial intended to provide Mongoose Framework developers and implementers with an introduction to how the framework integrates with ION to process inbound BODs.

In this tutorial, we will be populating tables that represent simplified sales order headers and lines, from a BOD named SyncSalesOrder.

Prerequisites

To complete this tutorial, you should have access to Mongoose Framework version 7.01 or later. We recommend that you follow the steps in this tutorial in order, to create the tables, IDOs, replication documents (BOD metadata), and .Net assembly (DLL) necessary to successfully process the sample SyncSalesOrder BODs referenced by this tutorial.

Using the IDO Runtime Development Server

As when doing any development with the Mongoose Framework, it is helpful to use the IDO Runtime Development Server (IDORuntimeHost.exe) instead of the IDO Runtime Service. If the service is running on your development machine, you can either stop it, or run IDORuntimeHost.exe with the "/multiuser" command line parameter.

Setting Up

To set up your system for this tutorial:

- Verify that your configuration name matches the **Site** name in your application database. To view the **Site** name, log in and open the **System Parameters** form, **Address** tab.

The **Site** name there must match the configuration name shown in the Configuration Manager *exactly*. If the name does not match, use the Configuration Manager (**Copy** button) to create a configuration with the correct name.

If you add or change a configuration, then:

a. Select the **Utilities** tab (in the Configuration Manager).

b. Select the **IDO Runtime on local machine** check box.

c. Click **Discard IDO Cache**.

Note: In fact, any time you add or change a configuration on your computer, you should verify through the Configuration Manager that these settings have been made. If you do not select the **IDO Runtime on local machine** option and discard the IDO cache here, errors can result.

- Make a note of your site's bus logical ID. To do this:

  a. Open the **Sites** form.

  b. In the left-hand grid, select the record that corresponds with your site.

  c. On the **System Info** tab, locate the **Message Bus Logical ID** field and take note of the field's contents.

  You will use this information later, when setting up the test environment.

Creating the Table Schema in the Application Database

To create the table schema in the application database:

1. Create the tables in the application database. To do this:

   a. Log in, if you are not already logged in to the system.

   b. Open the **Sql Tables** form.

   c. To cancel Filter-In-Place, press F3.

   d. To create a new table, press CTRL+N.

   e. From the **Schema** drop-down list, specify **dbo**.

   f. In the **Table Name** field, specify **SalesOrder**.

   g. Save the record.

   h. Repeat Steps d through g for a new table named **SalesOrderLines**.

2. Specify the columns for the new tables. To do this:

   a. Select the **SalesOrder** table that you just created.

   b. Click **Columns**.

The **Sql Columns** form opens with several predefined columns.

c. Add new columns with these specifications:

Column Name	Data Type	Length	Decimal Position
SalesOrderID	nvarchar	50	
AccountingEntity	nvarchar	50	
CustomerID	nvarchar	50	
TotalAmount	Decimal	21	8
TotalAmountCurrency	nvarchar	20	

d. Save the new columns and close the **Sql Columns** form.

e. On the **Sql Tables** form, verify that the **SalesOrder** table is still selected and click **Primary Key**.

f. In the **Sql Tables Primary Key** form, add **SalesOrderID** to the **Primary Keys** list.

g. Click **OK**.

h. Repeat steps a through g for the **SalesOrderLines** table, adding new columns with these specifications:

Column Name	Data Type	Length	Decimal Position
SalesOrderID	nvarchar	50	
LineNumber	nvarchar	20	
ItemID	nvarchar	50	
QuantityValue	Decimal	21	8
QuantityUOM	nvarchar	20	
PromisedDeliveryDateTime	datetime		

For the primary keys, add **SalesOrderID** and **LineNumber**.

Creating the IDOs

Now that the table schema is ready, the next step is to build IDOs based on the two new tables. These IDOs will be used to map data in the BOD to columns in our tables.

To create the IDOs:

1. Open the **IDO Projects** form.
2. To create a new project, press CTRL+N.

   An IDO project is essentially a convenient grouping of IDOs.

3. In the **Project Name** field, specify **SalesOrderTutorial**.

   You can also provide an optional description if you want.
4. Save the project.
5. With the new project selected, click **New IDO**.
6. On the first page of the **New IDO Wizard** form, specify these values:
   - **Primary Base Table** – **SalesOrder**
   - **Table Alias** – **slsord**
   - **IDO Name** – **SalesOrders**
7. Click **Next** and then (on the second page of the wizard) click **Finish**.

   The new IDO appears on the **IDOs** form.
8. Close the **IDOs** form and return to the **IDO Projects** form.
9. With your project still selected, click **New IDO**.
10. On the first page of the **New IDO Wizard** form, specify these values:
    - **Primary Base Table** – **SalesOrderLines**
    - **Table Alias** – **slsordlin**
    - **IDO Name** – **SalesOrderLines**
11. Click **Next** and then click **Finish**.

    Again, the new IDO appears on the **IDOs** form.
12. Close the **IDOs** form and return to the **IDO Projects** form.
13. With your project still selected, click **IDOs**.

    The **IDOs** form opens, and you should see both of the new IDOs.
14. On the **IDOs** form, select the **SalesOrders** IDO and then click **New Property**.
15. On the first page of the **New Property** wizard, select **Subcollection** and then click **Next**.
16. On the second page of the **New Property** wizard:
    - For the **Subcollection IDO**, specify **SalesOrderLines**.
    - For the **Property Name**, specify **Lines**.
17. Click **Finish**.

Mapping BOD Elements and Attributes to IDO Properties

In the Mongoose Framework, the mapping of BOD elements and attributes to IDO properties is accomplished using replication documents. Replication documents consist of metadata that defines

the rules for both generating outbound BODs and transforming inbound BODs. For this tutorial, we will not address the outbound BOD generation, which will simplify the metadata that is required.

Creating the Replication Document

To create the **SalesOrder** replication document:

1. Open the **Replication Documents** form and press F3 to cancel Filter-In-Place.
2. Press CTRL+N to create a new document.
3. Specify these values:
   - **Document Name** – **SalesOrder**
   - **IDO** – **SalesOrders**
4. Save the document.

Creating the Header Mappings

The next step is to add the metadata that maps the BOD elements and attributes to IDO properties. For the **SalesOrder** header, the end-result of the mapping is shown in this table:

IDO Property	BOD XPath
CustomerID	/SalesOrder/SalesOrderHeader/CustomerParty/PartyIDs/ID/text()
AccountingEntity	/SalesOrder/SalesOrderHeader/DocumentID/ID/@accountingEntity
TotalAmountValue	/SalesOrder/SalesOrderHeader/TotalAmount/text()
TotalAmountCurrency	/SalesOrder/SalesOrderHeader/TotalAmount/@currencyID

To create the header mappings:

1. Click **Elements**.
2. Add new element entries, using these values:

Sequence	BOD Tag Name	Value Type	Property Name
10	BODVERB()BODNOUN()/DataArea/BODNOUN()/BODNOUN()Header/CustomerParty/PartyIDs/ID	Property Tag	CustomerID
20	BODVERB()BODNOUN()/DataArea/BODNOUN()/BODNOUN()Header/TotalAmount	Property Tag	TotalAmount
30	BODVERB()BODNOUN()/DataArea/BODNOUN()/BODNOUN()Header/DocumentID/ID	Literal	

For more information about the construction of BODs and BOD tags, see the online help.

3. Save the new elements.

Mapping the XML Attributes

The AccountingEntity and TotalAmountCurrency values are XML attributes in the BOD, so they must be mapped as such. To map these attributes:

1. On the **Replication Document Elements** form, select the **TotalAmount** element (**Sequence = 20**).
2. Click **Attributes**.
3. On the **Replication Document Attributes** form, add a new attribute using these values:
   - **Attribute** – **currencyID**
   - **Value Type** – **Property Tag**
   - **Property Name** – **TotalAmountCurrency**
4. Save the attribute and close the **Replication Document Attributes** form.
5. On the **Replication Document Elements** form, select the **DocumentID/ID** element (**Sequence = 30**).
6. Click **Attributes**.
7. On the **Replication Document Attibutes** form, add a new attribute using these values:
   - **Attribute** – **accountingEntity**
   - **Value Type** – **Property Tag**
   - **Property Name** – **AccountingEntity**
8. Save the attribute and close the **Replication Document Attributes** form.

Mapping the Line Properties

Once the header properties are all mapped, the next step is to map the line properties. There can be 0-to-*n* number of lines in a SyncSalesOrder BOD. To represent that in a replication document, a subcollection IDO property is associated with the repeating BOD element's XPath.

To map the line properties:

1. On the **Replication Document Elements** form, add a new element using these values:

Sequence	BOD Tag Name	Value Type	Property Name
40	BODVERB()BODNOUN()/DataArea/ BODNOUN()/BODNOUN()Line	Property Tag	Lines

2. Save the element.

3. Map each line property in the same manner that the header properties were mapped.

   The only difference is that the line property names must be prefixed with the **Lines** subcollection property name.

   To map these line properties, add new elements using these values:

Sequence	BOD Tag Name	Value Type	Property Name
50	BODVERB()BODNOUN()/DataArea/ BODNOUN()/BODNOUN()Line/ LineNumber	Property Tag	Lines.LineNumber
60	BODVERB()BODNOUN()/DataArea/ BODNOUN()/BODNOUN()Line/Item/ ItemID/ID	Property Tag	Lines.ItemID
70	BODVERB()BODNOUN()/DataArea/ BODNOUN()/BODNOUN()Line/ PromisedDeliveryDateTime	Property Tag	Lines.PromisedDelivery DateTime
80	BODVERB()BODNOUN()/DataArea/ BODNOUN()/BODNOUN()Line/Quantity	Property Tag	Lines.QuantityValue

4. Save the new elements.
5. Close the **Replication Document Elements** form.

Processing Inbound BODs using IDO Methods

In order to process (or persist) the data in inbound BODs, Mongoose Framework currently requires the replication document to specify custom update methods on each IDO. The methods can be either pass-through calls to stored procedures or methods in an IDO extension class which is in a .NET assembly stored with and loaded from the IDO metadata. This tutorial uses the latter technique.

Creating the Visual Studio Project

We will use Visual Studio 2010 to create a new class library named **DemoExt**. This tutorial uses C#, but the library can be created using any .NET language.

To create the Visual Studio project:

1. Open Visual Studio 2010, if you do not already have it open.
2. On the Visual Studio **Start Page**, click **New Project**.
3. In the **New Project** dialog box navigation panel (on the left), select **Visual C# > Windows**.
4. From the list of templates, select **Class Library**.

5. In the **Name** field, specify **DemoExt**.
6. Accept the default **Location**, or specify another location for the project.
7. Click **OK**.
8. In the Solution Explorer (typically on the right side of the screen), right-click **DemoExt** and, from the context menu, select **Add Reference**.

   Visual Studio displays the **Add Reference** dialog box.
9. Select the **Browse** tab and navigate to the folder in which the Mongoose Framework is installed.
10. From the list of files and folders displayed, CTRL+click to select these files:
    - IDOCore.dll
    - IDOProtocol.dll
    - MGShared.dll
    - WSEnums.dll
    - WSFormServerProtocol.dll
11. Click **OK**.

    Visual Studio adds the selected DLLs to the list of references.
12. Close and delete the Class1.cs file created by Visual Studio.

Creating the SalesOrder Extension Class

Next we must create two extension classes, to provide methods for overriding the normal save action. For more information on custom overrides, see the topic, "Middleware Update/Load Override Dialog Box" in the WinStudio online help.

To create the SalesOrder extension class:

1. In your **DemoExt** project, add a new class:
   a. In the Solution Explorer, right-click **DemoExt** and, from the context menu, select **Add > Class**.
   b. In the **Add New Item** dialog box, select **Class**.
   c. In the **Name** field, specify **SalesOrders**.
   d. Click **Add**.

      Visual Studio adds the new class to your project and opens a window for editing.
2. Edit the class code to match this:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
```

```
using Mongoose.IDO;
using Mongoose.IDO.Protocol;
using Mongoose.Core.Common;

namespace DemoExt
{
  /// <summary>
  /// IDO extension class for the SalesOrders IDO
  ///
  /// The class and method attributes are optional.
  /// </summary>
  [IDOExtensionClass("SalesOrders")]
  public class SalesOrders : ExtensionClassBase
  {
    [IDOMethod(MessageParm = "message", Flags =
MethodFlags.RequiresTransaction)]
    public int ProcessSyncSalesOrder(
     string actionCode,
     string orderId,
     string customerId,
     string acctEntity,
     decimal totalAmount,
     string totalAmtCurrency,
     out string message)
    {
      int result = (int)StdMethodResult.Success;

      message = string.Empty;

      try
      {
        IDOUpdateItem salesOrder = new IDOUpdateItem() { Action =
UpdateAction.None };

        if (actionCode == "Add")
          salesOrder.Action = UpdateAction.Insert;
        else if (actionCode == "Change")
        {
          salesOrder.Action = UpdateAction.Update;
          salesOrder.UseOptimisticLocking = false;
        }

        // see below for "Delete"
        if (salesOrder.Action != UpdateAction.None)
```

```
            {
              UpdateCollectionRequestData update =
               new UpdateCollectionRequestData("SalesOrders");

              // Key fields must not be marked "modified" for updates,
              // only for inserts.
              salesOrder.Properties.Add(
               "SalesOrderID",
               orderId,
               salesOrder.Action == UpdateAction.Insert);

              salesOrder.Properties.Add("CustomerID", customerId);
              salesOrder.Properties.Add("AccountingEntity", acctEntity);
              salesOrder.Properties.Add("TotalAmount", totalAmount);
              salesOrder.Properties.Add("TotalAmountCurrency",
totalAmtCurrency);

              update.Items.Add(salesOrder);

              IDORuntime.LogUserMessage(
               "DemoExt.SalesOrders",
               UserDefinedMessageType.UserDefined0,
               "Executing action {0} for Sales Order {1}",
               actionCode, orderId);

              this.Context.Commands.UpdateCollection(update);
            }
            else if (actionCode == "Delete")
            {
              // This shows bypassing the IDOs and
              // executing database commands directly
              using (var db = this.CreateAppDB())
              {
                db.ExecuteNonQuery(
                 string.Format(
                  "DELETE SalesOrderLines WHERE SalesOrderID = {0}",
                  SqlLiteral.Format(orderId)));

                db.ExecuteNonQuery(
                 string.Format(
                  "DELETE SalesOrder WHERE SalesOrderID = {0}",
                  SqlLiteral.Format(orderId)));
              }
            }
          }
```

```
        catch (Exception ex)
        {
          message = MGException.ExtractMessages(ex);
          result = (int)StdMethodResult.MinError;

          IDORuntime.LogUserMessage(
           "DemoExt.SalesOrders",
           UserDefinedMessageType.UserDefined0,
           "Exception executing action {0} for Sales Order {1}: {2}",
           actionCode, orderId, message);
        }
        return result;
      }
    }
  }
```

Creating the SalesOrderLines Extension Class

To create the SalesOrderLines extension class:

1. In your **DemoExt** project, add a new class:
   a. In the Solution Explorer, right-click **DemoExt** and, from the context menu, select **Add > Class**.
   b. In the **Add New Item** dialog box, select **Class**.
   c. In the **Name** field, specify **SalesOrderLines**.
   d. Click **Add**.

      Visual Studio adds the new class to your project and opens a window for editing.
2. Edit the class code to match this:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

using Mongoose.IDO;
using Mongoose.IDO.Protocol;

namespace DemoExt
{
  /// <summary>
  /// IDO extension class for the SalesOrderLines IDO
  ///
```

```
    /// The class and method attributes are optional.
    /// </summary>
    [IDOExtensionClass("SalesOrderLines")]
    public class SalesOrderLines : ExtensionClassBase
    {
      [IDOMethod(MessageParm = "message", Flags =
MethodFlags.RequiresTransaction)]
      public int ProcessSyncSalesOrderLine(
       string actionCode,
       string orderId,
       string lineNumber,
       string itemId,
       decimal quantity,
       string uom,
       DateTime promisedDeliveryDate,
       out string message)
      {
        int result = (int)StdMethodResult.Success;

        message = string.Empty;

        try
        {
          IDOUpdateItem salesOrderLine = new IDOUpdateItem() { Action =
UpdateAction.None };

          if (actionCode == "Add")
            salesOrderLine.Action = UpdateAction.Insert;
          else if (actionCode == "Change")
          {
            salesOrderLine.Action = UpdateAction.Update;
            salesOrderLine.UseOptimisticLocking = false;
          }

          // Delete is handled by the SalesOrders IDO.

          if (salesOrderLine.Action != UpdateAction.None)
          {
            UpdateCollectionRequestData update =
             new UpdateCollectionRequestData("SalesOrderLines");

            // Key fields must not be marked "modified" for updates,
              // only for inserts
            salesOrderLine.Properties.Add(
             "SalesOrderID",
```

```
                orderId,
                salesOrderLine.Action == UpdateAction.Insert);

               salesOrderLine.Properties.Add(
                "LineNumber",
                lineNumber,
                salesOrderLine.Action == UpdateAction.Insert);

               salesOrderLine.Properties.Add("ItemID", itemId);
               salesOrderLine.Properties.Add("QuantityValue", quantity);
               salesOrderLine.Properties.Add("QuantityUOM", uom);
               salesOrderLine.Properties.Add(
                "PromisedDeliveryDateTime",
                promisedDeliveryDate);

               update.Items.Add(salesOrderLine);

               this.Context.Commands.UpdateCollection(update);
             }
           }
           catch (Exception ex)
           {
             message = ex.Message;
             result = (int)StdMethodResult.MinError;
           }

           return result;
         }
       }
     }
```

Creating and Assigning the Custom Assembly

Once these extension classes have been created, we must create the custom assembly from them and import the custom assembly into the objects database. We can then create an association between the IDO and the extension class by specifying the extension class namespace and class name in the IDO editor. The IDO then uses the association at run-time to look up the assembly and create the extension class for each IDO.

To create and assign a custom assembly:

1. In your Mongoose Framework application, open the **IDO Custom Assemblies** form.
2. Create a new custom assembly with the following **Assembly Name: DemoExt**
3. Click **Import Assembly**.

4. Browse for the DemoExt.dll assembly you created in the previous section. Double-click to import it into your custom assembly.
5. Click **Import Symbols**.
6. Browse for the DemoExt.pdb file and double-click to import it into your custom assembly.

   This file should be located in the same folder as DemoExt.dll.
7. Save the custom assembly.
8. Click **Check In** and follow the prompts to check in the custom assembly to your source control system.
9. Open the **IDO Projects** form and select your project.
10. Click **IDOs**.
11. In the **IDOs** form, select the **SalesOrders** IDO and specify these values:
    - **Custom Assembly Name** – **DemoExt**
    - **Ext Class Name** – **SalesOrders**
    - **Ext Class Namespace** – **DemoExt**
12. Select the SalesOrderLines IDO and specify these values:
    - **Custom Assembly Name** – **DemoExt**
    - **Ext Class Name** – **SalesOrderLines**
    - **Ext Class Namespace** – **DemoExt**
13. Save your changes.
14. For each IDO, click **Check In** and follow the prompts to check in the IDO to your source control system.

Mapping the Custom Methods

To be able to use the IDOs, each IDO must have a custom method mapped to it. To do this:

1. Open the **IDO Projects** form, if it is not already open.
2. In the left-side grid, select your IDO project.
3. Click **IDOs**.
4. On the **IDOs** form, select the **SalesOrders** IDO and then click **New Method**.
5. On the **New Method** form, specify these values:
    - **Method Type** – **Handcoded - Standard Method**
    - **Method Name** – **ProcessSyncSalesOrder**
    - **Transactional** check box – **Selected**
6. Click **OK**.

7. On the **IDOs** form, select the **SalesOrderLines** IDO and then click **New Method**.
8. On the **New Method** form, specify these values:
   - **Method Type** – **Handcoded - Standard Method**
   - **Method Name** – **ProcessSyncSalesOrderLine**
   - **Transactional** check box – **Selected**
9. Click **OK**.
10. Save the changes to your IDOs and close the **IDOs** form.

TIP: To verify that these methods have been added, you can use the **Methods** button for each IDO in turn. Your new methods should appear on the linked **IDO Methods** form.

Calling the Extension Class Methods for Inbound BODs

In this last stage of development, we need to direct the framework to call the extension class methods for inbound SyncSalesOrder BODs. Notice that the first parameter in each of the extension class methods is **actionCode**. This corresponds to the BOD's action code. There are several ways to map the action code to those parameters. One option is to create an unbound property and map it to the **actionCode** attribute. But for this tutorial, the action code will be extracted from the BOD by identifying it as a special parameter value using the **P()** keyword.

Extracting the actionCode from the BOD

To set up the document to extract the actionCode value from the inbound BOD:

1. On the **Replication Documents** form, select the **SalesOrder** BOD.
2. Click **Elements**.
3. Add a new element using these values:

Sequence	BOD Tag Name	Value Type
5	BODVERB()BODNOUN()/DataArea/BODVERB()/ActionCriteria/ActionExpression	Literal

Note: Note the **Sequence** number here. When creating the mapping between the BOD and the extension class parameters, it is important to order the parameter elements so that they match the order and hierarchy of the XML BOD. Order, however, is not important for properties that are at the same level in the hierarchy. So, it does not matter whether **actionCode** comes before or after the mapping for CustomerID or TotalAmount. We chose 5 because **actionCode** is at the top, but it could have been 15 or 16 or 25.

4. Save the new element.
5. With the new element selected, click **Attributes**.
6. Add a new attribute using these values:
   - **Attribute** – **actionCode**
   - **Value Type** – **Literal**
   - **Value Expression** – **P(P1)**
7. Save the new attribute.

   For inbound BODs, the value expression **P(P1)** allows the custom save method to reference BOD elements and attributes without the need to create a property on the IDO (see the next step). This is helpful for non-persistent data such as the actionCode.
8. Return to the **Replication Documents** form and verify that the **SalesOrder** document is still selected.
9. In the **Ins Upd Del Overrides** field, specify the save method with this:

   UPD(ProcessSyncSalesOrder(P1,BODIDKEY(),CustomerID,AccountingEntity,TotalAmount,TotalAmountCurrency,MESSAGE))

   This results in the ProcessSyncSalesOrder method getting called with parameters set by the BOD-IDO property mappings. Note that the first parameter is **P1**, which corresponds to the **actionCode** that was previously defined. Also, note the use of the BODIDKEY() keyword. This evaluates to the document key value embedded in the BODID element of the inbound BOD (ApplicationArea/Sender/BODID). If we assume this BODID:

   ```
   <BODID>infor-nid:infor.ln:ae180:dep180:SL1029434:?SalesOrder&verb=Sync
   </BODID>
   ```

   Then the BODIDKEY() evaluates to **SL12029434**.
10. Save the document.

Adding Metadata for Persisting Lines

The metadata required for persisting lines must be added to the **Lines** element (**Sequence** = 40). To do this:

1. On the **Replication Documents** form, verify that the **SalesOrder** document is still selected.
2. Click **Elements**.
3. On the **Replication Document Elements** form, select the **Lines** element (**Sequence** = 40).
4. Select the **Subcollection** tab.
5. In the **Ins Upd Del Overrides** field, specify the save method with this:

   UPD(ProcessSyncSalesOrderLine(P1,BODIDKEY(),LineNumber,ItemID,QuantityValue,QuantityUOM,PromisedDeliveryDateTime,MESSAGE))

This results in the ProcessSyncSalesOrderLine method getting called with parameters set by the BOD-IDO property mappings for each SalesOrderLine in the BOD.

Setting Up the Inbound BOD Cross Reference

For the inbound BOD to be able to associate with your replication document, you must set up an inbound BOD cross reference. To do this:

1. On the **Replication Documents** form, verify that the **SalesOrder** document is still selected.
2. Click **Inbound Cross-Refs**.
3. On the **Replication Document Inbound Cross-References** form, press CTRL+N (if the form is not already displaying that it is ready to create a new cross reference).
4. For the new cross reference, specify these values:
   - **BOD Noun** – **SalesOrder**
   - **BOD Verb** – **Sync**
5. Save the cross reference and close the **Replication Document Inbound Cross-References** form.

Testing

When developing inbound BOD support, we recommend that if an instance of the Infor Framework Inbound Bus Service is monitoring your application database, the service be stopped. Testing should be performed using the BUS Inbox Service UI (BusInboxHost.exe). This application should be located in the same folder where you have installed the Infor Mongoose Framework. For the processes to configure and run this application, see “Modifying the BusInboxHost.exe.Config File” on page 98.

Before Testing

Before you begin testing, there are a few other things you must set up.

Installing the XML Files to Simulate Inbound BODs

For the purposes of this tutorial, there are three XML files that you can use as inbound BODs. If you downloaded this guide from the www.infor.com/inforxtreme Web site, these XML files should have been included as part of the zip file:

- *SalesOrderAdd.xml* – This file simulates an inbound BOD that adds a sales order to the SalesOrder table. Once you use this in testing, if you attempt to use it again before you use the SalesOrderDelete.xml file, the test fails.
- *SalesOrderChange.xml* – This file simulates an inbound BOD that updates an existing sales order. You cannot use this file until you have successfully used the SalesOrderAdd.xml file. If you attempt to use this file before using the SalesOrderAdd.xml file, the test fails.
- *SalesOrderDelete.xml* – This file simulates an inbound BOD that deletes the sales order added with the SalesOrderAdd.xml file. You can use this file only after a SalesOrderAdd.xml or SalesOrderChange.xml file has been tested successfully.

If you do not have these files, you can create them using the code supplied in “Tutorial XML Files” on page 100.

Place these files in a location where you will be able to find them when you are ready to test. We recommend using a "Temporary" or "Tutorial" folder.

Modifying the BusInboxHost.exe.Config File

This modification is optional but recommended, because it makes iterative testing easier.
If you use your SyteLine user ID and password for this step, you will not be required to check in the IDOs after every change.

The BusInboxHost.exe.Config file is located in the same directory where the BUS Inbox Service utility (BusInboxHost.exe) is installed. To modify this file:

1. Locate the BusInboxHost.exe.Config file and open it with the XML editor of your choice.
2. Modify the code inside the ***<BusInboxHost.Properties.Settings> </BusInboxHost.Properties.Settings>*** tags that addresses user credentials:

```
<setting name="SessionUser" serializeAs="String">
   <value>userID</value>
</setting>
<setting name="SessionPassword" serializeAs="String">
   <value>userPwd</value>
</setting>
```

   where:
   - *userID* is the user’s SyteLine login ID (user name).
   - *userPwd* is the user’s login password.
3. Save and close the BusInboxHost.exe.Config file.

To start the Bus Inbox Service (BusInboxHost) process, then, double click the BusInboxHost.exe file.

Clearing the IDO Metadata Caches

Finally, before attempting to run your first test, you must make sure that all existing IDO metadata for your configuration has been cleared from the cache. For thoroughness, we recommend that you clear the metadata from these locations:

- The IDO Runtime Development Server – Select your configuration and then, from the **Configuration** menu select **Discard IDO Metadata Cache**.
- The Configuration Manager – On the **Utilities** tab, select your configuration, and then verify that **IDO Runtime on local machine** is selected and click **Discard IDO Cache**.
- The BUS Inbox Service – Although it is not, strictly speaking, IDO metadata, it is a good idea to clear the cached metadata when you first start testing with this tool. To do so, click **Clear Cached Metadata**.

Doing the Testing

To test your inbound BOD support work:

1. Start the BUS Inbox Service application (BusInboxHost.exe), if it is not already running.
2. In the **Bootstrap Site** field (**Inbox** tab), specify the configuration in which you have been working.
3. To start the service, click **Start**.
4. Submit the sample SyncSalesOrder BOD:
   a. Select the **BOD Workbench** tab.
   b. Click **Open**.
   c. Navigate to the folder where you placed your sample BOD (XML) files and open SalesOrderAdd.xml.

      The **BOD Type** field displays **Sync.SalesOrder**, and the panel just below that displays the XML code for the BOD file.
   d. Verify that the **To** field displays **lid://*siteBusLogicalID***, where *siteBusLogicalID* is the location you noted earlier (see page 82).
   e. In the **From** field, specify **lid://default**.
   f. Click **Submit**.
5. Select the **Inbox** tab and verify that the BOD was successfully submitted.

   The **Status** column indicates whether the process **Completed** or **Failed**.

Repeat this test using the SalesOrderChange.xml and SalesOrderDelete.xml files.

Viewing and Evaluating the Test Results

After running the test, whether it is successful or not, you can view details about the test run. To do this:

1. On the **Inbox** tab, select the test run for which you want to view the details.
2. Click **Details**.

3. On the **Inbox Message Details** dialog box, select the tab corresponding to the details you want to view:
   - **BOD** – Displays a copy of the original BOD request as it was submitted. This should be the contents of the XML file you submitted.
   - **IDO Request** – Displays the actual IDO request that was submitted, assuming the test got that far.
   - **IDO Response** – Displays the system's response to the request. If the test completed, this is in XML format. If the test failed, this tab should display an error message explaining why the test failed.

At the bottom of the **Inbox** tab, there should also be a trace log that you can view for further information about the test.

Tutorial XML Files

If you downloaded this guide from the www.infor.com/inforxtreme Web site, these XML files should have been included as part of the zip file. If you acquired the guide from another source, however, or if you have lost those files, you can recreate them using the code in the following sections.

SalesOrderAdd.xml

To recreate this sample BOD, copy and paste this code into a blank file and save it as **SalesOrderAdd.xml**:

```
<?xml version="1.0" ?>
<SyncSalesOrder xmlns="http://schema.infor.com/InforOAGIS/2"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://schema.infor.com/InforOAGIS/2 http://
schema.infor.com/2.5.0/InforOAGIS/BODs/Developer/SyncSalesOrder.xsd"
xmlns:xsd="http://www.w3.org/2001/XMLSchema" releaseID="9.2"
versionID="2.5.0">
<ApplicationArea>
<Sender>
 <LogicalID>lid://infor.ln.nlbaudv1180</LogicalID>
 <ComponentID>erp</ComponentID>
 <ConfirmationCode>OnError</ConfirmationCode>
 </Sender>
 <CreationDateTime>2011-03-09T02:23:29Z</CreationDateTime>
 <BODID>infor-nid:infor.ln:ae180:dep180:SL1029434:?SalesOrder&verb=Sync</
BODID>
 </ApplicationArea>
<DataArea>
<Sync>
```

```
 <TenantID>infor.ln</TenantID>
 <AccountingEntityID>ae180</AccountingEntityID>
 <LocationID>dep180</LocationID>
<ActionCriteria>
 <ActionExpression actionCode="Add" />
 </ActionCriteria>
 </Sync>
<SalesOrder>
<SalesOrderHeader>
<DocumentID agencyRole="Supplier">
 <ID accountingEntity="ae180" location="dep180" lid="lid://
infor.ln.nlbaudv1180" variationID="20291">SL1029434</ID>
 </DocumentID>
<AlternateDocumentID>
 <ID />
 </AlternateDocumentID>
 <LastModificationDateTime>2011-03-09T02:23:29Z</
LastModificationDateTime>
 <DocumentDateTime>2011-03-09T02:23:00Z</DocumentDateTime>
 <Note type="Header">SALES ORDER WITH TWO LINES</Note>
 <Note type="Footer">SALES ORDER WITH TWO LINES</Note>
<Status>
 <Code>Approved</Code>
 <EffectiveDateTime>2011-03-09T02:23:00Z</EffectiveDateTime>
 <ArchiveIndicator>false</ArchiveIndicator>
 </Status>
<CustomerParty>
<PartyIDs>
 <ID accountingEntity="customer180" lid="lid://
infor.ln.nlbaudv1180">RGT000043</ID>
 </PartyIDs>
 <Name>BP For purchase puposes</Name>
<Location>
<Address>
 <FormatCode />
 <AttentionOfName>Millers & Co.</AttentionOfName>
 <AddressLine sequence="1" />
 <AddressLine sequence="2">De Dam 1</AddressLine>
 <AddressLine sequence="3" />
 <AddressLine sequence="4" />
 <AddressLine sequence="5" />
 <AddressLine sequence="6" />
 <CityName>Amsterdam</CityName>
 <CountrySubDivisionCode>NH</CountrySubDivisionCode>
 <CountryCode>NL</CountryCode>
```

```
 <PostalCode>1001 BA</PostalCode>
 </Address>
 </Location>
 </CustomerParty>
<SupplierParty>
<Location type="Office">
 <ID accountingEntity="location180" lid="lid://
infor.ln.nlbaudv1180">D_RGSLS1</ID>
 </Location>
 </SupplierParty>
<ShipToParty>
<PartyIDs>
 <ID accountingEntity="ship_to180" lid="lid://
infor.ln.nlbaudv1180">RGT000043</ID>
 </PartyIDs>
 <Name>BP For purchase puposes</Name>
<Location>
<Address>
 <FormatCode />
 <AttentionOfName>Millers & Co.</AttentionOfName>
 <AddressLine sequence="1" />
 <AddressLine sequence="2">De Dam 1</AddressLine>
 <AddressLine sequence="3" />
 <AddressLine sequence="4" />
 <AddressLine sequence="5" />
 <AddressLine sequence="6" />
 <CityName>Amsterdam</CityName>
 <CountrySubDivisionCode>NH</CountrySubDivisionCode>
 <CountryCode>NL</CountryCode>
 <PostalCode>1001 BA</PostalCode>
 </Address>
 </Location>
 </ShipToParty>
 <ExtendedAmount currencyID="EUR">875</ExtendedAmount>
<BillToParty>
<PartyIDs>
 <ID accountingEntity="bill_to180" lid="lid://
infor.ln.nlbaudv1180">RGT000043</ID>
 </PartyIDs>
 <Name>BP For purchase puposes</Name>
<Location>
<Address>
 <FormatCode />
 <AttentionOfName>Millers & Co.</AttentionOfName>
 <AddressLine sequence="1" />
```

```
 <AddressLine sequence="2">De Dam 1</AddressLine>
 <AddressLine sequence="3" />
 <AddressLine sequence="4" />
 <AddressLine sequence="5" />
 <AddressLine sequence="6" />
 <CityName>Amsterdam</CityName>
 <CountrySubDivisionCode>NH</CountrySubDivisionCode>
 <CountryCode>NL</CountryCode>
 <PostalCode>1001 BA</PostalCode>
 </Address>
 </Location>
 </BillToParty>
<CarrierParty>
<PartyIDs>
 <ID />
 </PartyIDs>
 </CarrierParty>
<PayFromParty>
<PartyIDs>
 <ID accountingEntity="pay_from180" lid="lid://
infor.ln.nlbaudv1180">RGT000043</ID>
 </PartyIDs>
 <Name>BP For purchase puposes</Name>
<Location>
<Address>
 <FormatCode />
 <AttentionOfName>Millers & Co.</AttentionOfName>
 <AddressLine sequence="1" />
 <AddressLine sequence="2">De Dam 1</AddressLine>
 <AddressLine sequence="3" />
 <AddressLine sequence="4" />
 <AddressLine sequence="5" />
 <AddressLine sequence="6" />
 <CityName>Amsterdam</CityName>
 <CountrySubDivisionCode>NH</CountrySubDivisionCode>
 <CountryCode>NL</CountryCode>
 <PostalCode>1001 BA</PostalCode>
 </Address>
 </Location>
 </PayFromParty>
 <PartialShipmentAllowedIndicator>true</PartialShipmentAllowedIndicator>
 <DropShipmentAllowedIndicator>false</DropShipmentAllowedIndicator>
<TransportationTerm>
<PlaceOfOwnershipTransferLocation>
 <Description />
```

```
 </PlaceOfOwnershipTransferLocation>
 </TransportationTerm>
<PaymentTerm>
 <Description>Nett 14 Days Payment Term</Description>
 </PaymentTerm>
 <RequestedShipDateTime>2011-03-09T02:23:00Z</RequestedShipDateTime>
 <PromisedShipDateTime>2011-03-09T02:23:00Z</PromisedShipDateTime>
 <PromisedDeliveryDateTime>2011-03-09T02:23:00Z</
PromisedDeliveryDateTime>
 <PaymentMethodCode />
 <BackOrderIndicator>true</BackOrderIndicator>
<UserArea>
<Property>
 <NameValue name="ln.RateDeterminer" type="StringType">Document Date</
NameValue>
 </Property>
<Property>
 <NameValue name="ln.Blocked" type="StringType">No</NameValue>
 </Property>
<Property>
 <NameValue name="ln.Canceled" type="StringType">No</NameValue>
 </Property>
<Property>
 <NameValue name="ln.OrderType" type="StringType">RGT</NameValue>
 </Property>
 </UserArea>
 <BillingTriggerCode>Shipment</BillingTriggerCode>
 <InvoiceImmediatelyIndicator>false</InvoiceImmediatelyIndicator>
 <PricingRequiredIndicator>false</PricingRequiredIndicator>
 <RushIndicator>false</RushIndicator>
 <SelfBillingIndicator>false</SelfBillingIndicator>
<Classification>
<Codes>
 <Code listID="Office" sequence="1">RGSLS1</Code>
 </Codes>
 </Classification>
 <ExtendedPretaxAmount currencyID="EUR">875</ExtendedPretaxAmount>
 <TotalAmount currencyID="EUR">875</TotalAmount>
 <SubTotalAmount currencyID="EUR">875</SubTotalAmount>
<BaseCurrencyAmount type="ExtendedAmount">
 <Amount currencyID="EUR">875</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="ExtendedPretaxAmount">
 <Amount currencyID="EUR">875</Amount>
 </BaseCurrencyAmount>
```

```
<BaseCurrencyAmount type="SubTotalAmount">
 <Amount currencyID="EUR">875</Amount>
 </BaseCurrencyAmount>
 </SalesOrderHeader>
<SalesOrderLine>
 <LineNumber>10</LineNumber>
 <Note />
<Status>
 <Code>Open</Code>
 <EffectiveDateTime>2011-03-09T02:23:30Z</EffectiveDateTime>
 <ArchiveIndicator>false</ArchiveIndicator>
 </Status>
<Item>
<ItemID>
 <ID accountingEntity="itemmaster180" lid="lid://
infor.ln.nlbaudv1180">RGT-TD-SLS003</ID>
 </ItemID>
 <Description>SLS Automatic test item 003</Description>
<SerializedLot>
<Lot>
<LotIDs>
 <ID />
 </LotIDs>
 <SerialNumber />
 </Lot>
 <LotSelection>Any</LotSelection>
 </SerializedLot>
 </Item>
 <Quantity unitCode="EA">4</Quantity>
<UnitPrice>
 <Amount currencyID="EUR">175</Amount>
 <PerQuantity unitCode="EA">1</PerQuantity>
 </UnitPrice>
 <ExtendedAmount currencyID="EUR">700</ExtendedAmount>
 <RequiredDeliveryDateTime>2011-03-10T02:23:00Z</
RequiredDeliveryDateTime>
<ShipToParty>
<PartyIDs>
 <ID accountingEntity="ship_to180" lid="lid://
infor.ln.nlbaudv1180">RGT000043</ID>
 </PartyIDs>
 <Name>BP For purchase puposes</Name>
<Location>
<Address>
 <FormatCode />
```

```
 <AttentionOfName>Millers & Co.</AttentionOfName>
 <AddressLine sequence="1" />
 <AddressLine sequence="2">De Dam 1</AddressLine>
 <AddressLine sequence="3" />
 <AddressLine sequence="4" />
 <AddressLine sequence="5" />
 <AddressLine sequence="6" />
 <CityName>Amsterdam</CityName>
 <CountrySubDivisionCode>NH</CountrySubDivisionCode>
 <CountryCode>NL</CountryCode>
 <PostalCode>1001 BA</PostalCode>
 </Address>
 </Location>
 </ShipToParty>
 <PartialShipmentAllowedIndicator>true</PartialShipmentAllowedIndicator>
 <DropShipmentAllowedIndicator>false</DropShipmentAllowedIndicator>
<TransportationTerm>
<PlaceOfOwnershipTransferLocation>
 <Description />
 </PlaceOfOwnershipTransferLocation>
 </TransportationTerm>
<PaymentTerm>
 <Description>Nett 14 Days Payment Term</Description>
 </PaymentTerm>
 <PromisedShipDateTime>2011-03-10T02:23:00Z</PromisedShipDateTime>
 <PromisedDeliveryDateTime>2011-03-10T02:23:00Z</
PromisedDeliveryDateTime>
 <BackOrderIndicator>true</BackOrderIndicator>
 <ScheduledDeliveryDateTime>2011-03-10T02:23:00Z</
ScheduledDeliveryDateTime>
<UserArea>
<Property>
 <NameValue name="ln.SalesType" type="StringType">TRADE</NameValue>
 </Property>
<Property>
 <NameValue name="ln.Blocked" type="StringType">No</NameValue>
 </Property>
<Property>
 <NameValue name="ln.Canceled" type="StringType">No</NameValue>
 </Property>
 </UserArea>
 <RushIndicator>false</RushIndicator>
 <SelfBillingIndicator>false</SelfBillingIndicator>
<CarrierParty>
<PartyIDs>
```

```
 <ID />
 </PartyIDs>
 </CarrierParty>
<ShipFromParty>
<Location type="Warehouse">
 <ID accountingEntity="location180" lid="lid://
infor.ln.nlbaudv1180">W_RGT</ID>
 </Location>
 </ShipFromParty>
 <FixedPriceItemIndicator>false</FixedPriceItemIndicator>
 <ShippedQuantity unitCode="EA">0</ShippedQuantity>
 <UnitQuantityConversionFactor>1</UnitQuantityConversionFactor>
<PricingAmount>
 <UnitBaseAmount currencyID="EUR">175</UnitBaseAmount>
 <ExtendedUnitBaseAmount currencyID="EUR">700</ExtendedUnitBaseAmount>
 <UnitPretaxAmount currencyID="EUR">175</UnitPretaxAmount>
 <TotalPretaxAmount currencyID="EUR">700</TotalPretaxAmount>
 <UnitTotalAmount currencyID="EUR">700</UnitTotalAmount>
 </PricingAmount>
<BaseCurrencyAmount type="ExtendedAmount">
 <Amount currencyID="EUR">700</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="UnitPriceAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountExtendedUnitBaseAmount">
 <Amount currencyID="EUR">700</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountTotalPretaxAmount">
 <Amount currencyID="EUR">700</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountUnitBaseAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountUnitPretaxAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountUnitTotalAmount">
 <Amount currencyID="EUR">700</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="ExtendedPretaxAmount">
 <Amount currencyID="EUR">700</Amount>
 </BaseCurrencyAmount>
 </SalesOrderLine>
<SalesOrderLine>
```

```
 <LineNumber>20</LineNumber>
 <Note />
<Status>
 <Code>Open</Code>
 <EffectiveDateTime>2011-03-09T02:23:30Z</EffectiveDateTime>
 <ArchiveIndicator>false</ArchiveIndicator>
 </Status>
<Item>
<ItemID>
 <ID accountingEntity="itemmaster180" lid="lid://
infor.ln.nlbaudv1180">RGT-TD-SLS004</ID>
 </ItemID>
 <Description>SLS Automatic test item 004</Description>
<SerializedLot>
<Lot>
<LotIDs>
 <ID />
 </LotIDs>
 <SerialNumber />
 </Lot>
 <LotSelection>Any</LotSelection>
 </SerializedLot>
 </Item>
 <Quantity unitCode="EA">2</Quantity>
<UnitPrice>
 <Amount currencyID="EUR">225</Amount>
 <PerQuantity unitCode="EA">1</PerQuantity>
 </UnitPrice>
 <ExtendedAmount currencyID="EUR">450</ExtendedAmount>
 <RequiredDeliveryDateTime>2011-03-10T02:23:00Z</
RequiredDeliveryDateTime>
<ShipToParty>
<PartyIDs>
 <ID accountingEntity="ship_to180" lid="lid://
infor.ln.nlbaudv1180">RGT000043</ID>
 </PartyIDs>
 <Name>BP For purchase puposes</Name>
<Location>
<Address>
 <FormatCode />
 <AttentionOfName>Millers & Co.</AttentionOfName>
 <AddressLine sequence="1" />
 <AddressLine sequence="2">De Dam 1</AddressLine>
 <AddressLine sequence="3" />
 <AddressLine sequence="4" />
```

```
 <AddressLine sequence="5" />
 <AddressLine sequence="6" />
 <CityName>Amsterdam</CityName>
 <CountrySubDivisionCode>NH</CountrySubDivisionCode>
 <CountryCode>NL</CountryCode>
 <PostalCode>1001 BA</PostalCode>
 </Address>
 </Location>
 </ShipToParty>
 <PartialShipmentAllowedIndicator>true</PartialShipmentAllowedIndicator>
 <DropShipmentAllowedIndicator>false</DropShipmentAllowedIndicator>
<TransportationTerm>
<PlaceOfOwnershipTransferLocation>
 <Description />
 </PlaceOfOwnershipTransferLocation>
 </TransportationTerm>
<PaymentTerm>
 <Description>Nett 14 Days Payment Term</Description>
 </PaymentTerm>
 <PromisedShipDateTime>2011-03-10T02:23:00Z</PromisedShipDateTime>
 <PromisedDeliveryDateTime>2011-03-10T02:23:00Z</
PromisedDeliveryDateTime>
 <BackOrderIndicator>true</BackOrderIndicator>
 <ScheduledDeliveryDateTime>2011-03-10T02:23:00Z</
ScheduledDeliveryDateTime>
<UserArea>
<Property>
 <NameValue name="ln.SalesType" type="StringType">TRADE</NameValue>
 </Property>
<Property>
 <NameValue name="ln.Blocked" type="StringType">No</NameValue>
 </Property>
<Property>
 <NameValue name="ln.Canceled" type="StringType">No</NameValue>
 </Property>
 </UserArea>
 <RushIndicator>false</RushIndicator>
 <SelfBillingIndicator>false</SelfBillingIndicator>
<CarrierParty>
<PartyIDs>
 <ID />
 </PartyIDs>
 </CarrierParty>
<ShipFromParty>
<Location type="Warehouse">
```

```
 <ID accountingEntity="location180" lid="lid://
infor.ln.nlbaudv1180">W_RGT</ID>
 </Location>
 </ShipFromParty>
 <FixedPriceItemIndicator>false</FixedPriceItemIndicator>
 <ShippedQuantity unitCode="EA">0</ShippedQuantity>
 <UnitQuantityConversionFactor>1</UnitQuantityConversionFactor>
<PricingAmount>
 <UnitBaseAmount currencyID="EUR">175</UnitBaseAmount>
 <ExtendedUnitBaseAmount currencyID="EUR">175</ExtendedUnitBaseAmount>
 <UnitPretaxAmount currencyID="EUR">175</UnitPretaxAmount>
 <TotalPretaxAmount currencyID="EUR">175</TotalPretaxAmount>
 <UnitTotalAmount currencyID="EUR">175</UnitTotalAmount>
 </PricingAmount>
<BaseCurrencyAmount type="ExtendedAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="UnitPriceAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountExtendedUnitBaseAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountTotalPretaxAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountUnitBaseAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountUnitPretaxAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountUnitTotalAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="ExtendedPretaxAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
 </SalesOrderLine>
 </SalesOrder>
 </DataArea>
 </SyncSalesOrder>
```

SalesOrderChange.xml

To recreate this sample BOD, copy and paste this code into a blank file and save it as **SalesOrderChange.xml**:

```
<?xml version="1.0" ?>
<SyncSalesOrder xmlns="http://schema.infor.com/InforOAGIS/2"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://schema.infor.com/InforOAGIS/2 http://
schema.infor.com/2.5.0/InforOAGIS/BODs/Developer/SyncSalesOrder.xsd"
xmlns:xsd="http://www.w3.org/2001/XMLSchema" releaseID="9.2"
versionID="2.5.0">
<ApplicationArea>
<Sender>
 <LogicalID>lid://infor.ln.nlbaudv1180</LogicalID>
 <ComponentID>erp</ComponentID>
 <ConfirmationCode>OnError</ConfirmationCode>
 </Sender>
 <CreationDateTime>2011-03-09T02:23:29Z</CreationDateTime>
 <BODID>infor-nid:infor.ln:ae180:dep180:SL1029434:?SalesOrder&verb=Sync</
BODID>
 </ApplicationArea>
<DataArea>
<Sync>
 <TenantID>infor.ln</TenantID>
 <AccountingEntityID>ae180</AccountingEntityID>
 <LocationID>dep180</LocationID>
<ActionCriteria>
 <ActionExpression actionCode="Change" />
 </ActionCriteria>
 </Sync>
<SalesOrder>
<SalesOrderHeader>
<DocumentID agencyRole="Supplier">
 <ID accountingEntity="ae180" location="dep180" lid="lid://
infor.ln.nlbaudv1180" variationID="20291">SL1029434</ID>
 </DocumentID>
<AlternateDocumentID>
 <ID />
 </AlternateDocumentID>
 <LastModificationDateTime>2011-03-09T02:23:29Z</
LastModificationDateTime>
 <DocumentDateTime>2011-03-09T02:23:00Z</DocumentDateTime>
 <Note type="Header">SALES ORDER WITH TWO LINES</Note>
 <Note type="Footer">SALES ORDER WITH TWO LINES</Note>
<Status>
 <Code>Approved</Code>
 <EffectiveDateTime>2011-03-09T02:23:00Z</EffectiveDateTime>
```

```
 <ArchiveIndicator>false</ArchiveIndicator>
 </Status>
<CustomerParty>
<PartyIDs>
 <ID accountingEntity="customer180" lid="lid://
infor.ln.nlbaudv1180">RGT000043</ID>
 </PartyIDs>
 <Name>BP For purchase puposes</Name>
<Location>
<Address>
 <FormatCode />
 <AttentionOfName>Millers & Co.</AttentionOfName>
 <AddressLine sequence="1" />
 <AddressLine sequence="2">De Dam 1</AddressLine>
 <AddressLine sequence="3" />
 <AddressLine sequence="4" />
 <AddressLine sequence="5" />
 <AddressLine sequence="6" />
 <CityName>Amsterdam</CityName>
 <CountrySubDivisionCode>NH</CountrySubDivisionCode>
 <CountryCode>NL</CountryCode>
 <PostalCode>1001 BA</PostalCode>
 </Address>
 </Location>
 </CustomerParty>
<SupplierParty>
<Location type="Office">
 <ID accountingEntity="location180" lid="lid://
infor.ln.nlbaudv1180">D_RGSLS1</ID>
 </Location>
 </SupplierParty>
<ShipToParty>
<PartyIDs>
 <ID accountingEntity="ship_to180" lid="lid://
infor.ln.nlbaudv1180">RGT000043</ID>
 </PartyIDs>
 <Name>BP For purchase puposes</Name>
<Location>
<Address>
 <FormatCode />
 <AttentionOfName>Millers & Co.</AttentionOfName>
 <AddressLine sequence="1" />
 <AddressLine sequence="2">De Dam 1</AddressLine>
 <AddressLine sequence="3" />
 <AddressLine sequence="4" />
```

```
 <AddressLine sequence="5" />
 <AddressLine sequence="6" />
 <CityName>Amsterdam</CityName>
 <CountrySubDivisionCode>NH</CountrySubDivisionCode>
 <CountryCode>NL</CountryCode>
 <PostalCode>1001 BA</PostalCode>
 </Address>
 </Location>
 </ShipToParty>
 <ExtendedAmount currencyID="EUR">875</ExtendedAmount>
<BillToParty>
<PartyIDs>
 <ID accountingEntity="bill_to180" lid="lid://
infor.ln.nlbaudv1180">RGT000043</ID>
 </PartyIDs>
 <Name>BP For purchase puposes</Name>
<Location>
<Address>
 <FormatCode />
 <AttentionOfName>Millers & Co.</AttentionOfName>
 <AddressLine sequence="1" />
 <AddressLine sequence="2">De Dam 1</AddressLine>
 <AddressLine sequence="3" />
 <AddressLine sequence="4" />
 <AddressLine sequence="5" />
 <AddressLine sequence="6" />
 <CityName>Amsterdam</CityName>
 <CountrySubDivisionCode>NH</CountrySubDivisionCode>
 <CountryCode>NL</CountryCode>
 <PostalCode>1001 BA</PostalCode>
 </Address>
 </Location>
 </BillToParty>
<CarrierParty>
<PartyIDs>
 <ID />
 </PartyIDs>
 </CarrierParty>
<PayFromParty>
<PartyIDs>
 <ID accountingEntity="pay_from180" lid="lid://
infor.ln.nlbaudv1180">RGT000043</ID>
 </PartyIDs>
 <Name>BP For purchase puposes</Name>
<Location>
```

```
<Address>
 <FormatCode />
 <AttentionOfName>Millers & Co.</AttentionOfName>
 <AddressLine sequence="1" />
 <AddressLine sequence="2">De Dam 1</AddressLine>
 <AddressLine sequence="3" />
 <AddressLine sequence="4" />
 <AddressLine sequence="5" />
 <AddressLine sequence="6" />
 <CityName>Amsterdam</CityName>
 <CountrySubDivisionCode>NH</CountrySubDivisionCode>
 <CountryCode>NL</CountryCode>
 <PostalCode>1001 BA</PostalCode>
 </Address>
 </Location>
 </PayFromParty>
 <PartialShipmentAllowedIndicator>true</PartialShipmentAllowedIndicator>
 <DropShipmentAllowedIndicator>false</DropShipmentAllowedIndicator>
<TransportationTerm>
<PlaceOfOwnershipTransferLocation>
 <Description />
 </PlaceOfOwnershipTransferLocation>
 </TransportationTerm>
<PaymentTerm>
 <Description>Nett 14 Days Payment Term</Description>
 </PaymentTerm>
 <RequestedShipDateTime>2011-03-09T02:23:00Z</RequestedShipDateTime>
 <PromisedShipDateTime>2011-03-09T02:23:00Z</PromisedShipDateTime>
 <PromisedDeliveryDateTime>2011-03-09T02:23:00Z</
PromisedDeliveryDateTime>
 <PaymentMethodCode />
 <BackOrderIndicator>true</BackOrderIndicator>
<UserArea>
<Property>
 <NameValue name="ln.RateDeterminer" type="StringType">Document Date</
NameValue>
 </Property>
<Property>
 <NameValue name="ln.Blocked" type="StringType">No</NameValue>
 </Property>
<Property>
 <NameValue name="ln.Canceled" type="StringType">No</NameValue>
 </Property>
<Property>
 <NameValue name="ln.OrderType" type="StringType">RGT</NameValue>
```

```
 </Property>
 </UserArea>
 <BillingTriggerCode>Shipment</BillingTriggerCode>
 <InvoiceImmediatelyIndicator>false</InvoiceImmediatelyIndicator>
 <PricingRequiredIndicator>false</PricingRequiredIndicator>
 <RushIndicator>false</RushIndicator>
 <SelfBillingIndicator>false</SelfBillingIndicator>
<Classification>
<Codes>
 <Code listID="Office" sequence="1">RGSLS1</Code>
 </Codes>
 </Classification>
 <ExtendedPretaxAmount currencyID="EUR">1000</ExtendedPretaxAmount>
 <TotalAmount currencyID="EUR">1000</TotalAmount>
 <SubTotalAmount currencyID="EUR">1000</SubTotalAmount>
<BaseCurrencyAmount type="ExtendedAmount">
 <Amount currencyID="EUR">1000</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="ExtendedPretaxAmount">
 <Amount currencyID="EUR">1000</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="SubTotalAmount">
 <Amount currencyID="EUR">1000</Amount>
 </BaseCurrencyAmount>
 </SalesOrderHeader>
<SalesOrderLine>
 <LineNumber>10</LineNumber>
 <Note />
<Status>
 <Code>Open</Code>
 <EffectiveDateTime>2011-03-09T02:23:30Z</EffectiveDateTime>
 <ArchiveIndicator>false</ArchiveIndicator>
 </Status>
<Item>
<ItemID>
 <ID accountingEntity="itemmaster180" lid="lid://
infor.ln.nlbaudv1180">RGT-TD-SLS003</ID>
 </ItemID>
 <Description>SLS Automatic test item 003</Description>
<SerializedLot>
<Lot>
<LotIDs>
 <ID />
 </LotIDs>
 <SerialNumber />
```

```
 </Lot>
 <LotSelection>Any</LotSelection>
 </SerializedLot>
 </Item>
 <Quantity unitCode="EA">5</Quantity>
<UnitPrice>
 <Amount currencyID="EUR">175</Amount>
 <PerQuantity unitCode="EA">1</PerQuantity>
 </UnitPrice>
 <ExtendedAmount currencyID="EUR">875</ExtendedAmount>
 <RequiredDeliveryDateTime>2011-03-10T02:23:00Z</
RequiredDeliveryDateTime>
<ShipToParty>
<PartyIDs>
 <ID accountingEntity="ship_to180" lid="lid://
infor.ln.nlbaudv1180">RGT000043</ID>
 </PartyIDs>
 <Name>BP For purchase puposes</Name>
<Location>
<Address>
 <FormatCode />
 <AttentionOfName>Millers & Co.</AttentionOfName>
 <AddressLine sequence="1" />
 <AddressLine sequence="2">De Dam 1</AddressLine>
 <AddressLine sequence="3" />
 <AddressLine sequence="4" />
 <AddressLine sequence="5" />
 <AddressLine sequence="6" />
 <CityName>Amsterdam</CityName>
 <CountrySubDivisionCode>NH</CountrySubDivisionCode>
 <CountryCode>NL</CountryCode>
 <PostalCode>1001 BA</PostalCode>
 </Address>
 </Location>
 </ShipToParty>
 <PartialShipmentAllowedIndicator>true</PartialShipmentAllowedIndicator>
 <DropShipmentAllowedIndicator>false</DropShipmentAllowedIndicator>
<TransportationTerm>
<PlaceOfOwnershipTransferLocation>
 <Description />
 </PlaceOfOwnershipTransferLocation>
 </TransportationTerm>
<PaymentTerm>
 <Description>Nett 14 Days Payment Term</Description>
 </PaymentTerm>
```

```
 <PromisedShipDateTime>2011-03-10T02:23:00Z</PromisedShipDateTime>
 <PromisedDeliveryDateTime>2011-04-03T02:23:00Z</
PromisedDeliveryDateTime>
 <BackOrderIndicator>true</BackOrderIndicator>
 <ScheduledDeliveryDateTime>2011-03-10T02:23:00Z</
ScheduledDeliveryDateTime>
<UserArea>
<Property>
 <NameValue name="ln.SalesType" type="StringType">TRADE</NameValue>
 </Property>
<Property>
 <NameValue name="ln.Blocked" type="StringType">No</NameValue>
 </Property>
<Property>
 <NameValue name="ln.Canceled" type="StringType">No</NameValue>
 </Property>
 </UserArea>
 <RushIndicator>false</RushIndicator>
 <SelfBillingIndicator>false</SelfBillingIndicator>
<CarrierParty>
<PartyIDs>
 <ID />
 </PartyIDs>
 </CarrierParty>
<ShipFromParty>
<Location type="Warehouse">
 <ID accountingEntity="location180" lid="lid://
infor.ln.nlbaudv1180">W_RGT</ID>
 </Location>
 </ShipFromParty>
 <FixedPriceItemIndicator>false</FixedPriceItemIndicator>
 <ShippedQuantity unitCode="EA">0</ShippedQuantity>
 <UnitQuantityConversionFactor>1</UnitQuantityConversionFactor>
<PricingAmount>
 <UnitBaseAmount currencyID="EUR">175</UnitBaseAmount>
 <ExtendedUnitBaseAmount currencyID="EUR">875</ExtendedUnitBaseAmount>
 <UnitPretaxAmount currencyID="EUR">175</UnitPretaxAmount>
 <TotalPretaxAmount currencyID="EUR">875</TotalPretaxAmount>
 <UnitTotalAmount currencyID="EUR">875</UnitTotalAmount>
 </PricingAmount>
<BaseCurrencyAmount type="ExtendedAmount">
 <Amount currencyID="EUR">875</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="UnitPriceAmount">
 <Amount currencyID="EUR">175</Amount>
```

```
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountExtendedUnitBaseAmount">
 <Amount currencyID="EUR">875</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountTotalPretaxAmount">
 <Amount currencyID="EUR">875</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountUnitBaseAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountUnitPretaxAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountUnitTotalAmount">
 <Amount currencyID="EUR">875</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="ExtendedPretaxAmount">
 <Amount currencyID="EUR">875</Amount>
 </BaseCurrencyAmount>
 </SalesOrderLine>
<SalesOrderLine>
 <LineNumber>20</LineNumber>
 <Note />
<Status>
 <Code>Open</Code>
 <EffectiveDateTime>2011-03-09T02:23:30Z</EffectiveDateTime>
 <ArchiveIndicator>false</ArchiveIndicator>
 </Status>
<Item>
<ItemID>
 <ID accountingEntity="itemmaster180" lid="lid://
infor.ln.nlbaudv1180">RGT-TD-SLS004</ID>
 </ItemID>
 <Description>SLS Automatic test item 004</Description>
<SerializedLot>
<Lot>
<LotIDs>
 <ID />
 </LotIDs>
 <SerialNumber />
 </Lot>
 <LotSelection>Any</LotSelection>
 </SerializedLot>
 </Item>
 <Quantity unitCode="EA">2</Quantity>
```

```
<UnitPrice>
 <Amount currencyID="EUR">225</Amount>
 <PerQuantity unitCode="EA">1</PerQuantity>
 </UnitPrice>
 <ExtendedAmount currencyID="EUR">450</ExtendedAmount>
 <RequiredDeliveryDateTime>2011-03-10T02:23:00Z</
RequiredDeliveryDateTime>
<ShipToParty>
<PartyIDs>
 <ID accountingEntity="ship_to180" lid="lid://
infor.ln.nlbaudv1180">RGT000043</ID>
 </PartyIDs>
 <Name>BP For purchase puposes</Name>
<Location>
<Address>
 <FormatCode />
 <AttentionOfName>Millers & Co.</AttentionOfName>
 <AddressLine sequence="1" />
 <AddressLine sequence="2">De Dam 1</AddressLine>
 <AddressLine sequence="3" />
 <AddressLine sequence="4" />
 <AddressLine sequence="5" />
 <AddressLine sequence="6" />
 <CityName>Amsterdam</CityName>
 <CountrySubDivisionCode>NH</CountrySubDivisionCode>
 <CountryCode>NL</CountryCode>
 <PostalCode>1001 BA</PostalCode>
 </Address>
 </Location>
 </ShipToParty>
 <PartialShipmentAllowedIndicator>true</PartialShipmentAllowedIndicator>
 <DropShipmentAllowedIndicator>false</DropShipmentAllowedIndicator>
<TransportationTerm>
<PlaceOfOwnershipTransferLocation>
 <Description />
 </PlaceOfOwnershipTransferLocation>
 </TransportationTerm>
<PaymentTerm>
 <Description>Nett 14 Days Payment Term</Description>
 </PaymentTerm>
 <PromisedShipDateTime>2011-03-10T02:23:00Z</PromisedShipDateTime>
 <PromisedDeliveryDateTime>2011-03-10T02:23:00Z</
PromisedDeliveryDateTime>
 <BackOrderIndicator>true</BackOrderIndicator>
 <ScheduledDeliveryDateTime>2011-03-10T02:23:00Z</
ScheduledDeliveryDateTime>
```

```
<UserArea>
<Property>
 <NameValue name="ln.SalesType" type="StringType">TRADE</NameValue>
 </Property>
<Property>
 <NameValue name="ln.Blocked" type="StringType">No</NameValue>
 </Property>
<Property>
 <NameValue name="ln.Canceled" type="StringType">No</NameValue>
 </Property>
 </UserArea>
 <RushIndicator>false</RushIndicator>
 <SelfBillingIndicator>false</SelfBillingIndicator>
<CarrierParty>
<PartyIDs>
 <ID />
 </PartyIDs>
 </CarrierParty>
<ShipFromParty>
<Location type="Warehouse">
 <ID accountingEntity="location180" lid="lid://
infor.ln.nlbaudv1180">W_RGT</ID>
 </Location>
 </ShipFromParty>
 <FixedPriceItemIndicator>false</FixedPriceItemIndicator>
 <ShippedQuantity unitCode="EA">0</ShippedQuantity>
 <UnitQuantityConversionFactor>1</UnitQuantityConversionFactor>
<PricingAmount>
 <UnitBaseAmount currencyID="EUR">175</UnitBaseAmount>
 <ExtendedUnitBaseAmount currencyID="EUR">175</ExtendedUnitBaseAmount>
 <UnitPretaxAmount currencyID="EUR">175</UnitPretaxAmount>
 <TotalPretaxAmount currencyID="EUR">175</TotalPretaxAmount>
 <UnitTotalAmount currencyID="EUR">175</UnitTotalAmount>
 </PricingAmount>
<BaseCurrencyAmount type="ExtendedAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="UnitPriceAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountExtendedUnitBaseAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountTotalPretaxAmount">
 <Amount currencyID="EUR">175</Amount>
```

```
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountUnitBaseAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountUnitPretaxAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountUnitTotalAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="ExtendedPretaxAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
 </SalesOrderLine>
 </SalesOrder>
 </DataArea>
 </SyncSalesOrder>
```

SalesOrderDelete.xml

To recreate this sample BOD, copy and paste this code into a blank file and save it as **SalesOrderDelete.xml**:

```
<?xml version="1.0" ?>
<SyncSalesOrder xmlns="http://schema.infor.com/InforOAGIS/2"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://schema.infor.com/InforOAGIS/2 http://
schema.infor.com/2.5.0/InforOAGIS/BODs/Developer/SyncSalesOrder.xsd"
xmlns:xsd="http://www.w3.org/2001/XMLSchema" releaseID="9.2"
versionID="2.5.0">
<ApplicationArea>
<Sender>
 <LogicalID>lid://infor.ln.nlbaudv1180</LogicalID>
 <ComponentID>erp</ComponentID>
 <ConfirmationCode>OnError</ConfirmationCode>
 </Sender>
 <CreationDateTime>2011-03-09T02:23:29Z</CreationDateTime>
 <BODID>infor-nid:infor.ln:ae180:dep180:SL1029434:?SalesOrder&verb=Sync</
BODID>
 </ApplicationArea>
<DataArea>
<Sync>
 <TenantID>infor.ln</TenantID>
 <AccountingEntityID>ae180</AccountingEntityID>
 <LocationID>dep180</LocationID>
```

```
<ActionCriteria>
 <ActionExpression actionCode="Delete" />
 </ActionCriteria>
 </Sync>
<SalesOrder>
<SalesOrderHeader>
<DocumentID agencyRole="Supplier">
 <ID accountingEntity="ae180" location="dep180" lid="lid://
infor.ln.nlbaudv1180" variationID="20291">SL1029434</ID>
 </DocumentID>
<AlternateDocumentID>
 <ID />
 </AlternateDocumentID>
 <LastModificationDateTime>2011-03-09T02:23:29Z</
LastModificationDateTime>
 <DocumentDateTime>2011-03-09T02:23:00Z</DocumentDateTime>
 <Note type="Header">SALES ORDER WITH TWO LINES</Note>
 <Note type="Footer">SALES ORDER WITH TWO LINES</Note>
<Status>
 <Code>Approved</Code>
 <EffectiveDateTime>2011-03-09T02:23:00Z</EffectiveDateTime>
 <ArchiveIndicator>false</ArchiveIndicator>
 </Status>
<CustomerParty>
<PartyIDs>
 <ID accountingEntity="customer180" lid="lid://
infor.ln.nlbaudv1180">RGT000043</ID>
 </PartyIDs>
 <Name>BP For purchase puposes</Name>
<Location>
<Address>
 <FormatCode />
 <AttentionOfName>Millers & Co.</AttentionOfName>
 <AddressLine sequence="1" />
 <AddressLine sequence="2">De Dam 1</AddressLine>
 <AddressLine sequence="3" />
 <AddressLine sequence="4" />
 <AddressLine sequence="5" />
 <AddressLine sequence="6" />
 <CityName>Amsterdam</CityName>
 <CountrySubDivisionCode>NH</CountrySubDivisionCode>
 <CountryCode>NL</CountryCode>
 <PostalCode>1001 BA</PostalCode>
 </Address>
 </Location>
```

```
 </CustomerParty>
<SupplierParty>
<Location type="Office">
 <ID accountingEntity="location180" lid="lid://
infor.ln.nlbaudv1180">D_RGSLS1</ID>
 </Location>
 </SupplierParty>
<ShipToParty>
<PartyIDs>
 <ID accountingEntity="ship_to180" lid="lid://
infor.ln.nlbaudv1180">RGT000043</ID>
 </PartyIDs>
 <Name>BP For purchase puposes</Name>
<Location>
<Address>
 <FormatCode />
 <AttentionOfName>Millers & Co.</AttentionOfName>
 <AddressLine sequence="1" />
 <AddressLine sequence="2">De Dam 1</AddressLine>
 <AddressLine sequence="3" />
 <AddressLine sequence="4" />
 <AddressLine sequence="5" />
 <AddressLine sequence="6" />
 <CityName>Amsterdam</CityName>
 <CountrySubDivisionCode>NH</CountrySubDivisionCode>
 <CountryCode>NL</CountryCode>
 <PostalCode>1001 BA</PostalCode>
 </Address>
 </Location>
 </ShipToParty>
 <ExtendedAmount currencyID="EUR">875</ExtendedAmount>
<BillToParty>
<PartyIDs>
 <ID accountingEntity="bill_to180" lid="lid://
infor.ln.nlbaudv1180">RGT000043</ID>
 </PartyIDs>
 <Name>BP For purchase puposes</Name>
<Location>
<Address>
 <FormatCode />
 <AttentionOfName>Millers & Co.</AttentionOfName>
 <AddressLine sequence="1" />
 <AddressLine sequence="2">De Dam 1</AddressLine>
 <AddressLine sequence="3" />
 <AddressLine sequence="4" />
```

```
 <AddressLine sequence="5" />
 <AddressLine sequence="6" />
 <CityName>Amsterdam</CityName>
 <CountrySubDivisionCode>NH</CountrySubDivisionCode>
 <CountryCode>NL</CountryCode>
 <PostalCode>1001 BA</PostalCode>
 </Address>
 </Location>
 </BillToParty>
<CarrierParty>
<PartyIDs>
 <ID />
 </PartyIDs>
 </CarrierParty>
<PayFromParty>
<PartyIDs>
 <ID accountingEntity="pay_from180" lid="lid://
infor.ln.nlbaudv1180">RGT000043</ID>
 </PartyIDs>
 <Name>BP For purchase puposes</Name>
<Location>
<Address>
 <FormatCode />
 <AttentionOfName>Millers & Co.</AttentionOfName>
 <AddressLine sequence="1" />
 <AddressLine sequence="2">De Dam 1</AddressLine>
 <AddressLine sequence="3" />
 <AddressLine sequence="4" />
 <AddressLine sequence="5" />
 <AddressLine sequence="6" />
 <CityName>Amsterdam</CityName>
 <CountrySubDivisionCode>NH</CountrySubDivisionCode>
 <CountryCode>NL</CountryCode>
 <PostalCode>1001 BA</PostalCode>
 </Address>
 </Location>
 </PayFromParty>
 <PartialShipmentAllowedIndicator>true</PartialShipmentAllowedIndicator>
 <DropShipmentAllowedIndicator>false</DropShipmentAllowedIndicator>
<TransportationTerm>
<PlaceOfOwnershipTransferLocation>
 <Description />
 </PlaceOfOwnershipTransferLocation>
 </TransportationTerm>
<PaymentTerm>
```

```
 <Description>Nett 14 Days Payment Term</Description>
 </PaymentTerm>
 <RequestedShipDateTime>2011-03-09T02:23:00Z</RequestedShipDateTime>
 <PromisedShipDateTime>2011-03-09T02:23:00Z</PromisedShipDateTime>
 <PromisedDeliveryDateTime>2011-03-09T02:23:00Z</
PromisedDeliveryDateTime>
 <PaymentMethodCode />
 <BackOrderIndicator>true</BackOrderIndicator>
<UserArea>
<Property>
 <NameValue name="ln.RateDeterminer" type="StringType">Document Date</
NameValue>
 </Property>
<Property>
 <NameValue name="ln.Blocked" type="StringType">No</NameValue>
 </Property>
<Property>
 <NameValue name="ln.Canceled" type="StringType">No</NameValue>
 </Property>
<Property>
 <NameValue name="ln.OrderType" type="StringType">RGT</NameValue>
 </Property>
 </UserArea>
 <BillingTriggerCode>Shipment</BillingTriggerCode>
 <InvoiceImmediatelyIndicator>false</InvoiceImmediatelyIndicator>
 <PricingRequiredIndicator>false</PricingRequiredIndicator>
 <RushIndicator>false</RushIndicator>
 <SelfBillingIndicator>false</SelfBillingIndicator>
<Classification>
<Codes>
 <Code listID="Office" sequence="1">RGSLS1</Code>
 </Codes>
 </Classification>
 <ExtendedPretaxAmount currencyID="EUR">875</ExtendedPretaxAmount>
 <TotalAmount currencyID="EUR">875</TotalAmount>
 <SubTotalAmount currencyID="EUR">875</SubTotalAmount>
<BaseCurrencyAmount type="ExtendedAmount">
 <Amount currencyID="EUR">875</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="ExtendedPretaxAmount">
 <Amount currencyID="EUR">875</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="SubTotalAmount">
 <Amount currencyID="EUR">875</Amount>
 </BaseCurrencyAmount>
```

```
 </SalesOrderHeader>
<SalesOrderLine>
 <LineNumber>10</LineNumber>
 <Note />
<Status>
 <Code>Open</Code>
 <EffectiveDateTime>2011-03-09T02:23:30Z</EffectiveDateTime>
 <ArchiveIndicator>false</ArchiveIndicator>
 </Status>
<Item>
<ItemID>
 <ID accountingEntity="itemmaster180" lid="lid://
infor.ln.nlbaudv1180">RGT-TD-SLS003</ID>
 </ItemID>
 <Description>SLS Automatic test item 003</Description>
<SerializedLot>
<Lot>
<LotIDs>
 <ID />
 </LotIDs>
 <SerialNumber />
 </Lot>
 <LotSelection>Any</LotSelection>
 </SerializedLot>
 </Item>
 <Quantity unitCode="EA">4</Quantity>
<UnitPrice>
 <Amount currencyID="EUR">175</Amount>
 <PerQuantity unitCode="EA">1</PerQuantity>
 </UnitPrice>
 <ExtendedAmount currencyID="EUR">700</ExtendedAmount>
 <RequiredDeliveryDateTime>2011-03-10T02:23:00Z</
RequiredDeliveryDateTime>
<ShipToParty>
<PartyIDs>
 <ID accountingEntity="ship_to180" lid="lid://
infor.ln.nlbaudv1180">RGT000043</ID>
 </PartyIDs>
 <Name>BP For purchase puposes</Name>
<Location>
<Address>
 <FormatCode />
 <AttentionOfName>Millers & Co.</AttentionOfName>
 <AddressLine sequence="1" />
 <AddressLine sequence="2">De Dam 1</AddressLine>
```

```
 <AddressLine sequence="3" />
 <AddressLine sequence="4" />
 <AddressLine sequence="5" />
 <AddressLine sequence="6" />
 <CityName>Amsterdam</CityName>
 <CountrySubDivisionCode>NH</CountrySubDivisionCode>
 <CountryCode>NL</CountryCode>
 <PostalCode>1001 BA</PostalCode>
 </Address>
 </Location>
 </ShipToParty>
 <PartialShipmentAllowedIndicator>true</PartialShipmentAllowedIndicator>
 <DropShipmentAllowedIndicator>false</DropShipmentAllowedIndicator>
<TransportationTerm>
<PlaceOfOwnershipTransferLocation>
 <Description />
 </PlaceOfOwnershipTransferLocation>
 </TransportationTerm>
<PaymentTerm>
 <Description>Nett 14 Days Payment Term</Description>
 </PaymentTerm>
 <PromisedShipDateTime>2011-03-10T02:23:00Z</PromisedShipDateTime>
 <PromisedDeliveryDateTime>2011-03-10T02:23:00Z</
PromisedDeliveryDateTime>
 <BackOrderIndicator>true</BackOrderIndicator>
 <ScheduledDeliveryDateTime>2011-03-10T02:23:00Z</
ScheduledDeliveryDateTime>
<UserArea>
<Property>
 <NameValue name="ln.SalesType" type="StringType">TRADE</NameValue>
 </Property>
<Property>
 <NameValue name="ln.Blocked" type="StringType">No</NameValue>
 </Property>
<Property>
 <NameValue name="ln.Canceled" type="StringType">No</NameValue>
 </Property>
 </UserArea>
 <RushIndicator>false</RushIndicator>
 <SelfBillingIndicator>false</SelfBillingIndicator>
<CarrierParty>
<PartyIDs>
 <ID />
 </PartyIDs>
 </CarrierParty>
```

```
<ShipFromParty>
<Location type="Warehouse">
 <ID accountingEntity="location180" lid="lid://
infor.ln.nlbaudv1180">W_RGT</ID>
 </Location>
 </ShipFromParty>
 <FixedPriceItemIndicator>false</FixedPriceItemIndicator>
 <ShippedQuantity unitCode="EA">0</ShippedQuantity>
 <UnitQuantityConversionFactor>1</UnitQuantityConversionFactor>
<PricingAmount>
 <UnitBaseAmount currencyID="EUR">175</UnitBaseAmount>
 <ExtendedUnitBaseAmount currencyID="EUR">700</ExtendedUnitBaseAmount>
 <UnitPretaxAmount currencyID="EUR">175</UnitPretaxAmount>
 <TotalPretaxAmount currencyID="EUR">700</TotalPretaxAmount>
 <UnitTotalAmount currencyID="EUR">700</UnitTotalAmount>
 </PricingAmount>
<BaseCurrencyAmount type="ExtendedAmount">
 <Amount currencyID="EUR">700</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="UnitPriceAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountExtendedUnitBaseAmount">
 <Amount currencyID="EUR">700</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountTotalPretaxAmount">
 <Amount currencyID="EUR">700</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountUnitBaseAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountUnitPretaxAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountUnitTotalAmount">
 <Amount currencyID="EUR">700</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="ExtendedPretaxAmount">
 <Amount currencyID="EUR">700</Amount>
 </BaseCurrencyAmount>
 </SalesOrderLine>
<SalesOrderLine>
 <LineNumber>20</LineNumber>
 <Note />
<Status>
```

```
 <Code>Open</Code>
 <EffectiveDateTime>2011-03-09T02:23:30Z</EffectiveDateTime>
 <ArchiveIndicator>false</ArchiveIndicator>
 </Status>
<Item>
<ItemID>
 <ID accountingEntity="itemmaster180" lid="lid://
infor.ln.nlbaudv1180">RGT-TD-SLS004</ID>
 </ItemID>
 <Description>SLS Automatic test item 004</Description>
<SerializedLot>
<Lot>
<LotIDs>
 <ID />
 </LotIDs>
 <SerialNumber />
 </Lot>
 <LotSelection>Any</LotSelection>
 </SerializedLot>
 </Item>
 <Quantity unitCode="EA">2</Quantity>
<UnitPrice>
 <Amount currencyID="EUR">225</Amount>
 <PerQuantity unitCode="EA">1</PerQuantity>
 </UnitPrice>
 <ExtendedAmount currencyID="EUR">450</ExtendedAmount>
 <RequiredDeliveryDateTime>2011-03-10T02:23:00Z</
RequiredDeliveryDateTime>
<ShipToParty>
<PartyIDs>
 <ID accountingEntity="ship_to180" lid="lid://
infor.ln.nlbaudv1180">RGT000043</ID>
 </PartyIDs>
 <Name>BP For purchase puposes</Name>
<Location>
<Address>
 <FormatCode />
 <AttentionOfName>Millers & Co.</AttentionOfName>
 <AddressLine sequence="1" />
 <AddressLine sequence="2">De Dam 1</AddressLine>
 <AddressLine sequence="3" />
 <AddressLine sequence="4" />
 <AddressLine sequence="5" />
 <AddressLine sequence="6" />
 <CityName>Amsterdam</CityName>
```

```
 <CountrySubDivisionCode>NH</CountrySubDivisionCode>
 <CountryCode>NL</CountryCode>
 <PostalCode>1001 BA</PostalCode>
 </Address>
 </Location>
 </ShipToParty>
 <PartialShipmentAllowedIndicator>true</PartialShipmentAllowedIndicator>
 <DropShipmentAllowedIndicator>false</DropShipmentAllowedIndicator>
<TransportationTerm>
<PlaceOfOwnershipTransferLocation>
 <Description />
 </PlaceOfOwnershipTransferLocation>
 </TransportationTerm>
<PaymentTerm>
 <Description>Nett 14 Days Payment Term</Description>
 </PaymentTerm>
 <PromisedShipDateTime>2011-03-10T02:23:00Z</PromisedShipDateTime>
 <PromisedDeliveryDateTime>2011-03-10T02:23:00Z</
PromisedDeliveryDateTime>
 <BackOrderIndicator>true</BackOrderIndicator>
 <ScheduledDeliveryDateTime>2011-03-10T02:23:00Z</
ScheduledDeliveryDateTime>
<UserArea>
<Property>
 <NameValue name="ln.SalesType" type="StringType">TRADE</NameValue>
 </Property>
<Property>
 <NameValue name="ln.Blocked" type="StringType">No</NameValue>
 </Property>
<Property>
 <NameValue name="ln.Canceled" type="StringType">No</NameValue>
 </Property>
 </UserArea>
 <RushIndicator>false</RushIndicator>
 <SelfBillingIndicator>false</SelfBillingIndicator>
<CarrierParty>
<PartyIDs>
 <ID />
 </PartyIDs>
 </CarrierParty>
<ShipFromParty>
<Location type="Warehouse">
 <ID accountingEntity="location180" lid="lid://
infor.ln.nlbaudv1180">W_RGT</ID>
 </Location>
```

```
 </ShipFromParty>
 <FixedPriceItemIndicator>false</FixedPriceItemIndicator>
 <ShippedQuantity unitCode="EA">0</ShippedQuantity>
 <UnitQuantityConversionFactor>1</UnitQuantityConversionFactor>
<PricingAmount>
 <UnitBaseAmount currencyID="EUR">175</UnitBaseAmount>
 <ExtendedUnitBaseAmount currencyID="EUR">175</ExtendedUnitBaseAmount>
 <UnitPretaxAmount currencyID="EUR">175</UnitPretaxAmount>
 <TotalPretaxAmount currencyID="EUR">175</TotalPretaxAmount>
 <UnitTotalAmount currencyID="EUR">175</UnitTotalAmount>
 </PricingAmount>
<BaseCurrencyAmount type="ExtendedAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="UnitPriceAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountExtendedUnitBaseAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountTotalPretaxAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountUnitBaseAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountUnitPretaxAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="PricingAmountUnitTotalAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
<BaseCurrencyAmount type="ExtendedPretaxAmount">
 <Amount currencyID="EUR">175</Amount>
 </BaseCurrencyAmount>
 </SalesOrderLine>
 </SalesOrder>
 </DataArea>
 </SyncSalesOrder>
```

Appendix B: Troubleshooting

B

Handling Replication Errors

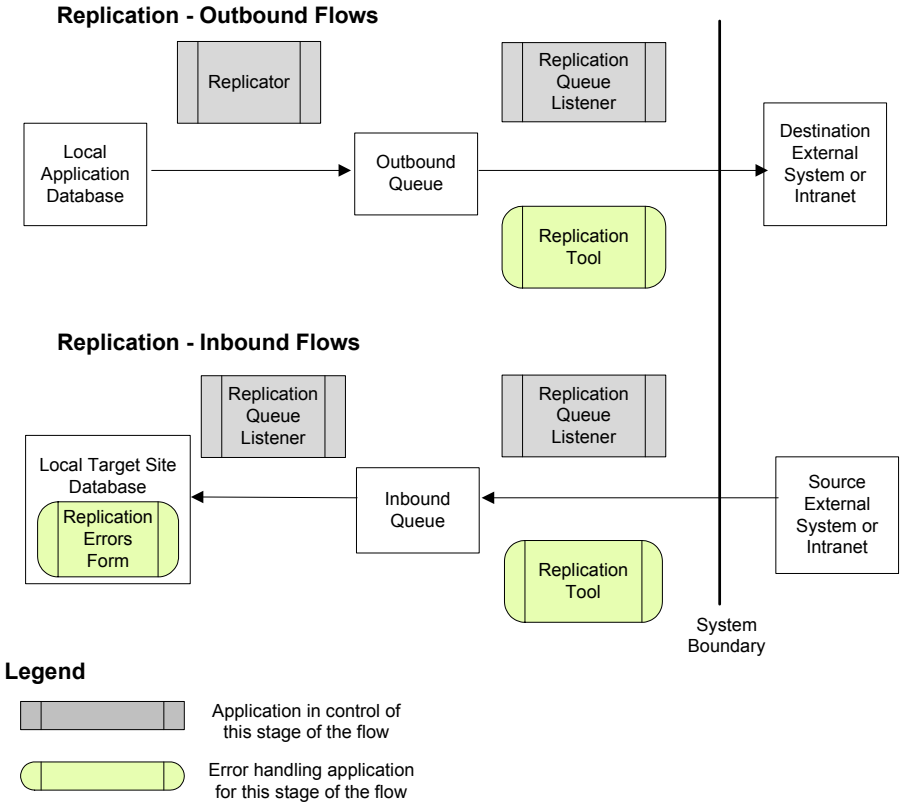
Errors During Transactional Replication

Validation errors relating to transactional replication generally display during data entry or saving of transaction records at the local site, and do not allow the user to continue.

However, if transactional replication appears to not be working (that is, the data isn't being moved to the target site as expected) and the cause is not immediately apparent, use the "Replication Troubleshooting Checklist" on page 134 to help find the problem.

Errors During Non-Transactional Replication

If errors occur when sending or receiving XML request documents through the replication system, the following processes handle the errors.



Replication Troubleshooting Checklist

Problem: Replication does not seem to be working. For example, a newly added item is not available from a drop-down menu in another, replicating site.

Solution: View the appropriate _all table(s) using SQL Server tools, to determine if data is being populated. For our example problem above, view the item_all table. If replication is not working, the table typically will not contain the newly entered data records. In this case, there probably is a problem with the replication rules.

Check the following areas:

1. **Configuration Manager utility** - Verify, for each site and entity, that you have created at least one configuration name that exactly matches the site/entity ID, including case, and that uses the site/entity's application database.
2. **Replication Rules form** (page 29) – Make sure the rules are set up properly between the site where the data was added and the one where it was supposed to appear.

a. Are the proper categories being replicated? Use the following table to see what types of results to expect on the target site for each replicated category.

Category	Expected results on target site (for forms with Site or Site Group fields)
A/P	• Posted A/P vouchers, adjustments and payments from source sites are visible on view forms. • A/P reports contain data from source sites. • _all tables contain account, currency, item, journal, period, vendor and tax information from source sites.
A/R	• Posted A/R invoices, debit memos, credit memos and payments from source sites are visible on view forms. • A/P reports contain data from source sites. • _all tables contain account, currency, item, journal, period, customer and tax information from source sites.
Centralized Order Entry	• Customer orders with lines that have a local ship site are visible. • Transfer orders to or from the local site are visible. • _all tables contain customer, customer order, account, currency, item, job, period, purchase order, project, transfer order and tax information from source sites.
ESB	Data is received in other Infor On-Ramp enabled application; acknowledgement data is received in SyteLine.
EXTFIN	Not typically used between SyteLine sites.
EXTFIN Customer	Not typically used between SyteLine sites.
EXTFIN Vendor	Not typically used between SyteLine sites.
G/L	• Financial and G/L reports and views show data from source sites. • _all tables contain ledger, account, currency, period, and journal information from source sites.
Inventory/ Transfers	• Transfer orders to or from the local site are visible. • _all tables contain account, currency, item, job, period, and transfer order information from source sites.
Journal Builder	• Pending journal transactions created by Journal Builder in source site appear in the local site. • Journal Builder site can validate data from the local site.
Ledger Consolidation	• Ledger records are consolidated from source sites when invoked. • _all tables contain account, currency, and period information from source sites.
Ledger Detail	• Ability to drill down into G/L records from remote sites if transaction data is also replicated. • _all tables contain account, material transaction and voucher pre-register information from source sites.

Category	Expected results on target site (for forms with Site or Site Group fields)
Manufacturer Item	_all tables contain manufacturer and manufacturer item information and cross-reference information between manufacturer item and SyteLine item records from source sites.
Multi-Site CRM	On the **Salesperson Home** form, information from source sites is visible.
Multi-Site Customers	On the **Multi-Site Customers** form, customer information defined on source sites is visible.
Multi-Site Items	On the **Multi-Site Customers** form, item information defined on source sites is visible.
Planning	• Planning is invoked for source sites when specified. • Planning views include requirements and receipts for source sites.
PO - CO Across Sites	The CO at the source site has fields that were populated by the PO that was created at the demanding site.
Purchase Order Builder	• POs are created in the local sites with data entered at the source site through the **Purchase Order Builder**. • **Purchase Order Builder** site can validate data from the local site.
Shared Currency	Currency information from source sites is visible.
Site Admin	• Site/Intranet information from source sites is visible. • _all tables contain account information from source sites.
Voucher Builder	• Vouchers and adjustments are created in the local sites with data entered at the source site through the **Voucher Builder**. • Vouchers are created in the local sites with data entered at the source site through the **Manual Voucher Builder**. • **Voucher Builder** site, or **Manual Voucher Builder** site, can validate data from the local site.

View the **Replication Rules** form in the source site (for example, if you're expecting to see CAL data in ONT, view the CAL database), and determine if there is a rule set up with **Target Site** equal to the site where you expect to see the data, and with **Category** equal to the category as determined above.

b. If such a record *does not exist*, this means the behavior was expected. You can create a new rule if desired, and retest to verify that the data was passed over.

c. If the record *exists*, check its **Interval Type**. If the **Interval Type** is:

- **Transactional**, there might be another problem with the replication setup. Continue through the other steps in this check list.
- **Immediate**, there might be a problem with the setup of the replication services.
- Something other than **Transactional** or **Immediate**, wait until the time specified in **Start Interval At** before testing again. If the interval time has passed and the change has still

not occurred on the target machine, there might be a problem with the setup of the replication services.

3. **Sites** form (page 26) - Make sure the database name and the linked servers (for transactional replication) or user maps (for non-transactional replication) are set up correctly.
4. **Replication Management** form (page 31) – You *must* click the **Regenerate Replication Triggers** button on this form after any replication rules have been changed. Failure to do so will result in replication not working properly.
5. **Find Replication Setup Issues** form (page 142) – Use this form to find problems relating to the setup of replication from the current site to another site/entity for a particular database table or stored procedure.

   Not all problems found by this utility are valid in all situations. For example, it checks to see if a user name is set up between the sites—which applies only if non-transactional replication is being used.
6. **Replication Errors** form (page 141) – This form shows the data that failed to submit to a target site during non-transactional replication. Use the form to fix and resubmit data, or delete it.
7. **Replication Tool** (page 141) – This tool on the utility server is used to troubleshoot non-transactional replication.
8. ReplicatedRows3 and ShadowValues tables – When something is to be replicated through *non-transactional* replication, it gets copied to the ReplicatedRows3 and ShadowValues tables. (See Chapter 4, “Replication Behind the Scenes," for details.) If the data is not there, it is likely that either your replication rules aren’t set properly or that you need to click the **Regenerate Replication Triggers** button on the **Replication Management** form.
9. **Services** – If you are using *non-transactional* replication, make sure the Infor Framework Replication Queue Listener and Replicator services are started on the utility server.
10. **MSMQ** or **Message Queuing** – Turn on journaling.
11. Check the replication logs in the Log Monitor.

    These are useful for non-transactional replication.
12. **Event Viewer** – Check the event viewer in the Windows Administrative Tools on the utility server or database server. The events that are logged may give you a clue about where replication failed.
13. **UETs** – If you are using user extended tables, and the UET information is not replicating properly, see the information about UETs and _all tables in Chapter 6, “Setting Up Custom Replication."

Problem: System performance is slow and you suspect replication might be causing this.

Solution: Check the following:

1. Use the stored procedure sp_spaceused (in the Master database) to determine which tables consume a lot of space.

2. If there are _all tables (that is, replicated data) that contain very large numbers of records, then make sure you are not replicating unnecessary data. See Chapter 3, “Replication Categories," particularly the information about replicating financial data.

   Also look at the descriptions of how the tables in each category are used. If your large _all tables are used only for reports that you rarely or never require, you may be able to remove that object from the category.

Problem: Replication failure due to SQL Linked Server error. If your system returns an error like one of these, the local server probably no longer has a link to itself:

- Server ’*servername*' is not configured for Data Access. Distributed Transaction Completed. Either enlist this session in a new transaction or the NULL transaction.
- New transaction cannot enlist in the specified transaction coordinator.
- The operation could not be performed because the OLE provider 'SQLOLEDB' was unable to begin a distributed transaction.

Solution: To fix this problem, run the following SQL commands on the application database:

```
exec sp_addserver servername, LOCAL
-- restart SQL Server.
EXEC sp_serveroption 'servername', 'Data Access', 'True'
```

Replace *servername* with the name of the local server.

Problem: Invalid Column name UF_xxxx. If your system returns an error message similar to this during replication, you may have user extended tables (UETs) set up in one site but not in another.

Solution: You must have identical schemas in all sites and entities to replicate data. If your system includes User Extended Tables (UETs), follow the steps under “Copying UET Table Schema Changes” on page 74.

Problem: Outbound BODs are not being sent from SyteLine to another system.

Solution: Follow these steps to determine the cause:

1. Does the BOD appear in the Replication Document Outbox?

   BODs are saved in the outbox after processing only if the BOD service is configured to not delete them.

   If **Yes**, continue to the next step.

   If **No**, then do this:

   a. If this is a multi-site system, verify that you are checking the inbox on the "bootstrap" site; that is, the site defined in the configuration designated as the bootstrap in the Service Configuration Manager utility.

   b. From the utility server, run the Log Monitor utility.

   c. In SyteLine, perform the processing that should send the BOD. (For a list of the application events that generate the BOD, see the **Replication Document Outbound Cross References** form.)

   d. In the Log Monitor utility, look for ReplQListener log information that might indicate an error in the BOD processing.

2. Is the **Processed** check box selected?

   If **Yes**, the BOD was picked up by Infor10 ION. Check **Infor CSI: Errors > History** to find failed message transmissions and information about why the failure occurred.

   If **No**, then there is a problem that is preventing ION from picking up the message; either ION is not running or the configuration and deployment of the sending/receiving applications is not set up properly.

Problem: Inbound BODs are not being received or processed by SyteLine.

Solution: Follow these steps to determine the cause:

1. Does the BOD appear in the Replication Document Inbox?

   (If there are many documents in the inbox, it may be difficult to find the BOD because currently you can’t filter by BOD name.)

   If **Yes**, continue to the next step.

   If **No**, then something prevented it from being delivered by Infor10 ION:

   - If this is a multi-site system, verify that you are looking at the inbox on the site designated in the **Inbound Bus Configuration** field of the Service Configuration Manager.
   - Check the BOD service logs to find failed message transmissions and information about why the failure occurred.
   - Check whether the ION service is running.
   - Check whether the configuration and deployment of the sending/receiving applications is set up properly. See the appropriate integration guide for more information.

2. Is the **Processed** check box selected?

If **Yes**, and the expected information does not appear in the appropriate form or table, then there was an error in processing. Check the Replication Document Outbox for a confirmation BOD. A confirmation BOD indicates an error. Check the following areas to determine the cause of the error:

- Windows application event viewer
- Log Monitor utility on the utility server, if it is running

If **No**, then either the Infor Framework Inbound Bus Service is not running, or it is still working through previous messages.

Problem: Error in Replication Document Manual Request Utility due to timeout.

If you receive either of the following errors during generation of BODs through the **Replication Document Manual Request** utility, then transactions are taking longer to run than the timeout values set in SyteLine or other related applications and services.

```
Exception from load collection: Error processing an IDO request
(Protocol=Http, URL=http://jokerswild/IDORequestService/
RequestService.aspx):
The remote server returned an error: (500) Internal Server Error.
```

OR

```
The transaction has aborted.
Transaction Timeout
```

Solution: For a list of timeout values that can be reset, see the chapter on improving performance in the *System Administration Guide* for your system.

Forms and Utilities Used to Troubleshoot Replication

Service Configuration Manager

Before you start using replication, you need to configure replication information in the Service Configuration Manager. To run this utility on the utility server, navigate to **Infor > Tools > Infor Service Configuration Manager** from the **Start** menu.

Instructions for specifying the replication information are found in the utility's online help. Clicking **Save** saves the replication information in an XML that is accessible by other framework utilities, including the Replication Tool (below).

Replication Tool (Inbound or Outbound Flow)

If errors happen while SyteLine replication is processing an inbound or outbound request, you can use the Replication Tool on the utility server to:

- View, correct, and resubmit inbound and outbound XML request documents;
  OR
- View the status of sites linked to this site for replication.

The Replication Tool can be used to view errors in any XML being sent from SyteLine to a different SyteLine intranet or an external system. To start this tool on the utility server, run ReplicationTool.exe in the root folder on the utility server (usually C:\Program Files\Infor\SyteLine). To limit the view to a single configuration, you can specify a configuration name on the command line, for example **ReplicationTool MI**.

Instructions for using the Replication Tool are found in its online help.

Replication Errors Form (Inbound Flow)

During non-transactional replication, if an XML document makes it into a SyteLine intranet, is retrieved from the inbound MSMQ by the Infor Framework Replication Queue Listener, but fails when executed against the target site, its errors are displayed in the **Replication Errors** form on the target site. Generally these documents contain valid login and target site information but are failing for another reason.

If an inbound XML document fails before that point, you can view and correct it through the Replication Tool described above.

Use the form to fix and resubmit the data. If the update succeeds this time, the data is removed from the ShadowValuesErrors table; if it fails again, it remains in the table.

The fields of interest are:

- **From Site** – Site where the record orginated.
- **Object Name/Object Type** – Name and type of record to be replicated (for example, co table).
- **Operation Number** – Distinguishes different elements in the errors. For example, an insert of an item record might use several lines of old and new values, all with the same Operation Number.
- **Operation Type** – Type of operation: Insert, Update, Delete, or Method Call.
- **Line** – Indicates which line of data for a particular record contained the error. The data is stored in rows of 55 name/value pairs, so for example, if there are 85 columns of data, there will be 2 lines.
- **Old/New** – Indicates whether a particular row of the collection represents the old or new values. For updates, the old values are also displayed.
- **Error** – Text of the error message received when this record was submitted.
- **Label** – Component label seen on forms.
- **Value** – Data for a particular column. For example, the column **Description** might have data "bike seat" as a value.

Find Replication Setup Issue

Use the **Find Replication Setup Issue** form to help troubleshoot why replication is not occurring from the current site to a specified remote site. You enter an object name and type (a table, stored procedure, or XML document) and select the remote site name. When you click the **List Issues** button, any setup errors for that site/object combination are listed.

Some of the setup problems that may be displayed include:

- The **From Site** or **To Site** does not exist in the site table.
- No replication category contains the specified object (table, SP, etc.).
- No replication rule exists for the specified **From Site** and **To Site**, and for any category that contains the specified object.
- A rule exists for a category using the specified object, but the rule is currently disabled on the **Replication Rules** form.
- Replication is currently disabled between the specified **From Site** and **To Site** on the **Sites** form.
- No replication trigger exists for the specified object (if it is a table), or the trigger is disabled.
- The **To Site** has not been set up with a linked server name on the **Sites** form or through the SyteLine Configuration Wizard. (Transactional replication).

  Caution: Transactional replication checks are made only if at least one transactional replication rule is enabled at this site.

- The **To Site** and **From Site** are on different database servers, and the **To Site**'s database server has no link to it from this site's database server. This is established through the Linked Servers node in SQL Server. (Transactional replication)
- The **To Site**'s database name is not specified on the **Sites** form. (Transactional replication)
- The user name specified for this **From Site** and **To Site** does not exist. (Non-transactional replication)
- The replication user name is not a super user or has not been given access to the proper license module and group. (Non-transactional replication)
- The **To Site** and/or **From Site** does not have an intranet defined on the **Sites** form. (Non-transactional replication)
- The **To Site**'s intranet has no URL defined on the **Intranets** form. This is a problem only if the **To Site** is on a different intranet than the **From Site**. (Non-transactional replication)
- No problems found. The replication triggers might need to be regenerated.

Index

S

T